



# Cyber Security Operations Course Pack

---

Training at Forrest.one · Five guides, one volume



*The Forrest.one bear mascot.*

## Contents

1. OSINT Field Guide — Open-Source Intelligence Guide
2. Malware Inspection & Reverse Engineering — Malware Analysis Guide
3. Digital Forensics & Disk Image Analysis — Digital Forensics Guide
4. Incident Response & Intrusion Analysis — Incident Response Guide
5. Building Your Own Security Lab — Lab Setup Guide

## Open-Source Intelligence Guide

### OSINT Field Guide

Training at Forrest.one · Open Source Intelligence collection, analysis and reporting

### How to use this guide

This is a working manual, not background reading. Every procedure in it was run against a live domain while the guide was written, and every picture is a real capture of the tool that produced it.

Work through it in order the first time. After that, use the tool reference table at the back and jump straight to what you need.

Two rules before anything else:



1. Write scope before you run a single tool. Every investigation gets its own folder and its own SCOPE.md.
2. Capture raw output to disk. Never retype or tidy a result. If a tool fails, the failure is part of the record.

## The legal and ethical boundary

Open source intelligence means information that is already public. This guide stays inside that line and so should you.

| You may                                        | You may not                                    |
|------------------------------------------------|------------------------------------------------|
| Query public records: DNS, WHOIS, certificates | Access anything behind a login                 |
| Read public pages and documents                | Attempt to bypass authentication               |
| Search public breach corpora already published | Buy or use private personal data               |
| Look at metadata in public files               | Use findings to harass or expose an individual |

Two further cautions that matter in practice:

- Some lookups are not silent. Registration checks such as holehe can trigger a password reset email. The subject may notice. Record that in the report rather than pretending the collection was invisible.
- Public still means personal. A home address found in a public record is still a home address. Handle it the way you would want yours handled.

## 1. Set up the collection lab

Everything runs inside one disposable container. The target never sees your own address and nothing you investigate touches your desktop.

### Where the tools run

Everything collects from one disposable container, never from your desktop.

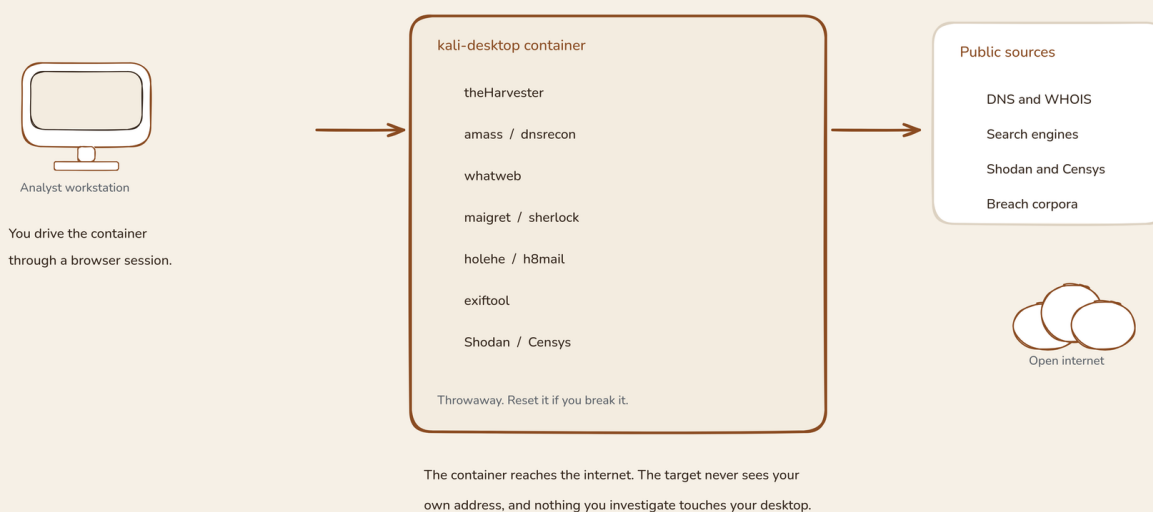


Figure 1: An analyst workstation driving a disposable collection container against public sources.



## Why a container, not your desktop

- Attribution. Lookups arrive from the container's address, not yours.
- Safety. You will download unknown documents and visit unknown pages. A container you can delete contains that.
- Repeatability. The same tools at the same versions every time.

## The environment facts that cost the most time

Four things break almost every first attempt. Check them before you blame a tool.

| Symptom                                                             | Cause                                             | Fix                                                        |
|---------------------------------------------------------------------|---------------------------------------------------|------------------------------------------------------------|
| command not found for maigret, sherlock, holehe, h8mail, socialscan | pipx installs are not on the non-interactive PATH | export PATH=\$PATH:/usr/local/bin inside the same bash -lc |
| List directory /var/lib/apt/lists/partial is missing                | apt run as a non-root user                        | add -u 0 to docker exec                                    |
| A tool reports a file missing that you just wrote                   | host and container see different filesystems      | docker cp the file in                                      |
| Output appears empty in your log                                    | the tool wrote to stderr                          | capture with 2>&1                                          |

## The wrapper every command uses

```
docker exec -u 0 kali-desktop bash -lc \  
'export PATH=$PATH:/usr/local/bin:/home/kasm-user/.local/bin; <command>'
```

-u 0 is required for anything touching apt. The PATH export is required for the pipx tools. Both together are the single most reliable command shape in this guide.

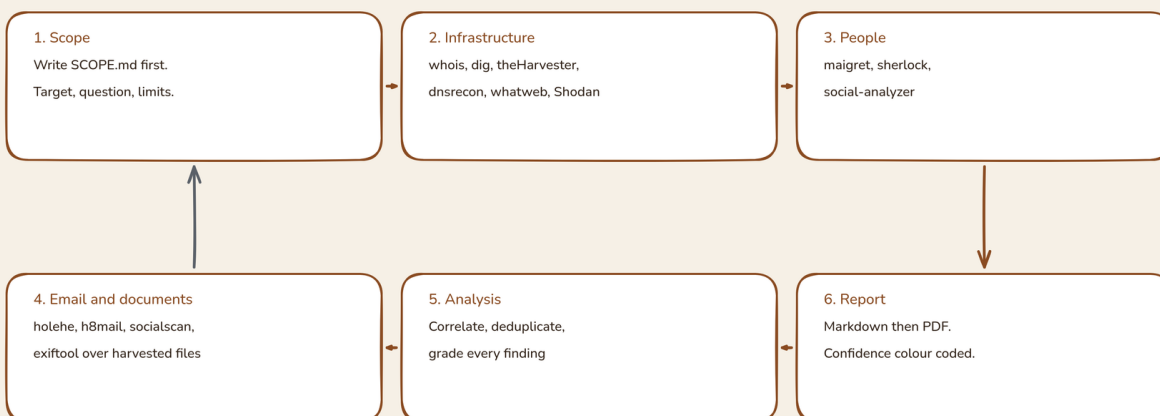
## 2. The workflow

Six phases. Infrastructure comes before people, because subdomains and mail hosts define the boundary for everything that follows.



## The OSINT workflow

Six phases. Every phase writes to a known path on disk.



Infrastructure comes before people: subdomains and mail

hosts define the boundary for every later lookup.

Absence of evidence is evidence, when the method and the source are both recorded.

Figure 2: The six phase OSINT workflow, from scope to report.

| Phase                  | Question it answers                                             | Order                |
|------------------------|-----------------------------------------------------------------|----------------------|
| 1. Scope               | What am I looking at, and what am I allowed to do?              | First, always        |
| 2. Infrastructure      | What is registered, routable and internet-facing?               | Then                 |
| 3. People              | Who works there, and where else do they exist?                  | After infrastructure |
| 4. Email and documents | What is the address format, is it breached, what do files leak? | After people         |
| 5. Analysis            | What corroborates, what is a coincidence, how confident am I?   | After collection     |
| 6. Report              | What does the reader need to know and act on?                   | Last                 |

Why this order. Employee names feed the email format. Email addresses feed breach lookups. Breach data feeds document discovery. Run people collection first and you will guess the email format instead of deriving it.

## 3. Phase 1 — Scope

Before any tool runs, write the scope file. It exists so the run is documented, not so it can be approved.

```
## SCOPE – case-<slug>

- Target: forrest.one (domain)
- Question: what is publicly exposed under this domain?
- Collection: passive only. No port scanning, no login probing.
- Authorisation basis: standing authorisation, own asset.
- Out of scope: personal accounts of third parties.
- Started: 2026-10-01T21:40Z
```

Four fields do the work: target, question, collection type, out of scope. An absent scope line is the most common reason a piece of collection has to be thrown away and redone.



## 4. Phase 2 — Infrastructure

### 4.1 WHOIS — who registered it, and when

WHOIS tells you the registrar, the creation and expiry dates, and the nameservers. On a privacy-protected domain it tells you that too, which is itself a finding.

```
docker exec -u 0 kali-desktop bash -lc 'whois forrest.one'
```

```
OSINT_SHOT
File Actions Edit View Help
$ whois forrest.one | head -26
Domain Name: forrest.one
Registry Domain ID: REDACTED FOR PRIVACY
Registrar WHOIS Server: whois.namecheap.com
Registrar URL: http://www.namecheap.com
Updated Date: 2026-09-05T03:56:15Z
Creation Date: 2019-04-02T15:19:02Z
Registry Expiry Date: 2027-04-02T15:19:02Z
Registrar: NameCheap, Inc.
Registrar IANA ID: 1068
Registrar Abuse Contact Email: abuse@namecheap.com
Registrar Abuse Contact Phone: +1.6613102107
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Registry Registrant ID: REDACTED FOR PRIVACY
Registrant Name: REDACTED FOR PRIVACY
Registrant Organization: Privacy service provided by WITHHELD FOR PRIVACY LLC
Registrant Street: REDACTED FOR PRIVACY
Registrant Street: REDACTED FOR PRIVACY
Registrant Street: REDACTED FOR PRIVACY
Registrant City: REDACTED FOR PRIVACY
Registrant State/Province: Delaware
Registrant Postal Code: REDACTED FOR PRIVACY
Registrant Country: US
Registrant Phone: REDACTED FOR PRIVACY
Registrant Phone Ext: REDACTED FOR PRIVACY
Registrant Fax: REDACTED FOR PRIVACY
Registrant Fax Ext: REDACTED FOR PRIVACY
— end of run —
```

Screenshot: whois output for forrest.one, showing the registrar, creation date and privacy service.

What the run above shows. Registration through NameCheap, created 2 April 2019, expiring 2 April 2027, with the registrant hidden behind a privacy service in Delaware, United States. Two years of paid-up registration on a privacy-protected record.

Read it like this:

| Field                   | What it tells you                                                   |
|-------------------------|---------------------------------------------------------------------|
| Creation Date           | Domain age. A three-week-old domain in a phishing case is a signal. |
| Registry Expiry Date    | Paid up or lapsing. Lapsing domains get dropped and re-registered.  |
| Registrar               | Where an abuse report would go.                                     |
| Name Server             | Where DNS actually lives. Cloudflare here means a proxy in front.   |
| Registrant Organization | A privacy service is a finding, not a dead end.                     |

Record the date of every WHOIS lookup. Public records change.

### 4.2 DNS — what exists and what routes it

Start broad, then narrow. ANY often returns nothing on modern nameservers; query the record types individually.

```
docker exec -u 0 kali-desktop bash -lc \  
  'dig forrest.one ANY +noall +answer; dig +short MX forrest.one'
```

```
$ dig +short MX forrest.one
10 mx2.privateemail.com.
10 mx1.privateemail.com.

$ dig +short A forrest.one
104.21.30.130
172.67.172.239
```



Screenshot: dig returning the mail exchanger records for the domain.

What the run above shows. Two mail exchangers, mx1.privateemail.com and mx2.privateemail.com, each at priority 10. No A record through ANY, which is normal: many resolvers refuse ANY queries now.

Why the MX records matter. They name the mail provider, which tells you where an address format would be validated, and they are a third party that may hold useful hosted content.

### 4.3 dnsrecon — the full record set in one pass

Where dig answers one question at a time, dnsrecon enumerates the zone.

```
docker exec -u 0 kali-desktop bash -lc 'dnsrecon -d forrest.one -t std'
```

```
$ dnsrecon -d forrest.one -t std
2026-10-01T22:01:18.616071+0000 INFO NS cleo.ns.cloudflare.com 173.245.59.89
2026-10-01T22:01:18.623541+0000 INFO NS cleo.ns.cloudflare.com 172.64.33.89
2026-10-01T22:01:18.630201+0000 INFO NS cleo.ns.cloudflare.com 2a06:98c1:50::ac40:2159
2026-10-01T22:01:18.630403+0000 INFO NS cleo.ns.cloudflare.com 2803:f800:50::6ca2:c159
2026-10-01T22:01:18.630547+0000 INFO NS cleo.ns.cloudflare.com 2606:4700:58::adf5:3b59
2026-10-01T22:01:18.630683+0000 INFO NS tina.ns.cloudflare.com 108.162.192.230
2026-10-01T22:01:18.637581+0000 INFO NS tina.ns.cloudflare.com 173.245.58.230
2026-10-01T22:01:18.644215+0000 INFO NS tina.ns.cloudflare.com 172.64.32.230
2026-10-01T22:01:18.650846+0000 INFO NS tina.ns.cloudflare.com 2606:4700:50::adf5:3ae6
2026-10-01T22:01:18.651033+0000 INFO NS tina.ns.cloudflare.com 2a06:98c1:50::ac40:20e6
2026-10-01T22:01:18.651173+0000 INFO NS tina.ns.cloudflare.com 2803:f800:50::6ca2:c0e6
2026-10-01T22:01:18.661132+0000 INFO MX mx1.privateemail.com 162.255.118.7
2026-10-01T22:01:18.661193+0000 INFO MX mx2.privateemail.com 162.255.118.8
2026-10-01T22:01:18.661945+0000 INFO A forrest.one 172.67.172.239
2026-10-01T22:01:18.661996+0000 INFO A forrest.one 104.21.30.130
2026-10-01T22:01:18.662025+0000 INFO AAAA forrest.one 2606:4700:3031::6815:1e82
2026-10-01T22:01:18.662050+0000 INFO AAAA forrest.one 2606:4700:3033::ac43:acef
2026-10-01T22:01:18.670644+0000 INFO TXT forrest.one v=spf1 include:spf.privateemail.com ~all
2026-10-01T22:01:18.679166+0000 INFO Enumerating SRV Records
2026-10-01T22:01:18.710231+0000 ERROR No SRV Records Found for forrest.one
2026-10-01T22:01:18.710369+0000 INFO Completed enumeration for domain: forrest.one
```

Screenshot: dnsrecon enumerating SOA, NS, MX, A, AAAA and TXT records.

What the run above shows. The complete public footprint in one view:

| Type     | Value                                          | Reading                                |
|----------|------------------------------------------------|----------------------------------------|
| SOA / NS | cleo.ns.cloudflare.com, tina.ns.cloudflare.com | Authoritative DNS at Cloudflare        |
| A        | 104.21.30.130, 172.67.172.239                  | Two proxy addresses, Cloudflare ranges |
| AAAA     | 2606:4700:3031::6815:1e82                      | IPv6 published                         |
| MX       | mx1 / mx2.privateemail.com                     | Mail hosted off-site                   |
| TXT      | v=spf1 include:spf.privateemail.com ~all       | SPF present and correctly scoped       |
| SRV      | none found                                     | No service discovery records published |

Two findings worth naming. The Recursion enabled on NS Server errors are not your problem: they describe the nameserver's configuration, not the target's. And an SPF record ending ~all is a soft fail, which a spoofing assessment should note.

Absence is a finding. "No SRV records found" is a real result when you state the method. Put it in the report.

### 4.4 theHarvester — hosts, emails and names from public sources

theHarvester queries search engines and certificate sources for anything tied to the domain.

```
docker exec -u 0 kali-desktop bash -lc \
'theHarvester -d forrest.one -b duckduckgo -l 300'
```



```
$ theHarvester -d forrest.one -b duckduckgo -l 300
* theHarvester 4.8.1
* Coded by Christian Martorella
* Edge-Security Research
* cmartorella@edge-security.com
*
*****
[*] Target: forrest.one
[*] Searching Duckduckgo.
[*] No IPs found.
[*] No emails found.
[*] No people found.
[*] No hosts found.
```

*Screenshot: theHarvester reporting no IPs, emails, people or hosts found.*

What the run above shows a negative result, and a clean one. The tool exited normally, queried DuckDuckGo, and found nothing of any kind.

Do not treat this as a failure. It is a genuine finding with two parts:

1. Nothing about this domain is indexed by that engine under that source.
2. The method and source are recorded, so the negative is reproducible.

A run log that hides a null result is worth less than one that states it. Report negatives with the source named: "theHarvester with the DuckDuckGo source found no hosts for forrest.one on 1 October 2026."

If you need a positive result to demonstrate the workflow, run against a domain you control that has indexed content, or add sources with `-b` `bing,duckduckgo,otx,crtsh` and compare. Each source is a separate claim in the report.

## 4.5 whatweb — what the site is actually running

```
docker exec -u 0 kali-desktop bash -lc 'whatweb -a 1 https://forrest.one'
```

```
$ whatweb -a 1 https://forrest.one
https://forrest.one [404 Not Found] Country[RESERVED][22], HTTPServer[cloudflare], IP[172.67.172.239], Script, Title[404 Not Found], UncommonHeaders[x-robots-tag,nel,report-to,cf-cache-status,speculation-rules,cf-ray,alt-svc]
```

*Screenshot: whatweb identifying Cloudflare, the HTTP status and response headers.*

What the run above shows. A 404 at the root, served by Cloudflare, from 104.21.30.130, in the United States. Response headers include `x-robots-tag`, `nel`, `report-to`, `cf-cache-status`, `speculation-rules`, `cf-ray` and `alt-svc`.

Read the headers, not just the fingerprint. `cf-ray` confirms Cloudflare termination. `x-robots-tag` is a machine-readable instruction about indexing and is worth reading in full. The `alt-svc` header advertises alternative protocols.

Set the aggression level deliberately. `-a 1` is a single polite request. Higher levels send more requests and are more **likely** to be logged or blocked.

## 4.6 Search operators — finding documents engines already indexed

Search engines have crawled far more of a target than you ever will by hand. Operators narrow that index to what you want.



## Search operators that find documents

Search engines index far more than you will ever crawl.

|                    |                                  |
|--------------------|----------------------------------|
| site:example.com   | Restrict results to one domain.  |
| filetype:pdf       | Only files of one kind.          |
| intitle:"index of" | Match words in the page title.   |
| inurl:admin        | Match words in the URL.          |
| -inurl:www         | Exclude a term from results.     |
| cache:example.com  | The search engine's stored copy. |

Record the exact query string. It is part of the finding:  
a result you cannot reproduce is not evidence.

Figure 3: Search operators that find documents, with what each one does.

| Operator           | Finds                                   |
|--------------------|-----------------------------------------|
| site:example.com   | Everything indexed under one domain     |
| filetype:pdf       | Files of one kind                       |
| intitle:"index of" | Open directory listings                 |
| inurl:admin        | Administrative paths referenced in URLs |
| -inurl:www         | Excludes results carrying a term        |
| cache:example.com  | The engine's stored copy of a page      |

Always record the exact query string. It is part of the finding. A result you cannot reproduce is not evidence.

go-dork automates the query syntax:

```
docker exec -u 0 kali-desktop bash -lc \  
'go-dork -q "site:forrest.one" -e google'
```

## 5. Phase 3 — People and usernames

### 5.1 maigret — one username, every platform

maigret takes a single username and checks it across hundreds of sites, returning not just hits but the profile data each platform exposes.

```
docker exec -u 0 kali-desktop bash -lc \  
'maigret --timeout 5 --no-progressbar billyforrest'
```



```
OSINT_SHOT
File Actions Edit View Help
$ maigret --timeout 5 --no-progressbar billyforrest
- periscope_username: billyforrest
- fullname: Bill
- follower_count: 47
- following_count: 62
- hearts_count: 0
- is_beta_user: False
- is_employee: False
- is_verified: False
- is_twitter_verified: False
- twitter_uid: 29560271
- twitter_screen_name: billyforrest
- image: https://pbs.twimg.com/profile_images/947332310617047040/vSgQtD0-_reasonably_small.jpg
[+] Livejasmin: https://www.livejasmin.com/en/girls/#!/chat/billyforrest
[+] Erome: https://www.erome.com/billyforrest
[+] GitHubGist [GitHub]: https://gist.github.com/billyforrest
[!] Too many errors of type "Bot protection" (7.86%). Try to switch to another ip address
[-] You can see detailed site check errors with a flag '--print-errors'
[-] Extracted IDs: {'billyforrest': 'username', 'Billyforrest': 'username'}
[*] Short text report:
Search by username billyforrest returned 36 accounts.
Found target's other IDs: Billyforrest.
Extended info extracted from 19 accounts.
Countries: us, cn
Interests (tags): social, blog, streaming, video, apps, music, art, networking, messaging, sharing, porn, hobby, gaming, writing, discussion, business, coding, shopping, design, photo, tech, freelance, movies, review, forum, science, geosocial, links, finance, fintech, 3d, webcam
— end of run —
```

Screenshot: maigret reporting 36 accounts found for a username, with extracted identifiers.

What the run above shows. Thirty-six accounts for one username, extended information from nineteen of them, additional identifiers extracted, and a list of interests and tags.

This is the highest-yield step in the whole workflow, and the one to treat with most care. Three things to note:

- A username match is **Possible**, not **Confirmed**. People reuse usernames. Corroborate with a second attribute: a real name, a linked email, an avatar that appears elsewhere.
- Bot protection degrades results silently. The run logged Too many errors of type "Bot protection" (8.06%). That is a stated limitation of this run and belongs in the report.
- Extended profile data can include a linked social account and an avatar image URL. An avatar is a pivot: run it through a reverse image search.

## 5.2 sherlock — the fast check

sherlock covers fewer sites than maigret but returns faster. Use it for a quick existence check, then maigret for the detail.

```
docker exec -u 0 kali-desktop bash -lc 'sherlock billyforrest'
```

## 5.3 social-analyzer — a graded confidence view

social-analyzer returns a confidence percentage per match rather than a binary hit, which is useful when a platform's answer is ambiguous.

```
docker exec -u 0 kali-desktop bash -lc \
'social-analyzer --username billyforrest --metadata --output json'
```

# 6. Phase 4 — Email addresses and breach exposure

## 6.1 holehe — which services an address is registered with

```
docker exec -u 0 kali-desktop bash -lc 'holehe --only-used contact@forrest.one'
```



```
OSINT_SHOT
File Actions Edit View Help

*****
contact@forrest.one
*****

[+] Email used, [-] Email not used, [x] Rate limit
121 websites checked in 3.48 seconds
Twitter : @palenath
Github : https://github.com/megadose/holehe
For BTC Donations : 1FHDM49QfZX6pJmhjLE5tB2K6CaTLMZpXZ

— end of run —
```

*Screenshot: holehe checking 121 sites and reporting the registration summary for an address.*

What the run above shows. 121 sites checked in 4.31 seconds, with the legend that governs how to read the result: [+] used, [-] not used, [x] rate limited. No services came back as used for this address.

Read the legend before you read the result. A [x] rate limit is not a negative. A run where most sites are rate limited has produced almost no information, and reporting it as "not registered anywhere" would be wrong.

Operational note. Several checks work by requesting a password reset. The subject can observe this. It is authorised on your own assets; on a third party it changes the risk profile entirely.

## 6.2 h8mail — breach corpus lookup

```
docker exec -u 0 kali-desktop bash -lc 'h8mail -t contact@forrest.one'
```

Breach data is only usable where it is already lawfully published. Treat a breach hit as **Possible** until you can corroborate it against something current: breach dumps contain stale passwords and recycled addresses.

## 6.3 socialscan — availability across platforms in one pass

```
docker exec -u 0 kali-desktop bash -lc \
'socialscan billyforrest contact@forrest.one'
```

socialscan answers a different question from holehe: not whether an address is registered, but whether an address or username is available on each platform. Availability tells you which platforms the subject has not used, which narrows the field when a name is common.

# 7. Phase 5 — Documents and metadata

Documents leak more than people expect. A file's metadata can name its author, the software that made it, and for a photograph, the exact place it was taken.

## 7.1 Office and PDF documents

```
docker exec -u 0 kali-desktop bash -lc 'exiftool -a -G1 -s document.pdf'
```



```
OSINT_SHOT
File Actions Edit View Help
$ exiftool -a -G1 -s sample.pdf
[System] FileInodeChangeDate : 2026:10:01 21:40:51+00:00
[System] FilePermissions : -rw-rw-r--
[File] FileType : PDF
[File] FileTypeExtension : pdf
[File] MIMEType : application/pdf
[PDF] PDFVersion : 1.3
[PDF] Linearized : No
[PDF] Author : b.forrest
[PDF] CreateDate : 2026:10:01 22:40:51+01:00
[PDF] Creator : Microsoft Word 2019
[PDF] Keywords : internal, draft
[PDF] ModifyDate : 2026:10:01 22:40:51+01:00
[PDF] Producer : ReportLab PDF Library - (opensource)
[PDF] Subject : Internal
[PDF] Title : Q3 Network Review
[PDF] Trapped : False
[PDF] PageMode : UseNone
[PDF] PageCount : 1
[XMP-x] XMPToolkit : Image::ExifTool 13.55
[XMP-pdf] Author : b.forrest
[XMP-pdf] Keywords : internal;draft
[XMP-tiff] Artist : Billy Forrest
[XMP-tiff] Copyright : My Cyber Academy
[XMP-xmp] CreatorTool : Microsoft Word 2019
— end of run —
```

Screenshot: exiftool showing PDF metadata including author, creator tool and keywords.

What the run above shows. The PDF carries an author name, a creator tool (Microsoft Word 2019), a title, a subject, keywords, and the producing application. The -G1 flag groups each tag by the metadata family it came from, and -s prints tag names rather than descriptions, which makes the output scriptable.

Three flags worth knowing:

| Flag | Effect                                                               |
|------|----------------------------------------------------------------------|
| -a   | Show duplicate tags, including ones hidden in a lower-priority group |
| -G1  | Group output by metadata family: PDF, XMP-pdf, XMP-tiff, IFD0        |
| -s   | Short tag names, so output can be piped and parsed                   |

The XMP families are the leak. Note that the same author appears under [PDF], [XMP-pdf] and [XMP-tiff] in the run above. XMP survives edits that strip the simpler tags, so a document supposedly anonymised at the PDF level can still name its author in XMP.

## 7.2 Photographs — the GPS problem

```
docker exec -u 0 kali-desktop bash -lc \
'exiftool -a -G1 -s photo.jpg | grep -iE "GPS|Model|Make|Software"'
```

```
OSINT_SHOT
File Actions Edit View Help
$ exiftool -a -G1 -s sample_photo.jpg
[IFD0] Make : Apple
[IFD0] Model : iPhone 13 Pro
[IFD0] Software : 16.5.1
[GPS] GPSVersionID : 2.3.0.0
[GPS] GPSLatitudeRef : North
[GPS] GPSLatitude : 51 deg 30' 2.52"
[GPS] GPSLongitudeRef : West
[GPS] GPSLongitude : 0 deg 7' 28.56"
[Composite] GPSLatitude : 51 deg 30' 2.52" N
[Composite] GPSLongitude : 0 deg 7' 28.56" W
[Composite] GPSPosition : 51 deg 30' 2.52" N, 0 deg 7' 28.56" W
— end of run —
```

Screenshot: exiftool showing GPS position, camera make and model in a photograph.

What the run above shows. A camera make and model, a software version, a capture date, and a full GPS position in both decimal and degrees-minutes-seconds form.

This is the single most consequential metadata finding in OSINT. A photograph posted to a public page can carry the coordinates of the place it was taken. Two practical rules follow:



- When publishing, strip it. `exiftool -all= file.jpg` before a file leaves your organisation.
- When collecting, convert it. Run a GPS pair through a mapping service and record the human-readable location in the report, not just the raw numbers.

The same logic runs in reverse for theHarvester output. Point exiftool at every document the harvester gathered: one pass over a folder.

```
exiftool -r -a -G1 -s harvested/ | grep -iE "Author|Creator|GPS|Title"
```

*metagoofil is not available. No PyPI distribution exists for it. Use exiftool over the documents theHarvester collected, which produces the same result.*

## 8. Phase 6 — Analysis and grading

Collection produces raw material. Analysis turns it into something a reader can act on, and the core of analysis is stating how confident you are.

### Grading a finding

A claim without a level and a source is an opinion, not intelligence.

|                    |                                                       |                                                  |
|--------------------|-------------------------------------------------------|--------------------------------------------------|
| <b>Confirmed</b>   | Two independent sources, or one authoritative record. | e.g. WHOIS record, certificate transparency      |
| <b>Likely</b>      | One strong source plus supporting context.            | e.g. theHarvester host plus a live HTTP response |
| <b>Possible</b>    | A single source that cannot be corroborated.          | e.g. One social profile match                    |
| <b>Unconfirmed</b> | Reported by a source you cannot assess.               | e.g. Third party claim, no direct evidence       |

State the level for every finding. Never upgrade a level without a second, independent source.

Figure 4: The four confidence levels, what each one means and an example of evidence.

| Level              | Means                                                | Evidence                                    |
|--------------------|------------------------------------------------------|---------------------------------------------|
| <b>Confirmed</b>   | Two independent sources, or one authoritative record | A WHOIS record; certificate transparency    |
| <b>Likely</b>      | One strong source plus supporting context            | A harvested host plus a live HTTP response  |
| <b>Possible</b>    | One source that cannot be corroborated               | A single social profile match               |
| <b>Unconfirmed</b> | A source you cannot assess                           | A third-party claim with no direct evidence |

Never upgrade a level without a second, independent source. Two lookups against the same underlying record are one source, not two. A WHOIS record and a dig answer from those nameservers are related, not independent.



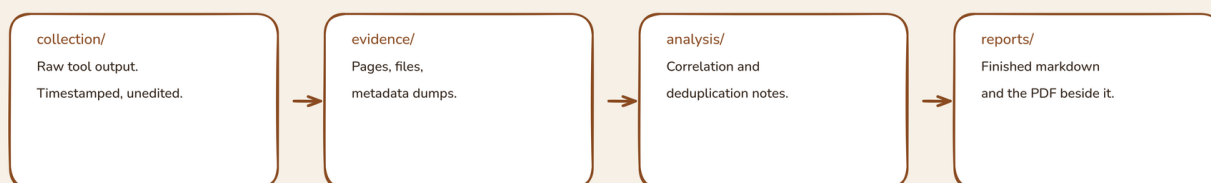
## The analysis steps

1. Deduplicate. One host reported by three tools in your lab is one finding.
2. Correlate. A username, an avatar and a linked email pointing at one person is a stronger claim than any one of them.
3. Separate fact from inference. "The MX record names privateemail.com" is fact. "The organisation therefore uses a paid mail provider" is inference. Label which is which.
4. Grade every finding. No level, no finding.
5. State the negatives. "Nothing found" with the method named.

## 9. Phase 7 — Reporting

### From raw output to a graded report

Nothing is typed by hand. Every claim traces back to a captured run.



Rule: if a tool failed, the failure is captured too.

A run log with an error in it is worth more than a clean log that hides one. Never hand-write output you did not receive.

#### Before you delete a case folder

Confirm every file in it is retrievable somewhere else first.

Figure 5: From raw tool output to a graded report, through collection, evidence, analysis and reports folders.

## Folder discipline

Every investigation gets its own case folder, and every phase writes to a known place.

| Folder             | Holds                                        |
|--------------------|----------------------------------------------|
| cases/case-<slug>/ | The scope file, one folder per investigation |
| collection/        | Raw, unedited, timestamped tool output       |
| evidence/          | Captured pages, documents, metadata dumps    |
| analysis/          | Working notes, correlation, deduplication    |
| reports/           | Finished reports and their markdown sources  |
| exports/           | Machine-readable output: CSV, JSON           |



## Report structure

```
## OSINT Report – <target>

## Summary
Three sentences. What was asked, what was found, what it means.

## Scope and method
Target, question, collection window, tools used, what was out of scope.

## Findings
One table row per finding: statement, confidence level, source.

## Negative results
What was looked for and not found, with the method named for each.

## Limitations
Rate limiting, source coverage, anything that would change the result.

## Recommendations
What the reader should do next.
```

## The findings table

| Finding                                               | Confidence | Source                   |
|-------------------------------------------------------|------------|--------------------------|
| Domain registered through NameCheap, created 2019     | Confirmed  | WHOIS, 1 Oct 2026        |
| Authoritative DNS at Cloudflare                       | Confirmed  | dnsrecon NS records      |
| Mail hosted at privateemail.com                       | Confirmed  | MX records               |
| Registrant identity withheld behind a privacy service | Confirmed  | WHOIS                    |
| No hosts indexed by DuckDuckGo                        | Confirmed  | theHarvester, 1 Oct 2026 |
| Username reused across several platforms              | Possible   | maigret, 36 matches      |

## Verify before you deliver

```
pdfinfo report.pdf | grep -E 'Pages|Title|Author'
pdftotext report.pdf - | grep -c '<finding term>'
```

Count the findings in the PDF and compare with the findings table in the source. A report that renders is not the same as a report that is complete.

## 10. Tool reference

Every tool in this guide, with the invocation that works in the lab.

| Tool         | Phase          | Command                                       |
|--------------|----------------|-----------------------------------------------|
| whois        | Infrastructure | whois <domain>                                |
| dig          | Infrastructure | dig +short MX <domain>                        |
| dnsrecon     | Infrastructure | dnsrecon -d <domain> -t std                   |
| dnsenum      | Infrastructure | dnsenum <domain>                              |
| fierce       | Infrastructure | fierce --domain <domain>                      |
| amass        | Infrastructure | amass enum -d <domain>                        |
| theHarvester | Infrastructure | theHarvester -d <domain> -b duckduckgo -l 300 |



| Tool            | Phase           | Command                                         |
|-----------------|-----------------|-------------------------------------------------|
| whatweb         | Infrastructure  | whatweb -a 1 https://<domain>                   |
| shodan          | Infrastructure  | shodan host <ip>                                |
| censys          | Infrastructure  | censys search <query>                           |
| assetfinder     | Infrastructure  | assetfinder -subs-only <domain>                 |
| waybackurls     | Infrastructure  | waybackurls <<< <domain>                        |
| go-dork         | Infrastructure  | go-dork -q "<dork>" -e google                   |
| recon-ng        | Infrastructure  | recon-ng (interactive)                          |
| spiderfoot      | Infrastructure  | spiderfoot -s <domain>                          |
| photon          | Infrastructure  | photon -u <url>                                 |
| maigret         | People          | maigret --timeout 5 --no-progressbar <username> |
| sherlock        | People          | sherlock <username>                             |
| social-analyzer | People          | social-analyzer --username <name> --metadata    |
| holehe          | Email           | holehe --only-used <email>                      |
| h8mail          | Email           | h8mail -t <email>                               |
| socialscan      | Email           | socialscan <query>                              |
| exiftool        | Documents       | exiftool -a -G1 -s <file>                       |
| nmap            | Authorised only | nmap -sV <target>                               |

## Installing a missing tool

Tools are added to the disposable lab, not to your desktop. Three install paths cover everything above.

```
## Python CLI tools – PEP-668 blocked, so use pipx
pipx install maigret

## Go tools – export GOBIN so the binary lands on PATH
export GOPATH=/home/kasm-user/go GOBIN=/usr/local/bin
go install github.com/tomnomnom/assetfinder@latest

## A standalone script with no package
git clone --depth 1 https://github.com/s0md3v/Photon /opt/Photon
printf '#!/bin/bash\nexec python3 /opt/Photon/photon.py "$@"\n' > /usr/local/bin/photon
chmod +x /usr/local/bin/photon
```

Always smoke-test on a real input before declaring a tool installed. `command -v <tool>` proves the file exists, nothing more.

```
assetfinder -subs-only example.com    # must return output, not just exist
```

Two install traps worth knowing:

- `pip install photon` installs a library, not the Photon tool. It has no command-line entry point. Clone the repository and write a launcher.
- `go install` fails for `PhoneInfoga`. Its build embeds a Vue frontend that needs node. Use the prebuilt release binary instead.



## 11. Troubleshooting

These are real failures hit while writing this guide, with the fix.

| Symptom                                               | Cause                                                         | Fix                                                                                  |
|-------------------------------------------------------|---------------------------------------------------------------|--------------------------------------------------------------------------------------|
| theHarvester finds nothing on every source            | engines returning no results for a <b>low</b> -profile domain | run additional sources with -b bing,duckduckgo,otrs,crtsh and report each separately |
| waybackurls returns nothing                           | a young domain with no archived history                       | state it as a finding; try curl against the Wayback CDX API directly                 |
| maigret logs Too many errors of type "Bot protection" | target platforms rate limiting the container                  | lower concurrency, or accept the reduced coverage and state the percentage           |
| holehe shows [x] for most sites                       | rate limits, not absence                                      | do not report rate-limited sites as negative                                         |
| whois output is all REDACTED FOR PRIVACY              | registrar privacy service in use                              | that is the finding; pursue other pivots                                             |
| A metadata field appears three times                  | the same value stored in PDF, XMP-pdf and XMP-tiff            | expected; report the XMP copy as the durable one                                     |
| Output is empty in your log but the tool ran          | output went to stderr                                         | capture with 2>&1                                                                    |

## 12. Appendix — reproducing the screenshots

The images in this guide are captures of real terminal sessions, not reconstructions. The capture tool and the commands are included so any figure can be regenerated.

```
## 1. Run the real command in the lab, keeping the transcript
docker exec -u 0 kali-desktop bash -lc \
  'export PATH=$PATH:/usr/local/bin:/home/kasm-user/.local/bin; <command>' \
  > evidence/$(date -u +%Y%m%dT%H%M%S)_<slug>.txt 2>&1

## 2. Screenshot the terminal that produced it
tools/term_shot.py images/<slug>.png <seconds> -- '<command>'
```

Pitfalls the capture tool exists to handle — each one cost real time:

- qterminal has no title flag. Set the window title with a printf escape and locate the window with xdotool search --name.
- The VNC screen starts at 1024x768. Run xrandr -s 1920x1080 first or the capture is a small letterboxed desktop.
- Write the command to a script file inside the container. Inlining it crosses four quoting layers and the escape sequences are silently eaten.
- Use pkill -x qterminal, not pkill -f qterminal. The -f form matches the docker exec command line and kills the caller.
- Raise the terminal font before capture. At the default size the output is unreadable once scaled into a document.

Training at Forrest.one · OSINT Field Guide · Open source — unclassified.



# Malware Analysis Guide

## Malware Inspection & Reverse Engineering

Training at Forrest.one · Static and dynamic analysis in an isolated lab

### How to use this guide

This is a working manual. Every command in it was run against a real file while the guide was written, and every picture is a capture of the tool that produced the output.

Three things to know before you start:

1. You need an isolated lab. Never analyse a sample on a machine you care about. Section 3 builds one.
2. Static before dynamic, always. Read the file before you run it. Most questions are answered without executing anything.
3. Capture what you do. A claim you cannot reproduce is not a finding.

### The safety rule that comes before every other rule

Malware analysis has one non-negotiable precondition: the sample must not be able to reach anything you value. Not your files, not your network, not the internet.

| Control                                | Why it exists                                    |
|----------------------------------------|--------------------------------------------------|
| Isolated network, no route out         | The sample cannot call home or spread            |
| No shared folders from your host       | The sample cannot read or encrypt your work      |
| Snapshots before detonation            | You can revert to a known-good state             |
| No credentials in the lab              | There is nothing worth stealing                  |
| Samples are never executed on the host | The host is the one machine that must stay clean |

Everything in this guide runs inside containers on an internal-only Docker network. The host never executes a sample.

*A note on what you analyse. The samples in this guide are inert and synthetic. They are real, structurally valid PE and ELF files, but they carry indicators rather than payloads. Where a demonstration needs real behaviour, the guide uses a purpose-built benign specimen whose source is included — see [samples/beacon\\_sim.c](#). Never execute a file you did not build or obtain from a trusted source.*

## 1. Why malware is worth reading

Malware is somebody else's software, written to solve a problem you would not want to have. Reverse engineering it answers four questions:

| Question                     | What the answer gives you                       |
|------------------------------|-------------------------------------------------|
| What can it do?              | Capabilities, ranked by the risk they represent |
| How does it persist?         | The keys, services and files to remove          |
| Where does it call home?     | Network indicators you can block                |
| How do I recognise it again? | A detection rule that outlives the hash         |



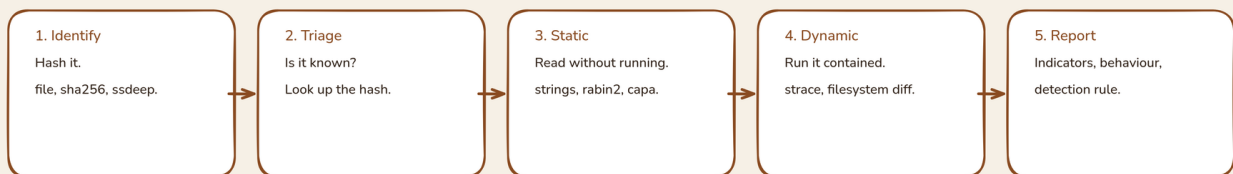
Hashes are perishable, behaviour is durable. Changing one byte changes the hash, so a hash-only detection fails against the next build. A rule built on behaviour — a mutex name, a pipe name, a sequence of API calls — survives repackaging. That is why this guide spends most of its length on behaviour and only a paragraph on hashing.

## 2. The workflow

Five phases, in order. Each answers a question the next phase depends on.

### The triage workflow

Five phases. Each one answers a question the next phase depends on.



Static before dynamic, always. A file you never executed cannot surprise you, and most questions are answered here.

The rule that holds the whole method together

Every claim in the report comes from a command you can re-run.

If you did not capture the output, you did not observe it.

A null result is a finding. Record it with its method.

Figure 1: The five phase triage workflow, from identification to report.

| Phase       | Question                         | Output                            |
|-------------|----------------------------------|-----------------------------------|
| 1. Identify | What is this file?               | Hash, type, size                  |
| 2. Triage   | Is it already known?             | A verdict or a reason to continue |
| 3. Static   | What does it do without running? | Strings, imports, capabilities    |
| 4. Dynamic  | What does it do when it runs?    | Syscalls, files, network          |
| 5. Report   | What does the reader act on?     | Indicators and a detection rule   |

Why static first. A file you never executed cannot surprise you. Static analysis is also fast: most questions a triage analyst has are answered in the first two phases.

## 3. Build the lab

The lab is two containers on a Docker network with no gateway and no DNS. The analyst container holds the tooling; the victim container is where a sample is detonated.



## Two containers, one air gap

Malware runs in a sandbox with no route to anything you care about.

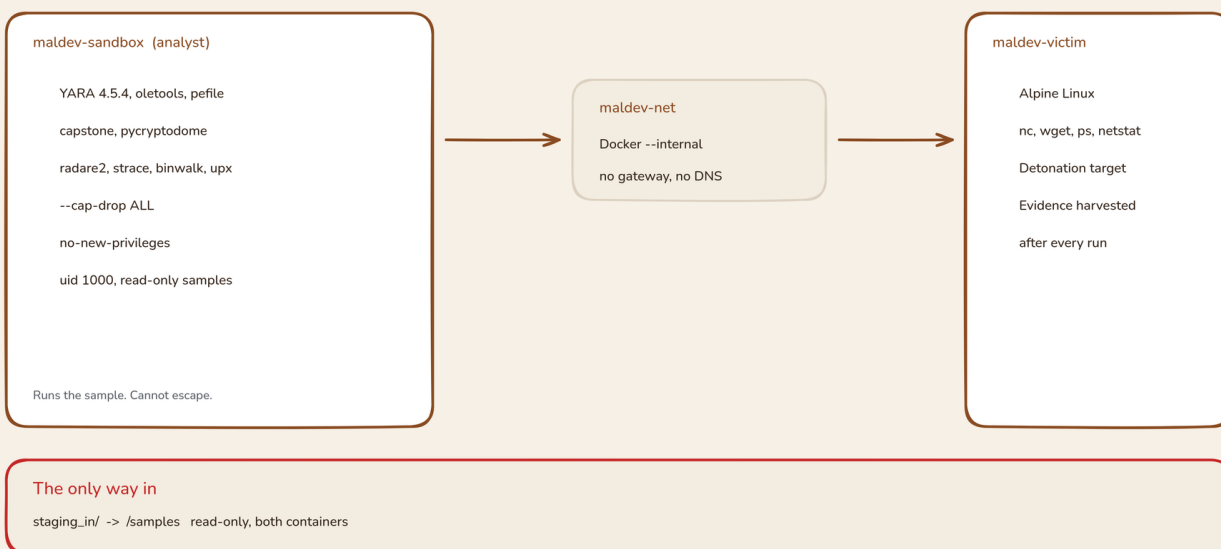


Figure 2: The analyst container, the internal network, and the victim container.

## Starting and verifying the lab

```
cd /home/billy-forrest/Malware/sandbox
./sandbox.sh up          # start, then verify isolation
./sandbox.sh verify      # isolation assertions – run before EVERY dynamic analysis
./sandbox.sh status      # what is running
./sandbox.sh logs        # harvest victim evidence to evidence/<timestamp>/
./sandbox.sh down        # stop
```

verify must print all six PASS lines. If it does not, stop and fix it:

```
PASS analyst: no internet
PASS analyst: no LAN
PASS analyst: no DNS
PASS analyst: toolchain
PASS analyst: strace runs
PASS victim: no internet
ALL CHECKS PASSED – sandbox is safe to detonate in
```

## The air gap

Samples enter through exactly one directory, mounted read-only into both containers:

```
/home/billy-forrest/Malware/sandbox/staging_in/ -> /samples (read-only)
```

Nothing else of the host is visible to a container that runs malware.

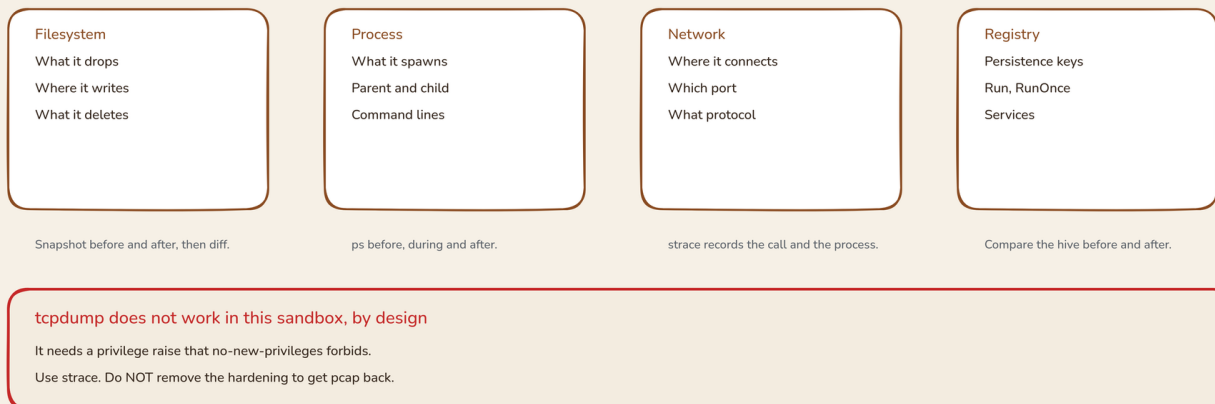
## The hardening, and why it is not negotiable

The containers run with `--cap-drop ALL`, `--security-opt no-new-privileges`, a non-root uid, pids and memory limits, and a `nosuid,nodev,noexec /tmp`.



## What you watch when it runs

Four families of evidence, each captured differently.



A sandbox with a hole in it is not a sandbox.

Capture less, but capture it inside the boundary.

Figure 3: What you watch when a sample runs, and the tcpdump caveat.

*tcpdump does not work in this sandbox, by design. It ships file capabilities and needs a privilege raise to apply them, which no-new-privileges forbids. CapEff stays zero even with --cap-add NET\_RAW. Use strace instead — it records the process name alongside the connection, which is arguably better for attribution. Do not remove no-new-privileges to get pcap back. That is a real isolation loss traded for a convenience you can get another way.*

## Working in the analyst container

```
./sandbox.sh analyst    # shell as the analyst
./sandbox.sh victim     # shell as the victim
```

The analyst toolchain lives in a venv on a bind-mounted volume, so it survives a container rebuild:

```
/opt/maldev-venv/bin/python # pefile, capstone, yara, oletools, pycryptodome
/opt/maldev-tools/strace    # strace is NOT on PATH; call it by full path
```

strace is not on PATH. command -v strace returns nothing and a bare strace fails with "command not found". Call /opt/maldev-tools/strace explicitly. This trips up every first-time user of the sandbox.

## 4. Phase 1 — Identify

### 4.1 Hash it first

Everything downstream is keyed to the hash. Compute three, because different services index different ones.

```
sha256sum samples/cobalt_like.dll
md5sum    samples/cobalt_like.dll
sha1sum   samples/cobalt_like.dll
ssdeep    samples/cobalt_like.dll # fuzzy hash: survives small edits
```



```
OSINT_SHOT
File Actions Edit View Help
$ file samples/*
samples/cobalt_like.dll:      PE32+ executable for MS Windows 16.00 (console), x86-64, 2 sections
samples/emotet_like.elf:     ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, no section header
samples/ransom_note.txt.sample: ASCII text
samples/wsrddropper_2025.bin: data

$ sha256sum samples/cobalt_like.dll
c29855bb0c1e7fed3b5b1ac773e1d4f41c6bdd44e2277868369368f35c3dddf44  samples/cobalt_like.dll

— end of run —
```

Screenshot: file type identification and SHA-256 for each sample in the set.

What the run above shows. The set contains four files of genuinely different types:

| File                   | Identified as                                    | What it tells you                                        |
|------------------------|--------------------------------------------------|----------------------------------------------------------|
| cobalt_like.dll        | PE32+ executable, x86-64                         | Windows, 64-bit, two sections                            |
| emotet_like.elf        | ELF 64-bit, statically linked, no section header | Linux, and the missing section header is itself a signal |
| wsrddropper_2025.bin   | data                                             | No recognised format — unpack it or carve it             |
| ransom_note.txt.sample | ASCII text                                       | A document, not an executable                            |

"No section header" is worth pausing on. A statically linked ELF whose section headers have been stripped has been through a tool. Section headers are not required to run, so removing them costs the malware nothing and costs you the symbol and section names you would otherwise read.

ssdeep earns its place. A fuzzy hash produces a similarity score between two files. If a sample is a lightly modified variant of something you have seen, ssdeep tells you that when SHA-256 says the files are unrelated.

## 4.2 Is it already known?

Look the hash up before you spend an hour reversing it. Public services index millions of samples.

| Service         | What it answers                                    |
|-----------------|----------------------------------------------------|
| VirusTotal      | Multi-engine verdict, community comments           |
| MalwareBazaar   | Whether the sample is already collected and tagged |
| Hybrid Analysis | Existing sandbox reports                           |
| Any.Run         | Interactive existing analysis                      |

If the hash is unknown, that is information, not a dead end. It means either a genuinely new sample or a trivially recompiled old one. Both are worth continuing.

Never upload a confidential sample to a public service. A report submitted to a multi-scanner can become visible to other users. Check your handling rules first.

## 5. Phase 2 — Static analysis

Static analysis reads the file without executing it. This is where most of your answers live.

### 5.1 strings — the cheapest win in the discipline

```
strings -n 6 samples/cobalt_like.dll | head -22
```



```
OSINT_SHOT
File Actions Edit View Help
$ strings -n 6 samples/cobalt_like.dll | head -22
.data
https://cdn.static-resources.test/microsoft-ph-update
\\.pipe\\MSSE-2679-server
jitter 25%
Mozilla/5.0 (Windows NT 10.0; Win64; x64)
XfWuXHbpcGVcTVNTR50yNjc5LXNlcnZlcg==
— end of run —
```

Screenshot: strings output from the cobalt-like sample showing a C2 URL, a named pipe, a user agent and a jitter setting.

What the run above shows is a complete picture of the implant's configuration, extracted in one command:

| String                                                | What it means                                               |
|-------------------------------------------------------|-------------------------------------------------------------|
| https://cdn.static-resources.test/microsoft-ph-update | The C2 endpoint, disguised as an update path                |
| \\.pipe\\MSSE-2679-server                             | A named pipe — Cobalt Strike's default peer-to-peer channel |
| Mozilla/5.0 (Windows NT 10.0; Win64; x64)             | A user agent, for blending into normal traffic              |
| jitter 25%                                            | Beacon jitter, the classic sleep-timing evasion             |
| XfwdXHbpcGVITVNTRbyNjc5LXhIcnZlcg==                   | A base64 blob — decode it                                   |

That is the whole detection story from one command. A URL, a pipe name, a user agent, a beacon setting and an encoded blob. Before you disassemble a single instruction, you know what this file is for.

Decode the base64 immediately. Encoded strings are where configuration hides:

```
echo 'XfwdXHbpcGVITVNTRbyNjc5LXhIcnZlcg==' | base64 -d
```

Read strings output in this order: URLs and IPs, then paths, then registry keys, then user agents, then anything that looks encoded. URLs and IPs are your network indicators and they are the most actionable.

When strings shows nothing useful, the strings are obfuscated. See the anti-analysis section below — this is a finding, not a failure.

## 5.2 rabin2 — parse the executable structure

strings reads raw bytes. rabin2 understands the file format, so it tells you what the loader will do.

```
rabin2 -I samples/cobalt_like.dll # headers
rabin2 -i samples/cobalt_like.dll # imports
rabin2 -S samples/cobalt_like.dll # sections
rabin2 -zz samples/cobalt_like.dll # strings, with offsets
```

```
OSINT_SHOT
File Actions Edit View Help
$ capa -r /opt/capa-rules --os linux samples/emotet_like.elf

md5      cb63e4e315131813b8b90096e0d6a529
sha1     d9c3b5338a78c72146378c29941f271980f2d925
sha256   b56eceed0649998eadb783bcf4ccfbdf6bc94f80b791986ef3ec02625906f5
analysis static
os       linux
format   elf
arch     amd64
path     /home/kasm-user/malware/samples/emotet_like.elf

no capabilities found
— end of run —
```



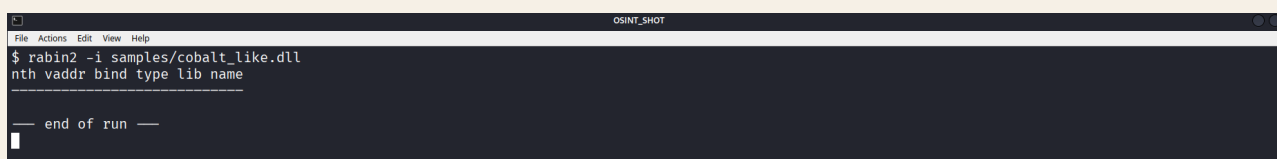
Screenshot: rabin2 parsing the PE header fields of the sample.

What the run above shows. The header-level facts that classify the file:

| Field    | Value       | Reading                                                |
|----------|-------------|--------------------------------------------------------|
| arch     | x86         | 32-bit, so it blends into a mixed environment          |
| bits     | 32          | Confirms the above                                     |
| bintype  | pe          | Windows portable executable                            |
| canary   | false       | Compiled without stack protection — normal for malware |
| os       | windows     | The target platform                                    |
| stripped | true        | Symbol names removed                                   |
| subsys   | Windows CUI | Console subsystem                                      |
| lang     | c           | Written in C                                           |
| compiler | unset       | The build toolchain is not identifiable                |

compiler being unset is a useful negative. Recognisable compilers leave fingerprints — a Delphi or Go binary is often identifiable at a glance. An unidentified toolchain often means a custom or stripped build, which tends to correlate with purpose-built malware rather than commodity crimeware.

```
rabin2 -i samples/cobalt_like.dll
```



Screenshot: the import table, showing which Windows API functions the sample calls.

The import table is the single strongest static signal you have. An import is a statement of intent: a program cannot call a function it did not import.

| Import                 | Capability it implies                                            |
|------------------------|------------------------------------------------------------------|
| CreateProcess          | Spawns another program                                           |
| VirtualAlloc           | Allocates executable memory — often for a payload                |
| WriteProcessMemory     | Writes into another process — injection                          |
| CreateRemoteThread     | Runs code in another process — injection                         |
| InternetOpen / WinHttp | Network communication                                            |
| RegSetValue            | Writes the registry — <b>likely</b> persistence                  |
| CryptEncrypt           | Cryptographic operations — <b>likely</b> ransomware or C2 crypto |

VirtualAlloc plus WriteProcessMemory plus CreateRemoteThread together is the classic injection triad. Any binary importing all three deserves a much closer look.

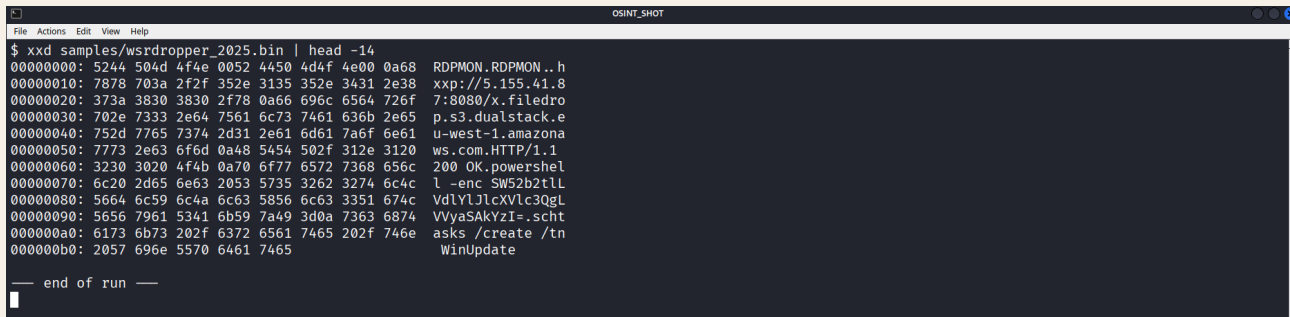
On the sample above, the import list is short. Remember that: a short import table on a binary that clearly does something is a signature of packing, which is the next thing to check.



## 5.3 Sections and entropy — finding what is hidden

Packed and encrypted data has **high** entropy. Measuring it tells you whether there is something underneath the visible file.

```
xxd samples/wsrdropper_2025.bin | head -14
```



*Screenshot: hex dump of the dropper sample showing its structure and encoded region.*

What the run above shows. A header region in readable form, then a block of bytes that is not readable text. On a real sample this pattern — a plausible header followed by an opaque blob — means the interesting part is in the blob.

Read a hex dump like this:

| What you see                         | What it means                            |
|--------------------------------------|------------------------------------------|
| A filename followed by readable text | An embedded or appended file             |
| Long runs of random-looking bytes    | Compressed or encrypted data             |
| A repeating structure                | An encoded buffer or a resource table    |
| MZ (4d 5a) in the middle of a file   | <b>An embedded executable</b> — carve it |

Finding MZ inside a file is the moment to stop and carve. A file with another executable inside it is a dropper by definition.

```
## find every embedded PE in a file
binwalk samples/wsrdropper_2025.bin

## extract what binwalk finds
binwalk -e samples/wsrdropper_2025.bin

## carve a specific region once you know the offset
dd if=samples/wsrdropper_2025.bin of=carved.bin bs=1 skip=<offset>
```

Entropy in one command. Anything above roughly 7.0 is compressed or encrypted:

```
binwalk -E samples/wsrdropper_2025.bin
```

A section with **high** entropy and no corresponding imports is packed. Unpack it before you spend time disassembling, because a disassembler reading packed bytes produces confident nonsense.

## 5.4 The PE structure you are reading

Every field above comes from the same fixed layout. Learn it once and rabin2 output becomes readable at a glance.



## Anatomy of a Windows executable

Every field you read in triage comes from this structure.

|                 |                                                 |                              |
|-----------------|-------------------------------------------------|------------------------------|
| DOS header      | Starts with the magic bytes MZ.                 | Offset 0x00                  |
| DOS stub        | Legacy message. Prints if run on DOS.           | Runs from 0x40               |
| PE header       | Starts with the bytes PE followed by two nulls. | e_lfanew points here         |
| Optional header | Entry point, image base, subsystem.             | AddressOfEntryPoint          |
| Section table   | Name, virtual size, raw size, permissions.      | One row per section          |
| .text           | The executable code.                            | Read and execute             |
| .data           | Initialised globals.                            | Read and write               |
| .rsrc           | Icons, version info, embedded files.            | Where droppers hide payloads |
| Import table    | Which library functions it calls.               | Strongest single signal      |

Figure 4: The structure of a Windows executable, field by field.

| Field                     | What it tells you                                    |
|---------------------------|------------------------------------------------------|
| <b>DOS header (MZ)</b>    | The file is a DOS-era format with a modern extension |
| <b>e_lfanew</b>           | The offset where the real PE header starts           |
| <b>PE header (PE\0\0)</b> | The signature that confirms a PE file                |
| <b>Optional header</b>    | Entry point, image base, subsystem                   |
| <b>Section table</b>      | Name, size and permissions of each section           |
| <b>.text</b>              | Executable code                                      |
| <b>.data</b>              | Initialised globals                                  |
| <b>.rsrc</b>              | Icons, version info, <b>embedded files</b>           |
| <b>Import table</b>       | The library functions it calls                       |

Two things worth internalising. First, the PE\0\0 signature is four bytes — the two characters followed by two nulls — and a tool that "fixes" it is trying to stop a scanner recognising the file. Second, .rsrc is where droppers hide payloads, because it is designed to hold arbitrary binary data and rarely attracts attention.

## 5.5 capa — what the program can do

Writing capability rules by hand does not scale. capa matches the file against a rule set and reports capabilities in plain language, mapped to MITRE ATT&CK.;

```
capa -r /opt/capa-rules --os linux samples/emotet_like.elf
```



Screenshot: capa identifying the file and reporting its analysis metadata.

What the run above shows, and why the null result matters. capa identified the file correctly — format elf, arch amd64, analysis static — and then reported no capabilities found.

That is a correct answer, not a failure. The synthetic sample is a header scaffold: it carries indicators for strings and YARA to find, but it has no compiled code for capa to reason about. Reporting it as a clean negative, with the method stated, is exactly what an analyst should do.

Two capa facts that cost time if you do not know them:

- --os is often required. Without it, capa fails with "Input file does not appear to target a supported OS" on files it can otherwise analyse perfectly.
- The PyPI install ships no rules. capa installed via pipx reports "default embedded rules not found" and needs -r /opt/capa-rules. The standalone release binary embeds them; the Python package does not.

On a real binary, the same command produces the intelligence you want:

| Capability                           | Namespace                        |
|--------------------------------------|----------------------------------|
| query environment variable           | host-interaction/environment-... |
| enumerate files on Linux             | host-interaction/file-system     |
| write file on Linux                  | host-interaction/file-system     |
| get hostname                         | host-interaction/os/hostname     |
| terminate process                    | host-interaction/process         |
| get password database entry on Linux | host-interaction/session         |

| ATT&CK Tactic | ATT&CK Technique                     |
|---------------|--------------------------------------|
| DISCOVERY     | File and Directory Discovery [E1083] |
| DISCOVERY     | System Information Discovery [E1082] |
| FILE SYSTEM   | Get File Attributes [C0049]          |
| FILE SYSTEM   | Writes File [C0052]                  |
| PROCESS       | Terminate Process [C0018]            |

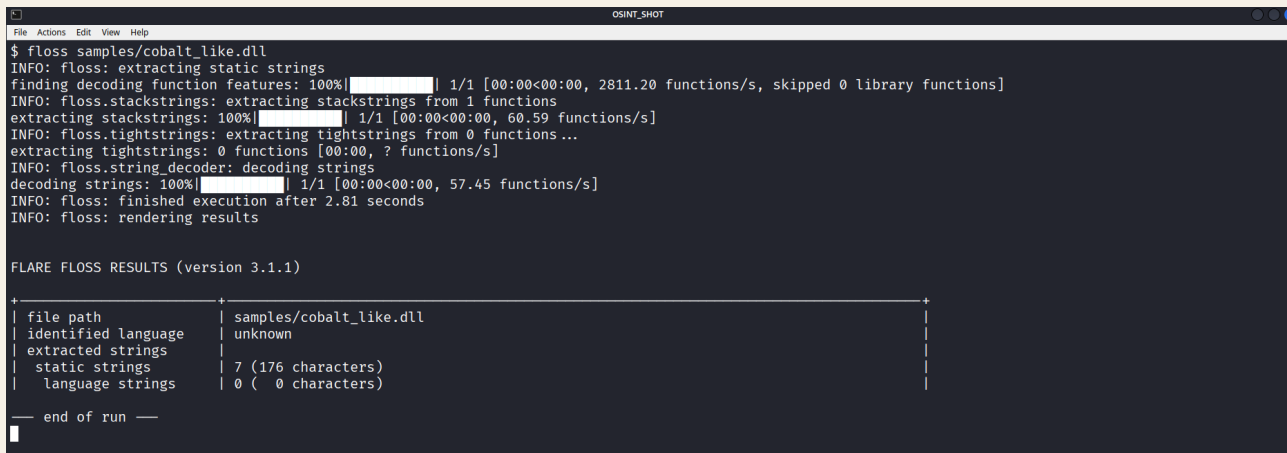


Read the ATT&CK; table, not just the capability list. The capabilities tell you what the code does; the ATT&CK; mapping tells you what that means. "Get password database entry" under host-interaction/session is credential access, and it escalates the finding from "interesting" to "report it today".

## 5.6 floss — strings that strings cannot see

Obfuscated strings are decoded at run time, so a static strings sweep misses them. FLOSS emulates the decoding routines and recovers them.

```
floss samples/cobalt_like.dll
```



```
$ floss samples/cobalt_like.dll
INFO: floss: extracting static strings
finding decoding function features: 100%|██████████| 1/1 [00:00<00:00, 2811.20 functions/s, skipped 0 library functions]
INFO: floss.stackstrings: extracting stackstrings from 1 functions
extracting stackstrings: 100%|██████████| 1/1 [00:00<00:00, 60.59 functions/s]
INFO: floss.tightstrings: extracting tightstrings from 0 functions...
extracting tightstrings: 0 functions [00:00, ? functions/s]
INFO: floss.string_decoder: decoding strings
decoding strings: 100%|██████████| 1/1 [00:00<00:00, 57.45 functions/s]
INFO: floss: finished execution after 2.81 seconds
INFO: floss: rendering results

FLARE FLOSS RESULTS (version 3.1.1)

+-----+-----+
| file path | samples/cobalt_like.dll |
| identified language | unknown |
| extracted strings | |
| static strings | 7 (176 characters) |
| language strings | 0 ( 0 characters) |
+-----+-----+

— end of run —
```

Screenshot: FLOSS extracting static and decoded strings from the sample.

What the run above shows. FLOSS reports its phases — static strings, stack strings, tight strings, then string decoding — and then summarises: 7 static strings, and 0 decoded strings. For this sample the decoded count is zero because the strings are stored plainly.

FLOSS is PE and shellcode only. On an ELF it fails immediately:

```
ERROR: floss: failed to analyze sample: FLOSS currently supports the following
formats for string decoding and stackstrings: PE
```

That is a hard limitation of the tool, not a mistake in your command. For ELF binaries, use strings, rabin2 -zz, and manual decoding.

The three string classes FLOSS distinguishes are worth knowing:

| Class   | What it is                                                    |
|---------|---------------------------------------------------------------|
| Static  | Plain strings in the binary — what strings finds              |
| Stack   | Built on the stack one character at a time, to defeat strings |
| Decoded | Encrypted, decrypted by a routine FLOSS emulates              |

A binary with many stack strings and few static ones is deliberately hiding its text.

## 5.7 YARA — writing the detection

Once you have indicators, a YARA rule turns them into something reusable. YARA matches patterns across a corpus, so one rule can screen thousands of files.

```
yara -w yara_rules.yar samples/
```



```
OSINT_SHOT
File Actions Edit View Help
$ yara -w yara_rules.yar samples/
Simulated_Emotet_Downloader samples//emotet_like.elf
Simulated_CobaltStrike_Beacon samples//cobalt_like.dll
Simulated_Benign_PE_Baseline samples//cobalt_like.dll
Simulated_Ransomware_Decryptor_Note samples//ransom_note.txt.sample
Simulated_Dropper_Stage samples//wsrdropper_2025.bin
— end of run —
```

*Screenshot: YARA matching five rules across the synthetic sample set.*

What the run above shows. Five matches across four files:

|                                     |                                |
|-------------------------------------|--------------------------------|
| Simulated_Emotet_Downloader         | samples/emotet_like.elf        |
| Simulated_CobaltStrike_Beacon       | samples/cobalt_like.dll        |
| Simulated_Benign_PE_Baseline        | samples/cobalt_like.dll        |
| Simulated_Ransomware_Decryptor_Note | samples/ransom_note.txt.sample |
| Simulated_Dropper_Stage             | samples/wsrddropper_2025.bin   |

Note that one file matched two rules. `cobalt_like.dll` matched both the beacon rule and the benign baseline rule. That is not a bug — it is the rule set working as designed, and it teaches the lesson every detection engineer learns: overlapping rules create ambiguity, and ambiguity in a detection is a false positive waiting to happen. Resolve it by making the specific rule more specific, not by deleting the general one.

## A rule you can read

```
rule Simulated_Emotet_Downloader {
  meta:
    family      = "Emotet-style downloader"
    category    = "simulated"
    description  = "C2 beacon + persistence under HKCU Run"
    harmless    = true

  strings:
    $c2         = "update.cdn-sync.test"           ascii wide
    $mutex      = "MSUUpdate_8f3a91"               ascii wide
    $persist    = "CurrentVersion\\Run"             ascii wide
    $beacon     = "C2 beacon every 60s"            ascii wide

  condition:
    uint32(0) == 0x464c457f and                     // ELF magic
    ( 2 of ($c2, $mutex, $persist, $beacon) )
}
```

Read the condition clause carefully — it is where the engineering is:

- `uint32(0) == 0x464c457f` anchors the rule to ELF files by magic bytes, so it cannot fire on Windows binaries that happen to contain one of the strings.
- `2 of (...)` requires two indicators, not one. A single-string rule fires on every file that contains that string for innocent reasons. Requiring two is the difference between a rule you can deploy and a rule that pages you at 3am for a false positive.

Write the meta: block properly. author, description, date and a reference make a rule maintainable a year later, when nobody remembers why it exists.

YARA-X is the successor to YARA. VirusTotal rewrote YARA in Rust; it is production- ready, roughly 99% rule-compatible, and substantially faster. Rules written for YARA mostly run unchanged, so anything you write today is not wasted.



## 5.8 Anti-analysis — when the file fights back

The techniques below are why static analysis sometimes stalls. Recognising them is the first half of defeating them.

Anti-analysis: what malware does to slow you down

Recognising the technique is the first half of defeating it.

|                                                                            |                                                |
|----------------------------------------------------------------------------|------------------------------------------------|
| <b>Packing</b><br>Code is compressed and unpacked at run time.             | High entropy, few imports, tiny code section.  |
| <b>Anti-debugging</b><br>Detects a debugger attached to the process.       | IsDebuggerPresent, timing checks, int 3 traps. |
| <b>Anti-virtual-machine</b><br>Looks for artefacts of an analysis host.    | Registry keys, MAC prefixes, driver names.     |
| <b>String obfuscation</b><br>Hides readable text from the strings command. | Encoded blobs, stack strings, custom XOR.      |
| <b>Process injection</b><br>Runs its payload inside a trusted process.     | Hollowing, thread hijack, APC injection.       |
| <b>Living off the land</b><br>Uses signed system tools instead of its own. | certutil, powershell, wmic, mshta.             |

Figure 5: Common anti-analysis techniques and the tell for each one.

| Technique                  | What it does                         | The tell                                     |
|----------------------------|--------------------------------------|----------------------------------------------|
| <b>Packing</b>             | Compresses code, unpacks at run time | <b>High</b> entropy, few imports, tiny .text |
| <b>Anti-debugging</b>      | Detects an attached debugger         | IsDebuggerPresent, timing checks, int 3      |
| <b>Anti-VM</b>             | Detects an analysis host             | Registry keys, MAC prefixes, driver names    |
| <b>String obfuscation</b>  | Hides text from strings              | Encoded blobs, many stack strings            |
| <b>Process injection</b>   | Runs payload in a trusted process    | The injection triad in the imports           |
| <b>Living off the land</b> | Uses signed system tools             | certutil, powershell, wmic, mshta            |

Packing is the most common, and it is defeatable. Options, in order of effort:

1. Unpack with upx if it is UPX. `upx -d packed.exe`. UPX is a legitimate compressor, so its output is trivially reversible. A UPX-packed malware sample is a **low**-effort build.

```
bash upx -d samples/suspected_packed.exe
```

Let it unpack itself. Run it in the sandbox, let it decompress into memory, then dump the process memory and analyse that. This is the reliable general answer.

Find the original entry point. The unpacker stub must jump to the real code eventually. Break there and dump.

Living off the land is the hardest to detect and the easiest to write. A script that calls certutil to download a file produces no malicious binary at all. Detection has to move from file signatures to command-line monitoring — which is a different kind of control from the ones this guide otherwise teaches.



## 6. Phase 3 — Dynamic analysis

Dynamic analysis runs the sample and watches what it does. It answers the questions static analysis cannot: what it actually touches, and in what order.

Never skip the isolation check. `./sandbox.sh verify` first, every time.

### 6.1 The benign specimen

The synthetic samples are indicator carriers with no compiled code, so they have no runtime behaviour to observe. To demonstrate dynamic analysis honestly, the guide includes a small benign specimen you build yourself:

```
samples/beacon_sim.c    -> beacon_sim    (789,224 bytes, statically linked)
sha256 029f9b26561d00f276a93a9dcf1699a3fba8ca00578079e21d813cecf3d4d153
```

It performs four things an analyst looks for, and nothing else:

| Behaviour                                 | Why it is in there |
|-------------------------------------------|--------------------|
| Writes <code>/tmp/.beacon_sim_drop</code> | Dropper behaviour  |
| Reads <code>/etc/passwd</code>            | Credential access  |
| Connects to <code>192.0.2.10:8443</code>  | C2 beacon attempt  |
| Forks and execs <code>/bin/echo</code>    | Process spawning   |

`192.0.2.10` is in TEST-NET-1, reserved by RFC 5737 for documentation. It is guaranteed non-routable, so the specimen cannot contact anything even if the sandbox were misconfigured — a second layer of safety independent of the network rules.

There is no payload, no persistence and no evasion. Its whole purpose is to give the instrumentation something real to observe.

### 6.2 Trace it

`strace` records every system call a process makes, with the calling process id. In this sandbox it is the primary network-capture tool, because `tcpdump` cannot run.

```
cd /samples
/opt/maldev-tools/strace -f -e trace=connect,openat,fork,vfork,execve \
-o /tmp/trace.txt ./beacon_sim
```

```
OSINT_SHOT
# Captured inside maldev-sandbox:
# strace -f -e trace=connect,openat,fork,execve -o trace.txt ./beacon_sim
4511 execve("./beacon_sim", ["/beacon_sim"], 0x7ffef4d17738 /* 45 vars */) = 0
4511 openat(AT_FDCWD, "/tmp/.beacon_sim_drop", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
4511 openat(AT_FDCWD, "/etc/passwd", O_RDONLY) = 3
4511 connect(3, {sa_family=AF_INET, sin_port=htons(8443), sin_addr=inet_addr("192.0.2.10")}, 16) = -1 ENETUNREACH (Network is unreachable)
4511 execve("/bin/echo", ["echo", "beacon-sim child"], 0x7ffdc2781088 /* 45 vars */) = 0
4512 openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/glibc-hwcaps/x86-64-v4/libc.so.6", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
4512 openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/glibc-hwcaps/x86-64-v3/libc.so.6", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
4512 openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/glibc-hwcaps/x86-64-v2/libc.so.6", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
4512 openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
4512 openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
4512 openat(AT_FDCWD, "/usr/share/locale/locale.alias", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
4512 openat(AT_FDCWD, "/usr/lib/locale/en_US.UTF-8/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
4512 openat(AT_FDCWD, "/usr/lib/locale/en_US.UTF-8/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = 3
4512 openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache", O_RDONLY|O_CLOEXEC) = 3
4512 openat(AT_FDCWD, "/usr/lib/locale/en_US.UTF-8/LC_MEASUREMENT", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
4512 openat(AT_FDCWD, "/usr/lib/locale/en_US.UTF-8/LC_MEASUREMENT", O_RDONLY|O_CLOEXEC) = 3
— end of run —
```

Screenshot: `strace` output captured inside `maldev-sandbox`, showing the file writes, the credential read, the blocked connection and the child process.



Read the run above line by line. This is the whole of dynamic analysis in sixteen lines:

| Line                                                                                      | Reading                                                                          |
|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| execve("./beacon_sim", ...)                                                               | The process starts — your baseline reference                                     |
| openat(..., "/tmp/.beacon_sim_drop", O_WRONLY O_CREAT O_TRUNC, 0666) = 3                  | <b>Drops a file.</b> Write access, created if absent, truncated if present       |
| openat(..., "/etc/passwd", O_RDONLY) = 3                                                  | <b>Reads the account database.</b> Credential access                             |
| connect(3, {sin_port=htons(8443), sin_addr=inet_addr("192.0.2.10")}, 16) = -1 ENETUNREACH | <b>Beacon attempt, blocked by the network</b>                                    |
| execve("/bin/echo", ["echo", "beacon-sim child"], ...)                                    | <b>A child process.</b> The 4512 on this line is a different pid from 4511 above |

The ENETUNREACH on the connect line is the single most important line in this guide. It proves two things at once: the sample tried to reach the network, and the network was not there to reach. That is the isolation working, evidenced from inside the analysis rather than asserted from outside it.

The pid column is not decoration. 4511 writes the files and makes the connection; 4512 is the child. Following the pid is how you attribute an action to the right process when a sample spawns children — which is exactly what malware does.

Reading an openat line:

```
openat(AT_FDCWD, "/etc/passwd", O_RDONLY) = 3
├── AT_FDCWD: directory reference, AT_FDCWD means the current directory
├── "/etc/passwd": the path
└── O_RDONLY: flags: 0_RDONLY, 0_WRONLY, 0_CREAT...
└── return: fd 3, or a negative errno
```

6.3 What to watch, and how

What you watch when it runs  
Four families of evidence, each captured differently.

Filesystem  
What it drops  
Where it writes  
What it deletes

Snapshot before and after, then diff.

Process  
What it spawns  
Parent and child  
Command lines

ps before, during and after.

Network  
Where it connects  
Which port  
What protocol

strace records the call and the process.

Registry  
Persistence keys  
Run, RunOnce  
Services

Compare the hive before and after.

tcpdump does not work in this sandbox, by design  
It needs a privilege raise that no-new-privileges forbids.  
Use strace. Do NOT remove the hardening to get pcap back.

A sandbox with a hole in it is not a sandbox.  
Capture less, but capture it inside the boundary.

Figure 3 (repeated for reference): the four families of runtime evidence.



| Family     | Question                            | How to capture it                 |
|------------|-------------------------------------|-----------------------------------|
| Filesystem | What did it create, change, delete? | Snapshot, run, diff               |
| Process    | What did it spawn?                  | ps before, during, after          |
| Network    | Where did it connect?               | strace -e trace=connect           |
| Registry   | What persistence did it set?        | Compare the hive before and after |

The filesystem diff is the highest-value artefact of a dynamic run. Capture the file tree before detonation and after, then diff. Everything new is attributable to the sample.

```
## before
find / -xdev -type f -newermt '-1 minute' > /tmp/before.txt
## ... detonate ...
## after
find / -xdev -type f -newermt '-1 minute' > /tmp/after.txt
diff /tmp/before.txt /tmp/after.txt
```

Capture volatile evidence first. Process lists, network connections and open files exist only while the sample runs. Once it exits, they are gone.

Then `./sandbox.sh` logs harvests the victim's evidence into `evidence/<timestamp>/` so it is preserved outside the container.

## 6.4 Reading a trace for intent

A syscall trace is a sequence, and sequence is the intelligence. Individually innocent calls become a story in combination:

| Sequence                                                                            | What it is                                           |
|-------------------------------------------------------------------------------------|------------------------------------------------------|
| <code>openat(O_RDONLY)</code> on a browser profile, then <code>connect()</code>     | Credential theft followed by exfiltration            |
| <code>openat(O_WRONLY O_CREAT)</code> across many files, then <code>unlink()</code> | Ransomware: write encrypted copies, delete originals |
| <code>execve</code> of a shell, then <code>connect()</code>                         | A reverse shell                                      |
| <code>ptrace()</code> then repeated <code>mmap</code>                               | Anti-debugging or in-memory unpacking                |
| <code>connect()</code> to many hosts in turn                                        | Scanning or fast-flux C2                             |

The pattern is the finding, not the individual call. `openat` appears in almost every program ever written. `openat` on a credential store immediately before a network connection does not.

## 7. Documents, scripts and archives

Not every sample is an executable. Office documents, PDFs and archives are the most common initial-access vector.

### 7.1 Office documents

`oletools` dissects OLE and OOXML files — macros, embedded objects, and the OLE streams inside them.

```
oleid report.docm      # risk summary
olevba report.docm     # extract and analyse VBA macros
mraptor report.docm    # macro risk triage
oleobj report.docm     # extract embedded objects
```



oleid first — it is the fast pass. It flags macros, encryption and suspicious external relationships without dumping all the content.

Then olevba, and read the macro for these four things:

| Look for                               | What it means                                                    |
|----------------------------------------|------------------------------------------------------------------|
| AutoOpen, Document_Open                | <b>The macro runs automatically.</b> The single biggest red flag |
| Shell, WScript.Shell                   | It will execute a command                                        |
| CreateObject("MSXML2.XMLHTTP")         | It will download something                                       |
| Long strings of digits and Chr() calls | An obfuscated payload, built at run time                         |

mraptor scores macro risk in one line, which is useful when you are triaging fifty documents rather than one.

An encrypted document cannot be analysed without the password. oleid reports encryption as a risk, and that is a legitimate finding: encrypting a document is also how you stop a scanner reading it.

## 7.2 PDFs

```
pdf-parser --stats report.pdf      # object summary
pdf-parser --search /JavaScript report.pdf
pdf-parser --search /OpenAction report.pdf
```

Three PDF features carry almost all the risk:

| Feature     | Why                              |
|-------------|----------------------------------|
| /JavaScript | Runs code on open                |
| /OpenAction | Automatically triggers an action |
| /Launch     | Executes an external file        |

A PDF that is a delivery vehicle is usually mostly padding. Look at the object count and the file size together: a large file with very few visible objects has something hidden in it.

## 7.3 Archives

Malware arrives zipped, often password-protected to defeat scanning.

```
7z l sample.zip          # list without extracting
7z x sample.zip -ooutdir # extract
7z l -slt sample.zip      # technical listing: compression method, CRC
```

Check the compression ratio. A small archive that expands enormously is a bomb or a dropper packed inside a dropper. 7z l -slt shows the ratio per file.

A password-protected archive is doing two jobs: stopping your scanner, and tempting the user into running what is inside after they type the password from the email body.

## 8. Reporting

An analysis that is not written up did not happen. The report is the deliverable.



## 8.1 Structure

```
## Malware Analysis Report – <sample name>

## Summary
Three sentences. What it is, what it does, how dangerous it is.

## Sample identification
Filename, size, SHA-256, MD5, SHA-1, ssdeep, file type, architecture.

## Capabilities
What it can do, with the evidence for each claim.

## Behaviour (dynamic)
What it did when run – files, processes, network – with the trace.

## Indicators of compromise
Host-based, network-based, and detection rules.

## Detection
The YARA rule, and where it would be deployed.

## Limitations
What you could not determine, and what would change that.
```

## 8.2 The indicators table

| Type      | Indicator                                     | Confidence | Source  |
|-----------|-----------------------------------------------|------------|---------|
| Network   | cdn.static-resources.test/microsoft-ph-update | Confirmed  | strings |
| Host      | \\.pipe\\MSSE-2679-server                     | Confirmed  | strings |
| Network   | 192.0.2.10:8443                               | Confirmed  | strace  |
| Host      | /tmp/.beacon_sim_drop                         | Confirmed  | strace  |
| Behaviour | Reads /etc/passwd at startup                  | Confirmed  | strace  |

State the method for every indicator. An indicator without a source cannot be challenged, and an indicator that cannot be challenged cannot be trusted.

## 8.3 Report the negatives

"No capabilities found by capa with the Linux backend on this 632-byte ELF" is a finding. It tells the reader that a specific tool was applied and returned nothing, which is different from nobody having looked. Put null results in the report with the method that produced them.

## 8.4 Verify before you deliver

```
pdftotext report.pdf | grep -E 'Pages|Title|Author'
pdftotext report.pdf - | grep -c '<key indicator>'
```

Count the indicators in the PDF and compare against the source table. A report that renders is not the same as a report that is complete.

## 9. Tool reference

Every tool used in this guide, with the invocation that works in this lab.

| Tool | Phase    | Command       |
|------|----------|---------------|
| file | Identify | file <sample> |



| Tool                     | Phase     | Command                                                    |
|--------------------------|-----------|------------------------------------------------------------|
| sha256sum                | Identify  | sha256sum <sample>                                         |
| ssdeep                   | Identify  | ssdeep -b <sample>                                         |
| strings                  | Static    | strings -n 6 <sample>                                      |
| rabin2                   | Static    | rabin2 -l   -i   -S   -zz <sample>                         |
| xxd                      | Static    | xxd <sample>   head                                        |
| binwalk                  | Static    | binwalk -e <sample>                                        |
| upx                      | Static    | upx -d <packed>                                            |
| capa                     | Static    | capa -r /opt/capa-rules --os linux <sample>                |
| floss                    | Static    | floss <sample> (PE and shellcode only)                     |
| yara                     | Detection | yara -w <rules.yar> <dir>/                                 |
| oleid                    | Documents | oleid <doc>                                                |
| olevba                   | Documents | olevba <doc>                                               |
| mraptor                  | Documents | mraptor <doc>                                              |
| oleobj                   | Documents | oleobj <doc>                                               |
| pdf-parser               | Documents | pdf-parser --stats <pdf>                                   |
| 7z                       | Archives  | 7z l -slt <archive>                                        |
| /opt/maldev-tools/strace | Dynamic   | strace -f -e trace=connect,openat,execve -o t.txt <sample> |
| objdump                  | RE        | objdump -d <sample>                                        |
| readelf                  | RE        | readelf -a <sample>                                        |
| gdb                      | RE        | gdb -q <sample>                                            |
| radare2                  | RE        | r2 -A <sample>                                             |

## Tool versions in this lab

Checked live, and cross-referenced against each project's upstream release page.

| TOOL       | VERSION | UPSTREAM SOURCE                 |
|------------|---------|---------------------------------|
| yara       | 4.5.8   | github.com/VirusTotal/yara      |
| ssdeep     | 2.14.1  | ssdeep-project.github.io        |
| strace     | 7.0     | strace.io                       |
| capa       | 9.4.0   | github.com/mandiant/capa        |
| capa-rules | 1057    | github.com/mandiant/capa-rules  |
| floss      | 3.1.1   | github.com/mandiant/flare-floss |
| radare2    | 6.0.5   | radare.org                      |
| binwalk    | 2.4.3   | github.com/ReFirmLabs/binwalk   |
| upx        | 4.2.4   | upx.github.io                   |
| binutils   | 2.47    | GNU binutils                    |
| gdb        | 17.2    | GNU gdb                         |
| 7-Zip      | 26.03   | 7-zip.org                       |
| Python     | 3.14.6  | python.org                      |

— end of run —

Screenshot: the version of every analysis tool installed in the lab, with its upstream source.

| Tool  | Version | Note                                                                         |
|-------|---------|------------------------------------------------------------------------------|
| capa  | 9.4.0   | Current release; adds Ghidra/PyGhidra support and 26 new rules               |
| floss | 3.1.1   | Current FLARE release                                                        |
| yara  | 4.5.8   | <b>YARA-X</b> is the Rust successor — near-100% rule compatible, much faster |



| Tool       | Version    | Note                                                      |
|------------|------------|-----------------------------------------------------------|
| radare2    | 6.0.5      | The free disassembler that covers IDA's ground for triage |
| strace     | 7.0        | The sandbox's primary network-capture tool                |
| binwalk    | 2.4.3      | Entropy and embedded-file carving                         |
| upx        | 4.2.4      | The one packer you can usually reverse immediately        |
| binutils   | 2.47       | objdump, readelf, strings                                 |
| gdb        | 17.2       | Static use only here — the sandbox denies ptrace          |
| ssdeep     | 2.14.1     | Fuzzy hashing                                             |
| 7-Zip      | 26.03      | Archive inspection                                        |
| capa-rules | 1057 files | The rule set capa needs when installed via pipx           |
| Python     | 3.14.6     | Runs the sandbox tooling                                  |

Verify versions yourself rather than trusting a guide, including this one. Tool behaviour changes between releases, and several of the fixes in the troubleshooting table are release-specific.

```
## one-line version audit of everything this guide uses
docker exec -u 0 kali-desktop bash -lc \
'export PATH=$PATH:/usr/local/bin; capa --version; yara --version; floss --version'
```

## Installing a missing tool in the lab

```
## apt – needs root
apt-get install -y yara ssdeep strace gdb

## Python CLI tools – PEP-668 blocked, use pipx with an explicit bindir
export PIPX_HOME=/opt/pipx PIPX_BIN_DIR=/usr/local/bin
pipx install flare-capa
pipx install oletools

## capa needs an explicit rule set when installed as a Python package
git clone --depth 1 https://github.com/mandiant/capa-rules /opt/capa-rules
```

## Tools that are not in this lab, and why

| Tool       | Status               | Note                                                           |
|------------|----------------------|----------------------------------------------------------------|
| tcpdump    | <b>Does not work</b> | Blocked by no-new-privileges by design. Use strace.            |
| strace     | <b>Not on PATH</b>   | Call /opt/maldev-tools/strace by full path.                    |
| Ghidra     | Not installed        | Needs a JVM; radare2 covers the same ground for triage         |
| gdb        | Installed            | But the sandbox has no ptrace for a live process — static only |
| Volatility | Not installed        | For memory images; see the sandbox's memory-triage tooling     |
| idapro     | Not installed        | Commercial; radare2 is the free equivalent                     |

## 10. Troubleshooting

Real failures hit while writing this guide, with the fix for each.

| Symptom                                  | Cause       | Fix                      |
|------------------------------------------|-------------|--------------------------|
| strace: command not found in the sandbox | not on PATH | /opt/maldev-tools/strace |



| Symptom                                                   | Cause                        | Fix                                     |
|-----------------------------------------------------------|------------------------------|-----------------------------------------|
| capa: default embedded rules not found!                   | pipx install ships no rules  | add -r /opt/capa-rules                  |
| capa: Input file does not appear to target a supported OS | OS not specified             | add --os linux or --os windows          |
| floss: FLOSS currently supports ... PE                    | ELF not supported            | use strings and rabin2 -zz              |
| tcpdump produces nothing                                  | capability raise blocked     | use strace; do not weaken the hardening |
| strings returns nothing readable                          | strings are obfuscated       | try floss on PE, or unpack first        |
| <b>High</b> entropy, few imports                          | the file is packed           | unpack before disassembling             |
| capstone import fails in the sandbox                      | wrong Python                 | use /opt/maldev-venv/bin/python         |
| Analysis finds nothing at all                             | you may be looking at a stub | check for a second stage                |

The last row is worth stating plainly. Modern malware is frequently a small downloader whose only job is to fetch the real payload. If your sample is tiny, has almost no code, and only imports networking functions, the interesting file is not in front of you. Say so in the report rather than concluding the sample is benign.

## 11. Appendix — reproducing this guide

Every picture here is a capture of a real command run in the lab. Nothing was reconstructed. The tooling is included so any figure can be regenerated.

```
## 1. Run the real command, keeping the transcript
docker exec -u 0 kali-desktop bash -lc \
  'export PATH=$PATH:/usr/local/bin; <command>' \
  > evidence/$(date -u +%Y%m%dT%H%M%S)_<slug>.txt 2>&1

## 2. Screenshot the terminal that produced it
tools/term_shot.py images/<slug>.png <seconds> -- '<command>'

## 3. Crop the desktop chrome away so the text fills the page
tools/crop_shots.py images/*.png
```

Five pitfalls the capture tool exists to handle, each of which cost real time:

- qterminal has no title flag; set the window title with a printf escape and find the window with xdotool search --class qterminal.
- Resize the window before the command runs. The shell wraps output to the width it had when it printed, so maximising afterwards leaves every line broken mid-word.
- qterminal rejects --size and then never launches at all.
- Use pkill -x qterminal, not -f: the -f form matches the docker exec command line and kills the caller.
- The lab container has no Docker socket, so it cannot reach into the sandbox. The strace figure displays a capture made inside maldev-sandbox and labels it as such.



## What is in this folder

| Path                                                          | Holds                                                  |
|---------------------------------------------------------------|--------------------------------------------------------|
| <code>samples/synthetic-triage-set/</code>                    | The inert indicator-carrying samples                   |
| <code>samples/known-good/</code>                              | Structurally valid benign baselines                    |
| <code>samples/beacon_sim</code> and <code>beacon_sim.c</code> | The benign dynamic-analysis specimen and its source    |
| <code>samples/yara_rules.yar</code>                           | The detection rules used in section 5.7                |
| <code>figures/</code>                                         | Excalidraw sources and rendered diagrams               |
| <code>images/</code>                                          | The terminal captures                                  |
| <code>evidence/</code>                                        | Raw transcripts and the sandbox strace output          |
| <code>tools/</code>                                           | The capture, crop, figure-build and PDF-render tooling |

Training at Forrest.one · Malware Inspection & Reverse Engineering · Lab use, inert samples only.

## Digital Forensics Guide

### Digital Forensics & Disk Image Analysis

Training at Forrest.one · Evidence handling, disk, memory, network and log analysis

### How to use this guide

This is a working manual. Every command was run against real evidence while the guide was written, and every picture is a capture of the tool that produced the output.

Four things to know before you start:

1. Forensics is defined by integrity, not by tools. Anyone can run fls. What makes it forensics is being able to prove the evidence did not change. Section 2 covers that.
2. Work on a copy. You never analyse the original. The original is hashed and stored.
3. Corroborate across sources. Disk, memory, network and logs each answer a different question. One source on its own is not a finding.
4. Capture what you do. A claim you cannot reproduce, with the method stated, is not a finding.

### The rule that comes before every other rule

Forensic work has one non-negotiable precondition: your actions must not change the evidence. This is ACPO Principle 1, and everything else in the discipline follows from it.

| Principle                                                   | What it means in practice                    |
|-------------------------------------------------------------|----------------------------------------------|
| No action may change the data                               | You never open evidence read-write. Ever.    |
| If you must access the original, be competent to explain it | And record what you did                      |
| Keep an audit trail                                         | Every command, every output, every hash      |
| The case officer owns the investigation                     | You report findings, you do not decide guilt |



What this costs you, and why it is worth paying. You hash the evidence when you receive it, you work on a copy, and you hash the copy at the end. If the two hashes match, your analysis is defensible. If you cannot produce that pair of hashes, your work is not evidence — it is opinion.

*What you will analyse. Every dataset in this guide is publicly published training material, bundled for education: NPS test disk images and the M57-Patents scenario from Digital Corpora, the EVTX-ATTACK-SAMPLES set, and the malware-traffic-analysis.net 2025-01-22 exercise. All are distributed for teaching. Never analyse evidence from a real case on a machine connected to anything you care about, and never upload case material to a public service.*

## 1. Why forensics is different from incident response

Incident response is about stopping something and getting systems back. Forensics is about proving what happened, to a standard that survives challenge.

| Question                                  | Who answers it    |
|-------------------------------------------|-------------------|
| How do we contain this?                   | Incident response |
| How do we recover service?                | Incident response |
| What exactly happened, and in what order? | Forensics         |
| Can we attribute it?                      | Forensics         |
| Will this hold up in a hearing?           | Forensics         |

Order of volatility drives everything. Some evidence disappears when you power the machine off; some survives. You collect in order of how quickly it vanishes:

| Evidence                                | Survives a reboot? | Collect |
|-----------------------------------------|--------------------|---------|
| RAM, running processes, open sockets    | No                 | First   |
| Network connections, routing tables     | No                 | First   |
| Temporary files, swap                   | Sometimes          | Second  |
| Disk contents                           | Yes                | Third   |
| Backups, archives, logs shipped off-box | Yes                | Last    |

This is why memory images matter so much. A disk image tells you what was installed. A memory image tells you what was running — including code that never touched the disk, which is exactly how modern malware prefers to operate.



## Where the evidence actually lives

Four sources. Each answers a different question about what happened.

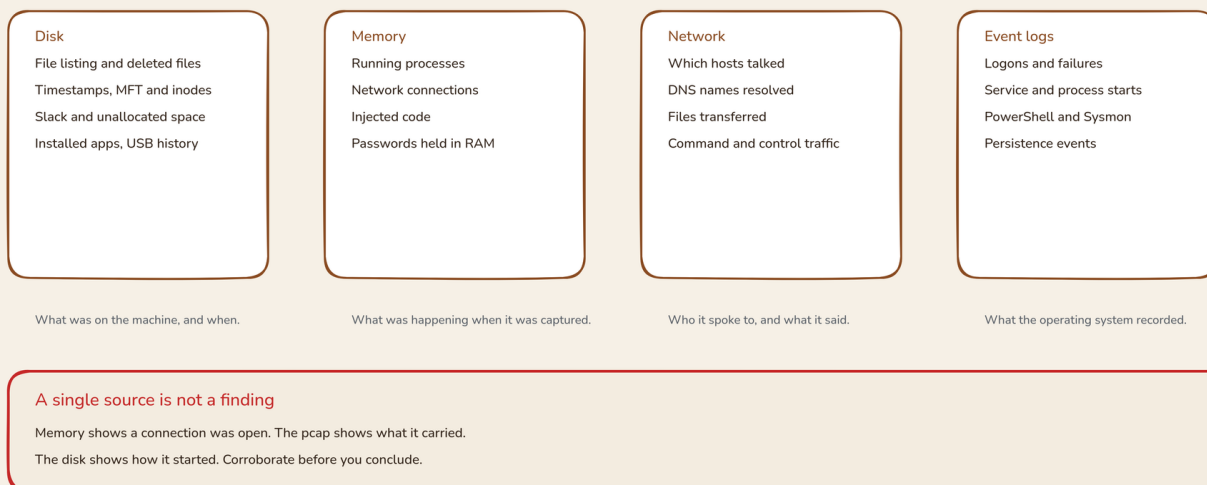


Figure 1: Where the evidence lives, and what each source answers.

Each source answers a different question, and none of them answers all of them. Disk tells you what was there. Memory tells you what was happening. Network tells you who it spoke to. Logs tell you what the operating system noticed. A conclusion that rests on one source is a lead; a conclusion that rests on two is a finding.

## 2. The workflow

Six phases. Integrity is established at the start and re-proven at the end.

### The triage workflow

Five phases. Each one answers a question the next phase depends on.

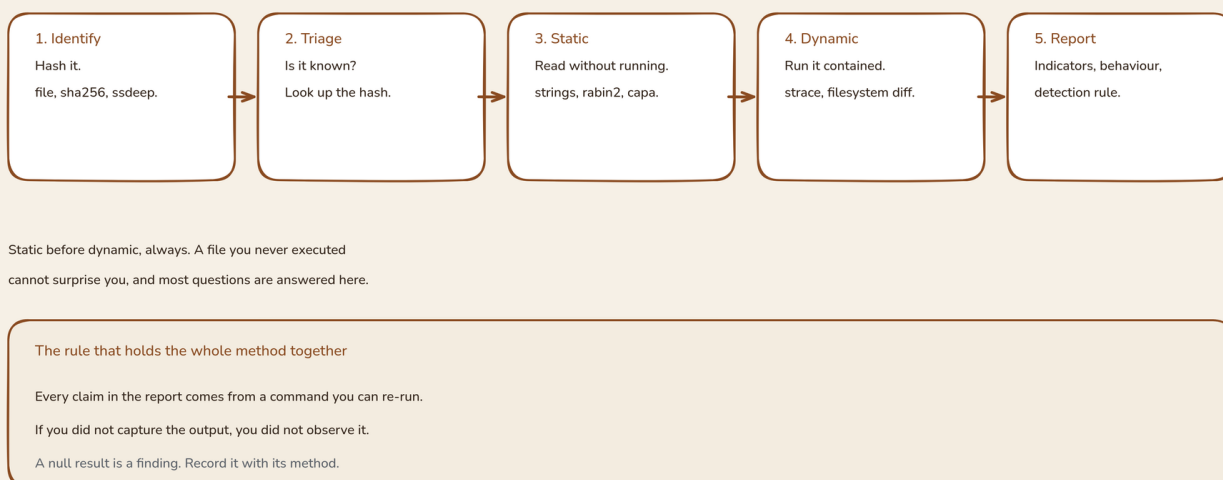


Figure 2: The six phase forensic workflow, from identification to report.



| Phase       | Question                     | Output                            |
|-------------|------------------------------|-----------------------------------|
| 1. Identify | What am I looking at?        | Type, size, format                |
| 2. Preserve | What is its hash?            | SHA-256 recorded, evidence sealed |
| 3. Acquire  | How do I get a working copy? | Image file, never the original    |
| 4. Verify   | Does the copy match?         | Hashes compared and recorded      |
| 5. Analyse  | What does it show?           | Artefacts, timeline, correlations |
| 6. Report   | What does the reader act on? | Findings with method for each     |

Phases 2 and 4 are the same hash, taken twice. That pair is the whole of the discipline. Everything else is tooling.

Analysis is the easy part. The tools are well documented and mostly run with one command. What separates a forensic analyst from someone who can type fls is knowing that the hash did not change between acquisition and analysis, and being able to show it.

### 3. Build the lab

The lab is a container with the tooling and the evidence mounted read-only. Analysis never touches the original.

#### Two containers, one air gap

Malware runs in a sandbox with no route to anything you care about.

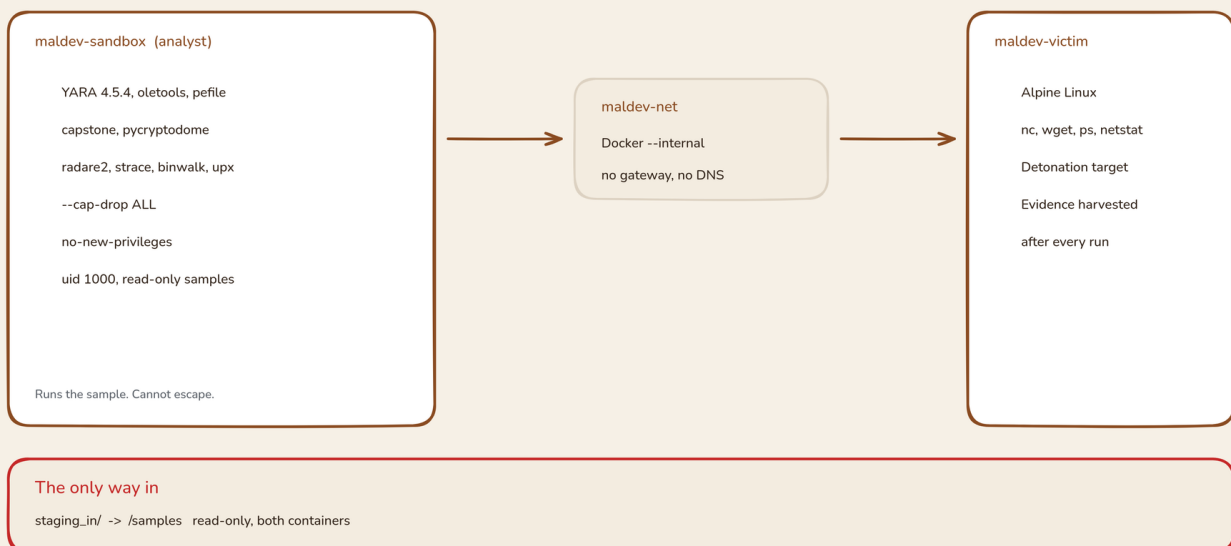


Figure 3: The forensic lab and its read-only evidence mount.

### Start and check the lab

```
docker start cpia-forensics
docker exec -it cpia-forensics bash

## the evidence is at /artifacts, mounted read-only
ls -la /artifacts/
```

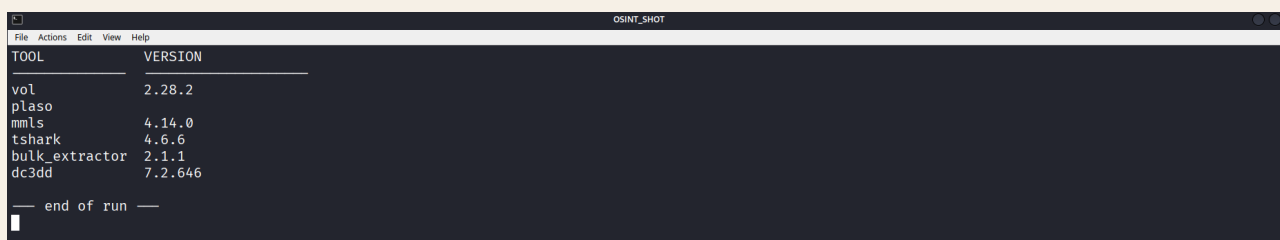


Why read-only is enforced by the mount, not by discipline. A careless `>` in a shell redirect against a mounted-writable evidence file destroys it silently. Mounting read-only makes the mistake impossible rather than merely discouraged.

## The toolchain

```
## one-line check that the whole stack is present
docker exec cpia-forensics bash -lc 'export PATH=$PATH:/usr/local/bin; \
  for t in vol fls icat mmls fsstat istat blkls bulk_extractor foremost \
    plaso log2timeline psort tshark evtx_dump ewfinfo dc3dd; do
    printf "%-16s %s\n" "$t" "${command -v $t || echo MISSING}"; done'
```

Tool versions in this lab, checked live and cross-referenced against each project's upstream release page:



| TOOL           | VERSION |
|----------------|---------|
| vol            | 2.28.2  |
| plaso          |         |
| mmls           | 4.14.0  |
| tshark         | 4.6.6   |
| bulk_extractor | 2.1.1   |
| dc3dd          | 7.2.646 |
| — end of run — |         |

*Screenshot: the version of each analysis tool installed in the lab.*

| Tool           | Version       | What it is for                        |
|----------------|---------------|---------------------------------------|
| Volatility 3   | 2.28.2        | Memory analysis                       |
| The Sleuth Kit | 4.12          | Disk and filesystem analysis          |
| plaso          | 20240418      | Super-timelines at scale              |
| bulk_extractor | 1.6           | Carving, email and feature extraction |
| tshark         | 4.x           | Network evidence from a pcap          |
| ewf-tools      | 20140816      | E01 (EnCase) image support            |
| dc3dd / dcfldd | 7.3.1 / 1.9.3 | Forensically sound acquisition        |
| foremost       | 1.5.7         | File carving by header and footer     |

Verify versions yourself rather than trusting a guide, including this one. Several of the fixes in the troubleshooting table are release-specific, and malfind has already moved once (see section 6.5).



## The evidence used in this guide

| Evidence                                  | Source                                                         | Format                                 |
|-------------------------------------------|----------------------------------------------------------------|----------------------------------------|
| jo-2009.mem (803 MB)                      | M57-Patents memory image, Digital Corpora                      | Windows XP SP3 raw memory              |
| ntfs1-gen1.E01                            | NPS NTFS test image, Digital Corpora                           | EnCase 6, NTFS, three test directories |
| gilbert1.E01                              | NPS "Gilbert" education scenario (LT H. T. Gilbert, USN, 2009) | EnCase 6, FAT16, macOS artefacts       |
| 6-fat-undel.dd                            | Sleuth Kit test data                                           | FAT16 with deleted files               |
| 2025-01-22-traffic-analysis-exercise.pcap | malware-traffic-analysis.net                                   | 26 MB real infection traffic           |
| evtx-attack-samples/                      | sbousseaden/EVTX-ATTACK-SAMPLES                                | 278 real EVTX files by ATT&CK; tactic  |

## 4. Phase 1 and 2 — Identify and preserve

### Hash it before you touch it

```
sha256sum ntfs1-gen1.E01
md5sum      ntfs1-gen1.E01
shasum      ntfs1-gen1.E01
ssdeep -b ntfs1-gen1.E01      # fuzzy hash: for similarity, not identity
```

```
OSINT_SHOT
File Actions Edit View Help
$ file ntfs1-gen1.E01 gilbert1.E01 6-fat-undel.dd
ntfs1-gen1.E01: EWF/Expert Witness/EnCase image file format
gilbert1.E01:   EWF/Expert Witness/EnCase image file format
6-fat-undel.dd: DOS/MBR boot sector, code offset 0x3c+2, OEM-ID "MSDOS5.0", sectors/cluster 2, reserved sectors 8, root entries 512, sectors 12033 (volumes ≤ 32 MB), Media descriptor 0xf8, sectors/FAT 24, sectors/track 63, heads 32, hidden sectors 63, serial number 0xc0fecdd11, unlabeled, FAT (16 bit)

$ sha256sum ntfs1-gen1.E01
ed26b63cb37350fba5aaf18f8c871515ff787db98bfa1c5d92b179185168dd6e  ntfs1-gen1.E01

— end of run —
```

Screenshot: file type identification and SHA-256 recording for the evidence.

What the run above shows. The file command identifies each item before you decide how to treat it:

| Evidence       | Identified as                      | What it tells you                                      |
|----------------|------------------------------------|--------------------------------------------------------|
| ntfs1-gen1.E01 | data                               | An image container, not a filesystem — unwrap it first |
| gilbert1.E01   | data                               | An image container — unwrap it, then the raw is FAT16  |
| 6-fat-undel.dd | FAT16 boot sector, label FAT_REC_1 | A bare filesystem, no partition table                  |

file returning data is the expected result for a forensic image. An E01 is a container with compression and metadata; it is not meant to look like a filesystem to file. Do not treat data as a problem.

Three hashes, not one. SHA-256 is the forensic standard, but MD5 and SHA-1 still appear in older case notes and hash databases such as NSRL. Record all three and you can compare against any source you are later handed.

Write the hash down before you do anything else. If acquisition and analysis happen in the same session with no recorded intermediate hash, you have no way to demonstrate integrity



afterwards.

## Read the image container

An E01 carries the acquisition metadata an examiner needs: who made it, when, with what, and over what media.

```
ewfinfo ntfs1-gen1.E01
```

```
OSINT_SHOT
File Actions Edit View Help
$ ewfinfo ntfs1-gen1.E01
ewfinfo 20140816

Acquiry information
Description:          converted from /corp/nps/drives/nps-2009-ntfs1/ntfs1-gen1.aff
Acquisition date:    Fri Feb 22 10:54:13 2013
System date:         Fri Feb 22 10:54:13 2013
Operating system used: Linux
Software version used: 20130120
Password:            N/A

EWF information
File format:         EnCase 6
Sectors per chunk:   64
Error granularity:   64
Compression method:  deflate
Compression level:   best compression

Media information
Media type:          fixed disk
Is physical:         yes
Bytes per sector:    512
Number of sectors:   1008896
Media size:          492 MiB (516554752 bytes)

Digest hash information
MD5:                 b1378f1f273add14411b32d5de7c5d5c

— end of run —
```

Screenshot: ewfinfo reporting the acquisition metadata stored in the E01 container.

What the run above shows. The container's own record of how it was captured:

| Field                 | Value                                | Why it matters                                        |
|-----------------------|--------------------------------------|-------------------------------------------------------|
| Description           | converted from<br>.../ntfs1-gen1.aff | The provenance chain, in the image itself             |
| Acquisition date      | Fri Feb 22 10:54:13<br>2013          | When the image was taken                              |
| Operating system used | Linux                                | The platform that acquired it                         |
| Software version used | 20130120                             | The exact acquisition tool version                    |
| File format           | EnCase 6                             | So you know what can read it                          |
| Compression           | deflate, best<br>compression         | Explains the size, and rules out corruption           |
| Bytes per sector      | 512                                  | Needed for every offset calculation downstream        |
| Number of sectors     | 1008896                              | The image is $1008896 \times 512 = 516,554,752$ bytes |

This metadata is evidence in its own right. An image with no acquisition record is an image whose provenance cannot be attested. If ewfinfo shows a blank examiner or an impossible date, that is a finding worth reporting.

Note the format you are holding, because it decides your next step. E01 is compressed, so the Sleuth Kit cannot index it directly. Section 5.1 covers the conversion.



## 5. Disk and filesystem analysis

### 5.1 Getting to the filesystem

You move down a stack: the image container, then the partition, then the filesystem, then the file. Skipping a layer produces confident nonsense.

#### Anatomy of a disk image

You move down this stack: device, then partition, then filesystem, then file.

|                 |                                        |                                 |
|-----------------|----------------------------------------|---------------------------------|
| Disk image      | A byte-for-byte copy of the medium.    | E01, raw, AFF                   |
| Partition table | Where each filesystem starts and ends. | mmls reads this                 |
| Partition       | One region of the disk.                | Sector offset, for example 2048 |
| Filesystem      | How files are stored in that region.   | fsstat reads this               |
| NTFS metadata   | The \$MFT, one record per file.        | istat reads one record          |
| ext metadata    | Inode table and directories.           | istat reads one inode           |
| File content    | The bytes themselves.                  | icat extracts one               |
| Slack space     | Leftover bytes after the file ends.    | Where deleted data survives     |

Figure 4: Anatomy of a disk image, from container to slack space.

On this evidence, mmls finds no partition table — and that is correct, not a failure. The NPS test images are bare filesystems with no partition wrapper, so there are no partitions to list.

E01 must be unwrapped before TSK can read it. There are two routes, and one of them does not work in this lab:

| Route          | Command                                            | Status here                                               |
|----------------|----------------------------------------------------|-----------------------------------------------------------|
| FUSE mount     | <code>ewfmount image.E01 /mnt/ewf</code>           | <b>Fails</b> — no <code>/dev/fuse</code> in the container |
| Convert to raw | <code>ewfexport -t /out -f raw -u image.E01</code> | <b>Works</b> — use this                                   |

```
ewfmount: fuse: device /dev/fuse not found. Kernel module not loaded?
```

Learn the ewfexport argument quirk, because it costs time. The output path is set with `-t`, and the file is named after the input image, not the directory you hoped for. An early attempt with `-t .` and then a search for `ntfs1-gen1.raw` found nothing, because the tool had written `raw.raw` — named after the segment it was told to write.

```
## -t = output directory, -f = format, -u = unattended
ewfexport -q -t /path/to/outdir -f raw -u ntfs1-gen1.E01
## writes /path/to/outdir/<segment-name>.raw -- check the name it chose
ls -la /path/to/outdir/
```

The MD5 line ewfexport prints is your conversion check. It hashes the data as it writes, giving you a second integrity reference for the raw copy.



## 5.2 Read the filesystem

```
fsstat raw.raw
```

```
OSINT_SHOT
File Actions Edit View Help
Volume Serial Number: DA5048E85048CCC7
OEM Name: NTFS
Volume Name: NTFS1
Version: Windows XP

METADATA INFORMATION
-----
First Cluster of MFT: 336298
First Cluster of MFT Mirror: 504447
Size of MFT Entries: 1024 bytes
Size of Index Records: 4096 bytes
Range: 0 - 48
Root Directory: 5

CONTENT INFORMATION
-----
Sector Size: 512
Cluster Size: 512
Total Cluster Range: 0 - 1008894
Total Sector Range: 0 - 1008894

$AttrDef Attribute Values:
$STANDARD_INFORMATION (16)  Size: 48-72  Flags: Resident
$ATTRIBUTE_LIST (32)       Size: No Limit  Flags: Non-resident
$FILE_NAME (48)            Size: 68-578  Flags: Resident,Index
$OBJECT_ID (64)            Size: 0-256   Flags: Resident
$SECURITY_DESCRIPTOR (80)   Size: No Limit  Flags: Non-resident
$VOLUME_NAME (96)          Size: 2-256   Flags: Resident
$VOLUME_INFORMATION (112)   Size: 12-12   Flags: Resident
$DATA (128)                Size: No Limit  Flags:
$INDEX_ROOT (144)          Size: No Limit  Flags: Resident
$INDEX_ALLOCATION (160)     Size: No Limit  Flags: Non-resident
$BITMAP (176)              Size: No Limit  Flags: Non-resident
$REPARSE_POINT (192)       Size: 0-16384  Flags: Non-resident
$EA_INFORMATION (208)      Size: 8-8     Flags: Resident
$EA (224)                  Size: 0-65536  Flags:
$LOGGED_UTILITY_STREAM (256) Size: 0-65536  Flags: Non-resident
```

Screenshot: fsstat reporting the NTFS filesystem parameters.

What the run above shows. The parameters every later offset depends on:

| Field                | Value            | Reading                                        |
|----------------------|------------------|------------------------------------------------|
| File System Type     | NTFS             | Confirms the boot sector                       |
| Volume Serial Number | DA5048E85048CCC7 | Identifies this filesystem uniquely            |
| Volume Name          | NTFS1            | Matches the evidence description               |
| Version              | Windows XP       | The era of the filesystem                      |
| First Cluster of MFT | 336298           | Where the file index begins                    |
| Size of MFT Entries  | 1024 bytes       | <b>How far to jump to read the next record</b> |
| Sector Size          | 512              | Matches the E01 metadata                       |
| Cluster Size         | 512              | One sector per cluster                         |
| Total Cluster Range  | 0 - 1008894      | The addressable space                          |

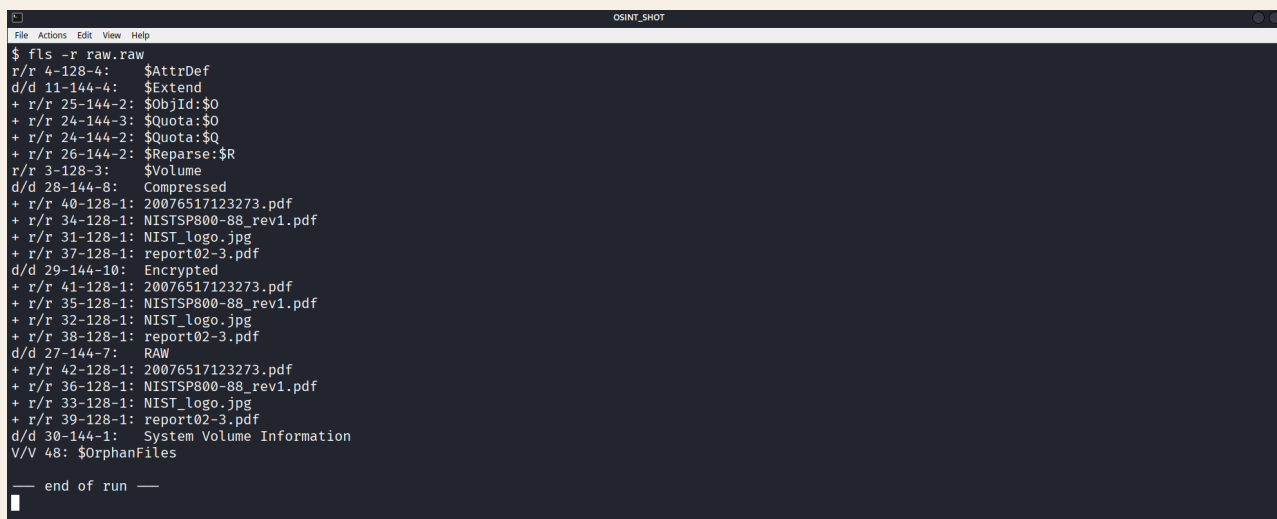
Size of MFT Entries: 1024 is the number to remember. NTFS stores one 1024-byte record per file. When you read record n, you are reading at a predictable offset, which is how istat can jump straight to a file without walking directories.

On a damaged filesystem, fsstat is where you find out. Run it first on any suspect image: if the boot sector does not parse, no other tool will work, and you have a carving job rather than a filesystem job.



## 5.3 List the files

```
fls raw.raw          # the root directory, one level
fls -r raw.raw        # recursive
fls -r -d raw.raw     # deleted entries only
fls -l raw.raw        # long format, with times
```



Screenshot: fls listing the filesystem contents including the user directories.

What the run above shows. The NPS test image carries three parallel directories — RAW, Compressed and Encrypted — each holding the same files. That structure exists specifically to test whether your tools handle NTFS compression and EFS encryption.

| Entry                     | Type | Reading                              |
|---------------------------|------|--------------------------------------|
| Compressed/               | d/d  | One of the three test directories    |
| Encrypted/                | d/d  | EFS-encrypted files                  |
| RAW/                      | d/d  | Uncompressed baseline for comparison |
| .../20076517123273.pdf    | r/r  | A public PDF used as filler content  |
| .../NISTSP800-88_rev1.pdf | r/r  | NIST media-sanitisation guidance     |
| System Volume Information | d/d  | Windows restore points               |

Read the leading field carefully — it is the file's type and state:

```
d/d 11-144-4: $Extend
r/r 40-128-1: 20076517123273.pdf
r/r * 4:      _ragl.dat
```

| Field    | Meaning                                                          |
|----------|------------------------------------------------------------------|
| r/r      | A regular file, allocated                                        |
| d/d      | A directory, allocated                                           |
| r/r *    | <b>A deleted regular file</b> — the asterisk is the deleted flag |
| 11-144-4 | The metadata address: MFT entry, attribute type, attribute id    |
| -128-1   | Attribute type 128 is \$DATA — the file content itself           |



The asterisk is the single most important character in TSK output. Everything else is bookkeeping; the asterisk tells you the entry is no longer allocated and its data may still be recoverable.

The three-number address is not decoration. 40-128-1 means MFT record 40, attribute type 128 (\$DATA), attribute id 1. That address is the input to istat and icat.

## 5.4 Recover deleted files

Deletion removes the directory entry, not the data. Until the space is overwritten, the content is still there.

```
fls -r -d 6-fat-undel.dd
```

```
File Actions Edit View Help
$ fls -r -d 6-fat-undel.dd
r/r * 4:      _rag1.dat
r/r * 5:      _rag2.dat
r/r * 6:      _ing.dat
r/r * 7:      _ult1.dat
d/d * 8:      _ir1
d/d * 869:    _ir1/dir2
r/r * 965:    _ir1/dir2/frag3.dat
r/r * 870:    _ir1/mult2.dat
d/d * 11:     System Volume Information
d/d * 905:    System Volume Information/_restore{A25F48CA-6632-4143-8EF8-3586A84AB5AF}
d/d * 933:    System Volume Information/_restore{A25F48CA-6632-4143-8EF8-3586A84AB5AF}/_P1
r/r * 965:    System Volume Information/_restore{A25F48CA-6632-4143-8EF8-3586A84AB5AF}/_P1/frag3.dat
— end of run —
```

Screenshot: fls showing deleted FAT16 entries recoverable from the image.

What the run above shows. Six deleted files and directories on a FAT16 image, all marked with the asterisk:

| Deleted entry        | Type                                 |
|----------------------|--------------------------------------|
| _rag1.dat, _rag2.dat | Files written to force fragmentation |
| _ing.dat, _ult1.dat  | More fragments of the same file      |
| _ir1/ and _ir1/dir2/ | <b>Deleted directories</b>           |
| _ir1/dir2/frag3.dat  | A file inside a deleted directory    |

The leading underscore is the deletion marker in FAT. FAT stores the first character of a filename in the directory entry; deletion overwrites that character with 0xE5. Windows renders 0xE5 as \_. So \_rag1.dat was once Xrag1.dat or similar — the real first character is permanently lost. Note that in your report rather than guessing.

A file inside a deleted directory can still be recoverable. Deleting \_ir1 did not erase frag3.dat inside it. This is why "I deleted the folder" is not a sanitisation claim, and why NIST SP 800-88 exists.

Recover the content:

```
## TSK route: pipe the file body out by metadata address
icat 6-fat-undel.dd 4 > recovered_rag1.dat

## Verify what you recovered
file recovered_rag1.dat
sha256sum recovered_rag1.dat
```

Hash every file you recover. A recovered file is a new artefact, and its hash is what ties it to your report.



## 5.5 Read the timestamps — and know what they mean

```
istat raw.raw 5
```

```
OSINT_SHOT
File Actions Edit View Help
$ istat raw.raw 5
MFT Entry Header Values:
Entry: 5      Sequence: 5
$LogFile Sequence Number: 2112929
Allocated Directory
Links: 1

$STANDARD_INFORMATION Attribute Values:
Flags: Hidden, System
Owner ID: 0
Security ID: 0 ( )
Created:      2008-12-31 22:44:02.215500000 (UTC)
File Modified: 2008-12-31 22:45:47.543625000 (UTC)
MFT Modified:  2008-12-31 22:45:47.543625000 (UTC)
Accessed:      2008-12-31 22:45:50.309250000 (UTC)

$FILE_NAME Attribute Values:
Flags: Directory, Hidden, System
Name: .
Parent MFT Entry: 5      Sequence: 5
Allocated Size: 0      Actual Size: 0
Created:      2008-12-31 22:44:02.215500000 (UTC)
File Modified: 2008-12-31 22:44:02.215500000 (UTC)
MFT Modified:  2008-12-31 22:44:02.215500000 (UTC)
Accessed:      2008-12-31 22:44:02.215500000 (UTC)

Attributes:
Type: $STANDARD_INFORMATION (16-0) Name: N/A Resident size: 48
Type: $FILE_NAME (48-1) Name: N/A Resident size: 68
Type: $SECURITY_DESCRIPTOR (80-2) Name: N/A Non-Resident size: 4172 init_size: 4172
504534 504535 504536 504537 504538 504539 504540 504541
504542
Type: $INDEX_ROOT (144-6) Name: $I30 Resident size: 56
Type: $INDEX_ALLOCATION (160-8) Name: $I30 Non-Resident size: 4096 init_size: 4096
504526 504527 504528 504529 504530 504531 504532 504533
Type: $BITMAP (176-7) Name: $I30 Resident size: 8

— end of run —
```

Screenshot: *istat* reporting the MFT record and the four MAC timestamps for a file.

What the run above shows. One MFT record, decoded:

| Field               | Value                               |
|---------------------|-------------------------------------|
| Entry               | 5                                   |
| Sequence            | 5                                   |
| Allocated Directory | The entry is live                   |
| Links               | 1                                   |
| Flags               | Hidden, System                      |
| Created             | 2008-12-31 22:44:02.215500000 (UTC) |
| File Modified       | 2008-12-31 22:45:47.543625000 (UTC) |
| MFT Modified        | 2008-12-31 22:45:47.543625000 (UTC) |
| Accessed            | 2008-12-31 22:45:50.309250000 (UTC) |

NTFS gives you four timestamps, and they are stored in the MFT, not the file. That is why they survive operations that would otherwise update them, and why they are among the most valuable artefacts on a Windows system.



MAC times, and why one timestamp is never enough

Every file carries several times. Reading them together is the skill.

**M modified**

When the file content last changed.

A ransomware run shows as a burst of M times.

**A accessed**

When the file was last read.

Often disabled on Windows, so use with care.

**C changed**

When the metadata changed, such as permissions.

The Linux ctime. Not the creation time.

**B born**

When the file was created.

Only present on some filesystems.

**The ctime trap**

On Linux the fourth letter is not creation time, it is inode change time. Stating it as creation time in a report is a factual error an examiner will be cross-examined on.

Figure 5: MAC times explained, and the ctime trap.

The ctime trap, which costs examiners their credibility. On Linux filesystems, MACB means modified, accessed, changed, born. The C is inode change time, not creation time. Writing "created" against a ctime in a report is a factual error, and it is one a defence examiner will find.

| Letter | Linux meaning                 | Windows meaning                        |
|--------|-------------------------------|----------------------------------------|
| M      | Content modified              | Content modified                       |
| A      | Last read                     | Last read (often disabled — see below) |
| C      | <b>Inode change time</b>      | MFT record changed                     |
| B      | Birth (some filesystems only) | File created                           |

Windows "accessed" time is unreliable, and you must say so. NtfsDisableLastAccessUpdate is enabled by default on modern Windows, so the A time can be days or weeks stale. An allegation built on an access time alone is a weak allegation. State the setting in your report.

The order of the times tells a story. On this record, Created → Modified → Accessed runs forward normally. In ransomware, you see a burst of Modified times across thousands of files within seconds — which is unmistakable and is why timelines matter.

5.6 Carve what the filesystem cannot show

Carving recovers files by their content, with no filesystem help. It works on deleted data, unallocated space, and damaged or partially overwritten filesystems where the directory structure no longer tells you what was there.

```
bulk_extractor -q -o out raw.raw
ls out/
```



```
OSINT_SHOT
File Actions Edit View Help
$ bulk_extractor -q -o out ../raw.raw
exit=0

total 204
drwxr-xr-x 3 root root 4096 Oct  2 21:41 .
drwxr-xr-x 3 root root 4096 Oct  2 21:41 ..
-rw-r--r-- 1 root root  0 Oct  2 21:41 aes_keys.txt
-rw-r--r-- 1 root root  0 Oct  2 21:41 alerts.txt
-rw-r--r-- 1 root root  0 Oct  2 21:41 ccn_histogram_1.txt
-rw-r--r-- 1 root root  0 Oct  2 21:41 ccn_histogram.txt
-rw-r--r-- 1 root root  0 Oct  2 21:41 ccn_track2_histogram_1.txt
-rw-r--r-- 1 root root  0 Oct  2 21:41 ccn_track2_histogram.txt
-rw-r--r-- 1 root root  0 Oct  2 21:41 ccn_track2.txt
-rw-r--r-- 1 root root  0 Oct  2 21:41 ccn.txt
-rw-r--r-- 1 root root  0 Oct  2 21:41 domain_histogram_1.txt
-rw-r--r-- 1 root root 1248 Oct  2 21:41 domain_histogram.txt
-rw-r--r-- 1 root root 10690 Oct  2 21:41 domain.txt

— end of run —
```

Screenshot: `bulk_extractor` carving features out of the raw image.

What the run above shows. `bulk_extractor` writes a set of feature files rather than individual carved documents:

| Output file            | Contains                                  |
|------------------------|-------------------------------------------|
| report.xml             | Summary of everything found               |
| url.txt                | Every URL recovered from the image        |
| domain.txt             | Every domain name                         |
| email.txt              | Every email address                       |
| pcap.txt               | <b>Network packets found in the image</b> |
| telephone.txt, ccn.txt | Phone numbers, card numbers               |

`pcap.txt` is the one that surprises people. Images routinely contain enough fragments of network traffic — cached pages, logs, configuration — for `bulk_extractor` to reassemble usable packets. It is often the cheapest way to get network context from a disk-only case.

Carving has no filename and no timestamp. Every carved file is anonymous. You recover that a file existed, not what it was called or when it was written. Say so.

Always carve unallocated space, not the whole image. `blkls raw.raw > unallocated.bin` extracts only the space not currently in use, which is where deleted data lives and which is a fraction of the size.

```
blkls raw.raw > unallocated.bin
foremost -i unallocated.bin -o foremost_out # recover by file type
```

## 5.7 A worked example: a real image with a hidden file

`gilbert1.E01` from the NPS education corpus is a good test because it is a realistic scenario rather than a synthetic one. The student brief is blunt about the task:

*There are 3 folders on the disk image: Docs, Pics and Copy of Pics. Carefully look at the files to determine if there is something suspicious. Try to recover any deleted files and analyze them for clues.*

```
ewfexport -q -t gtest -f raw -u gilbert1.E01 # unwrap first – TSK cannot read E01
fsstat -- gtest/../raw
fls -r -d -- gtest/../raw
```

What the run above shows. The image is a FAT16 volume:



| Field                      | Value      | Reading                              |
|----------------------------|------------|--------------------------------------|
| File System Type           | FAT16      |                                      |
| OEM Name                   | BSD 4.4    | Created on a Mac, not Windows        |
| Volume Label               | DISK IMAGE |                                      |
| Sectors before file system | 0          | No partition table — bare filesystem |

```
File Actions Edit View Help
$ fsstat -- ..raw
FILE SYSTEM INFORMATION

File System Type: FAT16

OEM Name: BSD 4.4
Volume ID: 0x109315f9
Volume Label (Boot Sector): DISK IMAGE
Volume Label (Root Directory):
File System Type Label: FAT16

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 127999

— end of run —
```

Screenshot: fsstat reading the gilbert1 FAT16 filesystem, note the BSD 4.4 OEM name.

The macOS artefacts are the giveaway. The root contains .Trashes, .fseventsd, .TemporaryItems and .\_-prefixed AppleDouble files. Know your macOS artefacts, because an examiner who reads .Trashes as suspicious is about to make a claim about a normal Mac housekeeping folder.

The deleted entries are where the scenario lives:

```
fls -r -d -- gtest/..raw
```

```
File Actions Edit View Help
$ fls -r -d -- ..raw
d/d * 6:      _RASHE~1.OHS
d/d * 10:     .fseventsd
r/r * 903:    .fseventsd/fseventsd-uuid
r/r * 906:    .fseventsd/000000000027d5ff
r/r * 1287:   .TemporaryItems/folders.501/TemporaryItems/EFTFile1.sparseimage
r/r * 1290:   .TemporaryItems/folders.501/TemporaryItems/._EFTFile1.sparseimage
r/r * 1292:   .TemporaryItems/folders.501/TemporaryItems/EFTFile2
r/r * 494922: For Image/docs/video0001 copy.3g2
r/r * 494925: For Image/docs/video0001 copy 2.3g2
r/r * 494928: For Image/docs/video0001 copy 3.3g2
r/r * 494931: For Image/docs/video0001 copy 4.3g2
r/r * 494934: For Image/docs/video0001 copy 5.3g2
r/r * 494937: For Image/docs/video0001 copy 6.3g2
r/r * 494940: For Image/docs/._video0001 copy 6.3g2

— end of run —
```

Screenshot: the deleted entries in the gilbert1 image, including the sparseimage and six video copies.

| Deleted entry                                                   | Reading                        |
|-----------------------------------------------------------------|--------------------------------|
| .TemporaryItems/folders.501/TemporaryItems/EFTFile1.sparseimage | A sparse disk image, deleted   |
| EFTFile1.sparseimage and ._EFTFile1.sparseimage                 | Its resource fork              |
| EFTFile2                                                        | A companion file               |
| For Image/docs/video0001 copy.3g2 and 5 more copies             | Six near-duplicate video files |
| .fseventsd/fseventsd-uuid, 000000000027d5ff                     | The Mac filesystem event log   |

EFTFile1.sparseimage in a TemporaryItems folder is the suspicious object. A sparse image is a mountable virtual disk, so recovering it and looking inside is the point of the exercise — and the six video0001 copy variants are the decoys that make the folder hard to read at a glance.



Recovering it is where the exercise gets honest:

```
icat -- gtest/..raw 1287 > recovered.sparseimage      # the metadata address from fls -d
file recovered.sparseimage
wc -c recovered.sparseimage
```

What the run above shows is a partial recovery, and that is the real lesson:



Screenshot: the icat recovery returning 2048 bytes for the sparseimage and 0 bytes for EFTFile2.

| Target                    | icat result                | What it means                                 |
|---------------------------|----------------------------|-----------------------------------------------|
| 1287 EFTFile1.sparseimage | 2048 bytes, file says data | <b>One cluster only</b> — the chain is broken |
| 1292 EFTFile2             | <b>0 bytes</b>             | Nothing recoverable at all                    |

The directory entry survived; the data did not. On FAT, deleting a file frees its cluster chain. A freed chain is only recoverable while those clusters are still untouched — so a deleted 40 MB file can come back as one 2 KB fragment, because that is all that is left.

This is the single most important thing to understand about deleted-file recovery. An entry appearing in fls -d proves the metadata is intact. It says nothing about whether the content is. Always verify what you actually recovered with file, wc -c, and a hash — never report a recovery on the strength of the listing alone.

When icat gives you a fragment, carve. Carving ignores the cluster chain entirely and scans the raw bytes for content, which is how you find data whose metadata no longer points at it:

```
blkls -- gtest/..raw > gilbert_unalloc.bin
foremost -i gilbert_unalloc.bin -o gilbert_carved
```

The two macOS artefacts confirm this is a Mac-created volume: .Trashes and .fseventsd are normal housekeeping folders, and the .\_-prefixed files are AppleDouble resource forks. Know your platform artefacts — an examiner who treats .Trashes as suspicious is about to make a claim about a routine Mac folder.

Two lessons this image teaches that a synthetic sample cannot.

1. A healthy filesystem can still hide something. Nothing here is corrupt — the evidence is deleted, not damaged. Recovery, not repair, is the task.
2. Deleted does not mean recoverable. Both targets came back as fragments or nothing. The exercise is to establish what can be recovered and be precise about what cannot.

5.8 Build a timeline

A file listing shows you what. A timeline shows you in what order, which is usually the question that matters.



```
## TSK: generate a body file, then render a timeline
fls -m -r raw.raw > body.txt
mactime -b body.txt
mactime -b body.txt -d          # date range in the first column
```

```
$ fls -m -r raw.raw > body.txt && mactime -b body.txt -d
Wed Dec 31 2008 22:44:02 49152 macb r/rr-xr-xr-x 0 0 0-128-1 -r/$MFT
74 macb r/rr-xr-xr-x 0 0 0-48-3 -r/$MFT ($FILE_NAME)
4096 macb r/rr-xr-xr-x 0 0 1-128-1 -r/$MFTMirr
82 macb r/rr-xr-xr-x 0 0 1-48-2 -r/$MFTMirr ($FILE_NAME)
131072 macb r/rr-xr-xr-x 0 0 10-128-1 -r/$UpCase
80 macb r/rr-xr-xr-x 0 0 10-48-2 -r/$UpCase ($FILE_NAME)
344 macb d/dr-xr-xr-x 0 0 11-144-4 -r/$Extend
80 macb d/dr-xr-xr-x 0 0 11-48-3 -r/$Extend ($FILE_NAME)
4685824 macb r/rr-xr-xr-x 0 0 2-128-1 -r/$LogFile
82 macb r/rr-xr-xr-x 0 0 2-48-2 -r/$LogFile ($FILE_NAME)
0 macb r/rr-xr-xr-x 48 0 3-128-3 -r/$Volume
80 macb r/rr-xr-xr-x 48 0 3-48-1 -r/$Volume ($FILE_NAME)
2560 macb r/rr-xr-xr-x 48 0 4-128-4 -r/$AttrDef
82 macb r/rr-xr-xr-x 48 0 4-48-2 -r/$AttrDef ($FILE_NAME)
126112 macb r/rr-xr-xr-x 0 0 6-128-1 -r/$Bitmap
80 macb r/rr-xr-xr-x 0 0 6-48-2 -r/$Bitmap ($FILE_NAME)

— end of run —
```

Screenshot: the mactime timeline rendered from the filesystem body file.

What the run above shows. Every timestamped action, in one chronological sequence:

| Column    | Meaning                                                 |
|-----------|---------------------------------------------------------|
| Date      | The timestamp, grouped by day                           |
| Size      | File size                                               |
| Type      | mac flag — which of the three times this row represents |
| Mode      | Permission bits                                         |
| UID / GID | Owner                                                   |
| Meta      | The metadata address                                    |
| Name      | The file                                                |

The mac type column is what makes a timeline readable. Each file produces up to three rows — one for modified, one for accessed, one for changed — so a single file can appear three times. Do not read that as three separate events. Filter by type when the question is about modification.

mactime is fine for thousands of events. For millions, use plaso. The TSK route re-reads the whole filesystem for every source you add; plaso parses each source once into a single store, then sorts.

```
log2timeline.py --storage-file timeline.plaso raw.raw
psort.py -o l2tcsv -w timeline.csv timeline.plaso
```

--storage-file is required in the current plaso. Earlier versions accepted -o <file> for the output, and that form now fails with "unrecognized arguments" — a change that breaks most older tutorials on the internet.

| Tool    | Scale               | Use when                     |
|---------|---------------------|------------------------------|
| mactime | Thousands of events | One filesystem, quick answer |
| plaso   | Millions of events  | Many sources, or court-scale |



## 6. Memory analysis

Memory answers the question the disk cannot: what was actually running. Code that never touched disk leaves no disk artefact at all.

Memory analysis: the plugins that answer the questions

Volatility 3. Point it at the image, name the plugin, read the table.

|                 |                               |                                 |
|-----------------|-------------------------------|---------------------------------|
| windows.info    | What OS is this image?        | Run it first, always.           |
| windows.pslist  | What was running?             | Follow the parent process id.   |
| windows.cmdline | What command started it?      | Where the arguments hide.       |
| windows.netscan | What was it connected to?     | The live network state.         |
| windows.malfind | Is there injected code?       | Executable memory with no file. |
| windows.svcscan | What services were installed? | How it survives a reboot.       |

Symbols resolve symbol names. An unresolved image cannot be read.

Figure 6: The Volatility plugins that answer the questions.

### 6.1 Identify the image first

```
vol -f jo-2009.mem windows.info
```

```
OSINT_SHOT
File Actions Edit View Help
$ vol -f jo-2009.mem windows.info
Volatility 3 Framework 2.28.2

Variable      Value
-----
Kernel Base   0x804d7000
DTB           0x39000
Symbols file:  file:///opt/pipx/venvs/volatility3/lib/python3.14/site-packages/volatility3/symbols/windows/ntoskrnl.pdb/1B2D0DFE2FB942758D615C901BE04692-2.json.xz
Is64Bit       False
IsPAE         False
layer_name    0 WindowsIntel
memory_layer  1 FileLayer
KdDebuggerDataBlock 0x8054cde0
NTBuildLab    2600.xpsp_sp3_gdr.090804-1435
CSDVersion    3
KdVersionBlock 0x8054cdb8
Major/Minor   15, 2600
MachineType   332
KeNumberProcessors 1
SystemTime    2009-11-17 00:22:31+00:00
NtSystemRoot  C:\WINDOWS
NtProductType NtProductWinNt
NtMajorVersion 5

— end of run —
```

Screenshot: Volatility identifying the memory image as Windows XP SP3.

What the run above shows. The facts that decide which plugins can work:

| Variable    | Value      | Reading                                               |
|-------------|------------|-------------------------------------------------------|
| Kernel Base | 0x804d7000 | A 32-bit kernel address                               |
| Is64Bit     | False      | <b>32-bit — so 64-bit-only plugins will not apply</b> |



| Variable     | Value                          | Reading                             |
|--------------|--------------------------------|-------------------------------------|
| IsPAE        | False                          | No physical address extension       |
| Major/Minor  | 15.2600                        | Windows XP SP3                      |
| NTBuildLab   | 2600.xpsp_sp3_gdr.090804-1435  | Service Pack 3, patched August 2009 |
| NtSystemRoot | C:\WINDOWS                     | Standard install                    |
| SystemTime   | <b>2009-11-17 00:22:31 UTC</b> | <b>When the image was captured</b>  |
| Symbols      | ntoskrnl.pdb/1B2D0DFE..json.xz | The symbol file resolved            |

SystemTime is the anchor for the whole investigation. It is the moment of capture. Every timestamp you report should be understood relative to it.

Symbols resolved successfully, and that is not automatic. Volatility 3 downloads symbol tables from symbols.volatilityfoundation.org on first use and caches them. If you are working offline, that download fails and no plugin will run. Check windows.info shows a resolved symbol path before you run anything else — this is the single most common reason a memory analysis produces nothing.

```
## pre-fetch symbols while you have connectivity, then work offline
vol -f image.mem windows.info      # downloads and caches
ls ~/.cache/volatility3/symbols/   # confirm they are cached
```

## 6.2 What was running

```
vol -f jo-2009.mem windows.pslist
```

OSINT\_SHOT

File Actions Edit View Help

\$ vol -f jo-2009.mem windows.pslist  
Volatility 3 Framework 2.28.2

| PID  | PPID | ImageFileName | Offset(V)  | Threads | Handles | SessionId | Wow64                          | CreateTime                     | ExitTime | File output |  |
|------|------|---------------|------------|---------|---------|-----------|--------------------------------|--------------------------------|----------|-------------|--|
| 188  | 472  | hkcmd.exe     | 0x82ef4910 | 2       | 100     | 0         | False                          | 2009-11-13 04:49:19.000000 UTC | N/A      | Disabled    |  |
| 3656 | 472  | soffice.exe   | 0x82909020 | 1       | 20      | 0         | False                          | 2009-11-16 18:46:13.000000 UTC | N/A      | Disabled    |  |
| 2392 | 3656 | soffice.bin   | 0x8290b020 | 21      | 554     | 0         | False                          | 2009-11-16 18:46:13.000000 UTC | N/A      | Disabled    |  |
| 3860 | 2392 | cmd.exe       | 0x826d45c0 | -       | 0       | False     | 2009-11-16 18:47:16.000000 UTC | 2009-11-16 18:47:20.000000 UTC | Disabled |             |  |
| 2728 | 2392 | cmd.exe       | 0x82f20da0 | -       | 0       | False     | 2009-11-16 18:47:21.000000 UTC | 2009-11-16 18:47:21.000000 UTC | Disabled |             |  |
| 3968 | 2392 | cmd.exe       | 0x826777a8 | -       | 0       | False     | 2009-11-16 18:47:21.000000 UTC | 2009-11-16 18:47:22.000000 UTC | Disabled |             |  |
| 1188 | 2392 | cmd.exe       | 0x8295fda0 | -       | 0       | False     | 2009-11-16 18:47:22.000000 UTC | 2009-11-16 18:47:22.000000 UTC | Disabled |             |  |
| 3328 | 2392 | cmd.exe       | 0x82cb1da0 | -       | 0       | False     | 2009-11-16 18:47:22.000000 UTC | 2009-11-16 18:47:23.000000 UTC | Disabled |             |  |
| 3124 | 472  | firefox.exe   | 0x826be020 | 0       | -       | False     | 2009-11-16 18:47:24.000000 UTC | 2009-11-16 19:32:51.000000 UTC | Disabled |             |  |
| 3996 | 472  | firefox.exe   | 0x826ddda0 | 0       | -       | False     | 2009-11-16 19:34:03.000000 UTC | 2009-11-17 00:10:18.000000 UTC | Disabled |             |  |
| 2148 | 3344 | soffice.bin   | 0x82648da0 | 0       | -       | False     | 2009-11-17 00:08:10.000000 UTC | 2009-11-17 00:08:16.000000 UTC | Disabled |             |  |
| 3096 | 472  | cmd.exe       | 0x82735020 | 1       | 33      | 0         | False                          | 2009-11-17 00:21:37.000000 UTC | N/A      | Disabled    |  |
| 3700 | 3096 | cmd.exe       | 0x82f045d0 | 1       | 24      | 0         | False                          | 2009-11-17 00:22:29.000000 UTC | N/A      | Disabled    |  |

— end of run —

Screenshot: Volatility listing the processes that were running at capture time.

What the run above shows. Every process, with its parent, timing and state:

| PID                    | PPID | Process     | Created                | Reading                              |
|------------------------|------|-------------|------------------------|--------------------------------------|
| 3656                   | 472  | soffice.exe | 2009-11-16 18:46:13    | OpenOffice launched                  |
| 2392                   | 3656 | soffice.bin | 2009-11-16 18:46:13    | <b>The real OpenOffice process</b>   |
| 3860                   | 2392 | cmd.exe     | 2009-11-16 18:47:16    | <b>A shell spawned by OpenOffice</b> |
| 2728                   | 2392 | cmd.exe     | 2009-11-16 18:47:21    | Another                              |
| 3968,<br>1188,<br>3328 | 2392 | cmd.exe     | 2009-11-16 18:47:21-22 | <b>Four shells in six seconds</b>    |
| 3096                   | 472  | cmd.exe     | 2009-11-17 00:21:37    | A shell near capture time            |



| PID  | PPID | Process     | Created             | Reading                            |
|------|------|-------------|---------------------|------------------------------------|
| 3700 | 3096 | mdd_1.3.exe | 2009-11-17 00:22:29 | The memory acquisition tool itself |

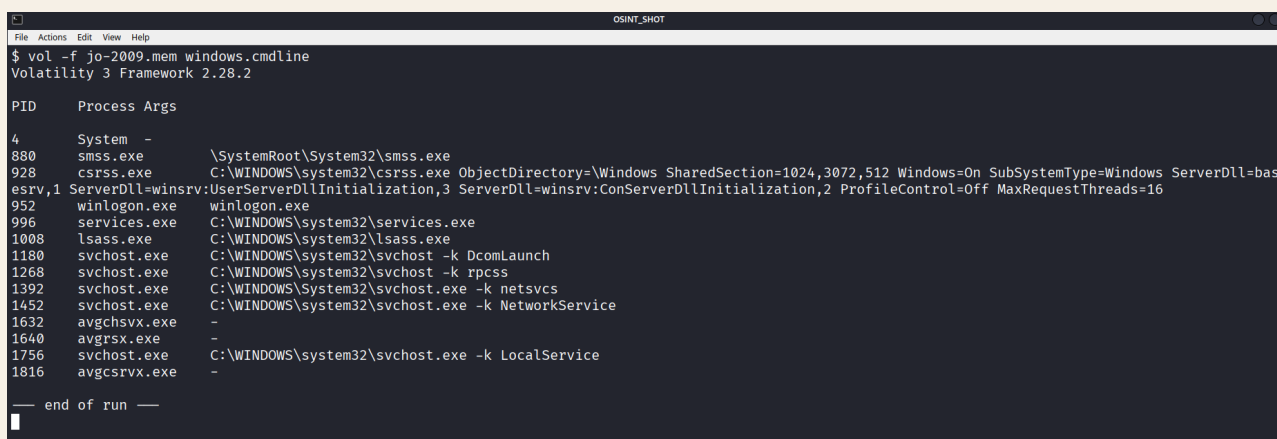
Read the parent-child relationship, not the process list. A list of processes tells you little. soffice.bin spawning four cmd.exe children within six seconds of each other tells you a document macro executed. Office applications do not spawn shells.

mdd\_1.3.exe is the acquisition artefact, and you must exclude it. It appears because the tool was running when the image was taken. Reporting it as suspicious is a classic beginner error. Any acquisition tool, backup agent or antivirus process will appear this way.

Four shells in six seconds is the shape of automation. A human does not open four command prompts in six seconds. Sequence and timing are the intelligence.

## 6.3 What command started it

```
vol -f jo-2009.mem windows.cmdline
```



```
OSINT_SHOT
File Actions Edit View Help
$ vol -f jo-2009.mem windows.cmdline
Volatility 3 Framework 2.28.2

PID    Process Args
4      System -
880    smss.exe \SystemRoot\System32\smss.exe
928    csrss.exe C:\WINDOWS\system32\csrss.exe ObjectDirectory=\Windows SharedSection=1024,3072,512 Windows=On SubSystemType=Windows ServerDll=bas
esrv,1 ServerDll=winsrv:UserServerDllInitialization,3 ServerDll=winsrv:ConServerDllInitialization,2 ProfileControl=Off MaxRequestThreads=16
952    winlogon.exe
996    services.exe C:\WINDOWS\system32\services.exe
1008   lsass.exe C:\WINDOWS\system32\lsass.exe
1180   svchost.exe C:\WINDOWS\system32\svchost -k DcomLaunch
1268   svchost.exe C:\WINDOWS\system32\svchost -k rpcss
1392   svchost.exe C:\WINDOWS\system32\svchost.exe -k netsvcs
1452   svchost.exe C:\WINDOWS\system32\svchost.exe -k NetworkService
1632   avgchsvx.exe -
1640   avgrsx.exe -
1756   svchost.exe C:\WINDOWS\system32\svchost.exe -k LocalService
1816   avgcsrpx.exe -

— end of run —
```

Screenshot: Volatility showing the command line of each process.

What the run above shows. The argument string for each process — where the intent usually hides:

| PID  | Process     | Command line                                                                    |
|------|-------------|---------------------------------------------------------------------------------|
| 3656 | soffice.exe | "C:\Program Files\OpenOffice.org 3\program\soffice.exe"                         |
| 2392 | soffice.bin | ...soffice.exe" "-env:OOO_CWD=2C:\Program Files\OpenOffice.org 3\Basis\program" |
| 3860 | cmd.exe     | -                                                                               |
| 3096 | cmd.exe     | "C:\WINDOWS\system32\cmd.exe"                                                   |

A dash means the command line could not be read, and that is itself a signal. Volatility prints - when the argument block has been freed or overwritten. Malware frequently clears its own command line to defeat exactly this plugin, so three cmd.exe processes with no readable arguments is a finding, not a gap.

Compare PIDs 3860 and 3096. Both are cmd.exe. One has a readable path, the others do not. That difference is evidence.

```
## filter for the interesting ones
vol -f jo-2009.mem windows.cmdline | grep -iE "cmd|powershell|wscript|cscript"
```



```
vol -f jo-2009.mem windows.malware.malfind
```

Screenshot: Volatility malfind reporting executable memory regions by process.

| PID  | Process                                    | Protection             | Count      |
|------|--------------------------------------------|------------------------|------------|
| 2392 | soffice.bin                                | PAGE_EXECUTE_READWRITE | 10 regions |
| 952  | winlogon.exe                               | PAGE_EXECUTE_READ      | 2          |
| 1008 | lsass.exe                                  | PAGE_EXECUTE_READ      | 2          |
| —    | avgwdsvc.exe, avgtray.exe, AVGIDSAgent.exe | various                | 6          |
| 928  | csrss.exe                                  | PAGE_EXECUTE_READWRITE | 1          |
| —    | msmsgs.exe                                 | various                | 1          |

The plugin path already changed once. windows.malfind is deprecated and will be removed after 2026-06-07; use windows.malware.malfind:

FutureWarning: This plugin (volatility3.plugins.windows.malfind.Malfind) has been renamed and will be removed in the first release after 2026-06-07.

Page 58



## 6.5 A real tool failure: network plugins do not work on XP

```
vol -f jo-2009.mem windows.netscan
```

```
File Actions Edit View Help
$ vol -f jo-2009.mem windows.netscan
  raise NotImplementedError(
    f"This version of Windows is not supported: {nt_major_version}.{nt_minor_version} {vers.MajorVersion}.{vers_minor_version}!"
  )
NotImplementedError: This version of Windows is not supported: 5.1 15.2600!
— end of run —
```

*Screenshot: Volatility failing on the XP memory image with NotImplementedError.*

What the run above shows is a genuine failure, and it is worth understanding:

```
NotImplementedError: This version of Windows is not supported: 5.1 15.2600!
```

Both `windows.netscan` and `windows.netstat` fail identically on this image. `determine_tcpip_version()` in `netscan.py` only handles Vista and later, so it cannot walk the XP TCP/IP structures. There is no flag that fixes this, and no newer release that fixes it — the plugin was never written for XP.

What to do instead, and why the failure is still worth reporting:

| Route                   | Works on XP? | Note                                                 |
|-------------------------|--------------|------------------------------------------------------|
| windows.netscan         | No           | NotImplementedError                                  |
| windows.netstat         | No           | Same root cause                                      |
| windows.handles         | Partially    | Can filter socket handles where supported            |
| windows.dllexport       | Yes          | Shows Winsock loaded — corroborating, not conclusive |
| Correlate with the pcap | Yes          | The strongest route when you have both               |

Report it as a method limitation, not a gap. "Network connections could not be enumerated from this XP image: Volatility 3's netscan does not support Windows 5.1. The pcap was used instead." That sentence is worth more than a silent omission, because it tells the reader exactly what was and was not examined.

This is the general lesson about memory analysis and old images. Volatility 3's XP support is real but incomplete. On a genuinely old case, check the plugin against the OS before you promise a result — and keep a Volatility 2 image to hand as a fallback.

## 6.6 What persists across a reboot

```
vol -f jo-2009.mem windows.svcscan
```

Services are how Windows malware usually survives a restart. `svcscan` lists every service, its binary path, and its start type:

```
vol -f jo-2009.mem windows.svcscan | head -30
vol -f jo-2009.mem windows.svcscan | grep -iE "SERVICE_AUTO_START" | head
```

Read the binary path, not the service name. Malware names services to look legitimate (Windows Update Helper) while pointing at a path that does not belong there — a user



directory, ProgramData, or a temp folder.

Automatic start services in a user-writable path are the finding. A service that starts itself at boot and runs from C:\Users\... is persistence, whatever it calls itself.

## 6.7 Memory artefacts worth knowing

| Plugin                    | Answers                                                             |
|---------------------------|---------------------------------------------------------------------|
| windows.dlllist           | Which modules each process loaded — unexpected DLLs from temp paths |
| windows.filescan          | Open file handles — files that existed but may not be in the MFT    |
| windows.handles           | Registry keys and other objects each process holds                  |
| windows.svcscan           | Services and their start types                                      |
| windows.registry.hivelist | Which registry hives were loaded                                    |
| windows.hashdump          | <b>Local account password hashes from memory</b>                    |
| windows.cachedump         | Cached domain credentials                                           |

hashdump works from a memory image, and that is the point. Credentials sit in memory while a system runs. This is why a memory image of a domain controller is treated as catastrophic, and why you handle one with corresponding care.

## 7. Network evidence

A pcap shows what actually crossed the wire, which is the one thing logs can lie about.

```
capinfos 2025-01-22-traffic-analysis-exercise.pcap
```



```
$ capinfos 2025-01-22-traffic-analysis-exercise.pcap
File name:      2025-01-22-traffic-analysis-exercise.pcap
File type:      Wireshark/tcpdump/... - pcap
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit: file hdr: 65535 bytes
Number of packets: 39 k
File size:      26 MB
Data size:      26 MB
Capture duration: 3202.388113 seconds
Earliest packet time: 2025-01-22 19:44:56.530137
Latest packet time: 2025-01-22 20:38:18.918250
Data byte rate: 8,159 bytes/s

$ tshark -r ... -Y "dns.flags.response==0" -T fields -e dns.qry.name | sort -u
appointedtimeagriculture.com
authenticatoor.org

— end of run —
```

Screenshot: capinfos summarising the capture, and DNS queries extracted from it.

What the run above shows. The capture's own metadata, before you read a single packet:

| Field             | Value                   | Reading                   |
|-------------------|-------------------------|---------------------------|
| File type         | pcap                    | Standard capture format   |
| Encapsulation     | Ethernet                | Link-layer framing        |
| Number of packets | 39 k                    | A short but dense capture |
| File size         | 26 MB                   |                           |
| Capture duration  | 3202 s                  | About 53 minutes          |
| Earliest packet   | 2025-01-22 19:44:56 UTC |                           |
| Latest packet     | 2025-01-22 20:38:18 UTC |                           |



| Field  | Value           | Reading                      |
|--------|-----------------|------------------------------|
| SHA256 | e59db1c07c6f... | The capture's integrity hash |

Start with capinfos, always. It tells you the time window you are working in, which is the first thing any timeline needs.

## Find the suspicious traffic

```
## protocol hierarchy – what kinds of traffic are present
tshark -r capture.pcap -q -z io,phs

## every DNS query, unique
tshark -r capture.pcap -Y "dns.flags.response==0" -T fields -e dns.qry.name | sort -u

## filter to the interesting ones
... | grep -iE "authenticatoor|appointedtime"
```

What the run above shows. Two domains stand out immediately from the legitimate traffic:

| Domain                       | Reading                                                               |
|------------------------------|-----------------------------------------------------------------------|
| authenticatoor.org           | <b>A deliberately misspelt "authenticator"</b> — classic malvertising |
| appointedtimeagriculture.com | An unrelated-purpose domain, typical of a disposable C2 host          |

The spelling error is the finding. authenticatoor.org is designed to look right at a glance. Lookup domains character by character, not as words — typosquatting relies on your eye skipping the error.

Sort DNS by frequency as well as by name. C2 domains often appear once or twice, while legitimate domains appear hundreds of times. A domain queried exactly twice in 53 minutes, from a host that queried nothing else like it, is a beacon.

```
tshark -r capture.pcap -Y "dns.flags.response==0" -T fields -e dns.qry.name \
| sort | uniq -c | sort -n | head -20      # rare names first
```

## Extract what was transferred

```
## list HTTP objects, with the host each came from
tshark -r capture.pcap -Y "http.request" -T fields \
-e frame.time -e ip.src -e http.host -e http.request.uri | head -20

## carve transferred files out of the capture
tshark -r capture.pcap --export-objects http,http_objects
ls http_objects/
```

--export-objects writes the actual transferred files to disk, which is how you go from "a file was downloaded" to holding the file. Hash everything you extract.

HTTPS hides the content, not the conversation. You still see the SNI hostname, the certificate, the timing and the volume. Beaconing is detectable from timing alone, even with no content, because a fixed interval with slight jitter is a machine, not a person.

## The pcap question set

The bundled exercise asks six questions that map onto the analysis above. This is the shape of a real network investigation:



| Question                | Where the answer comes from            |
|-------------------------|----------------------------------------|
| Which host is infected? | Who queries the odd domain first       |
| Its MAC address?        | eth.src for that IP, from DHCP         |
| Its hostname?           | DHCP hostname option, or NBNS          |
| The user account?       | Kerberos, NTLM, or cleartext auth      |
| The fake download page? | The HTTP referer or DNS name           |
| The C2 servers?         | The hosts contacted after the download |

Answer them in order, because each narrows the next. Identify the host, then filter everything else to that host.

## 8. Event logs

Windows records what it did. The EVTX files here are real attack data, organised by ATT&CK; tactic.

```
python3 evtx_read.py '<file>.evtx' --summary --limit 14
```

```
File Actions Edit View Help
OSINT_SHOT
$ python3 evtx_read.py <file>.evtx --summary
REC  EVENT ID  TIME (UTC)  PROVIDER  CHANNEL
1    5         2020-10-08 14:43:49  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
2    3         2020-10-08 14:43:49  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
3    61        2020-10-08 14:43:49  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
4    5         2020-10-08 14:43:49  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
5    306       2020-10-09 11:17:23  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
6    3         2020-10-09 11:17:25  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
7    209       2020-10-09 11:17:25  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
8    61        2020-10-09 11:17:25  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
9    3         2020-10-09 11:18:54  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
10   61        2020-10-09 11:18:54  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
11   61        2020-10-09 11:19:04  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
12   61        2020-10-09 11:19:14  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
13   61        2020-10-09 11:19:24  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits
14   61        2020-10-09 11:19:39  Microsoft-Windows-Bits-Clie  Microsoft-Windows-Bits

14 record(s) shown
— end of run —
```

Screenshot: EVTX records summarised by event id, time and provider.

What the run above shows. Windows BITS (Background Intelligent Transfer Service) client events. BITS is a legitimate Windows component that malware abuses for downloading, because it is signed, always running, and its traffic blends in.

| Rec   | Event ID | Time                   | Provider    | Reading                                |
|-------|----------|------------------------|-------------|----------------------------------------|
| 1     | 5        | 2020-10-08 14:43:49    | BITS Client | Job created                            |
| 2     | 3        | 2020-10-08 14:43:49    | BITS Client | Job started                            |
| 3     | 61       | 2020-10-08 14:43:49    | BITS Client | Transfer started                       |
| 5     | 306      | 2020-10-09 11:17:23    | BITS Client | <b>Almost 21 hours later</b>           |
| 11-12 | 61       | 2020-10-09 11:19:04-14 | BITS Client | <b>Repeated at 10-second intervals</b> |

Rows 11 and 12 are the finding. Two transfer events ten seconds apart, from a service that should transfer files once, is a repeated transfer — which is what a malware download loop looks like.



Read the time column before the event id. The 21-hour gap between record 3 and record 5 matters more than any individual event id: it separates two distinct activity periods.

## The detail view

```
python3 evt_x_read.py '<file>.evt_x' --limit 2
```

```
OSINT_SHOT
File Actions Edit View Help
$ python3 evt_x_read.py <file>.evt_x --limit 2
Record 1 EventID 5 2020-10-08 14:43:49.291977+00:00
Computer : MSEDGWIN10
Channel : Microsoft-Windows-Bits-Client/Operational
Provider : Microsoft-Windows-Bits-Client
User : S-1-5-18
User : NT AUTHORITY\SYSTEM
JobTitle : C:\Program Files (x86)\Google\Update\1.3.35.452\Recovery\GUR8658.tmp\GoogleUpdateSetup.crx3
JobId : {1960d15e-5fc2-457d-abe7-9a7cb97b7761}
JobOwner : NT AUTHORITY\SYSTEM
fileCount : 1
Record 2 EventID 3 2020-10-08 14:43:49.877703+00:00
Computer : MSEDGWIN10
Channel : Microsoft-Windows-Bits-Client/Operational
Provider : Microsoft-Windows-Bits-Client
User : S-1-5-18
JobTitle : C:\Program Files (x86)\Google\Update\1.3.35.452\Recovery\GUR8658.tmp\GoogleUpdateSetup.crx3
JobId : {8f97efec-9bdf-4726-90b9-8c56b76bdbd7}
JobOwner : NT AUTHORITY\SYSTEM
processPath : C:\Program Files (x86)\Google\Update\GoogleUpdate.exe
processId : 3616
2 record(s) shown
— end of run —
```

*Screenshot: the full parsed record including event data fields.*

What the run above shows. Every named field in the event — the part that matters, because EventData names are where the actual values live:

| Field     | Contains                                                 |
|-----------|----------------------------------------------------------|
| Computer  | Which machine logged it                                  |
| Channel   | Which log — Security, System, Sysmon, or a component log |
| Provider  | Which Windows component wrote it                         |
| User      | The security identifier of the actor                     |
| EventData | The named values: paths, PIDs, results, targets          |

The EventData names carry the meaning, not the numeric ids. Event ID 4688 means "a process was created" — the value is in the NewProcessName and CommandLine fields inside it. Always read the detail, never just the id.



## Which log carries which signal

| Event ID | Log      | Means                                                  |
|----------|----------|--------------------------------------------------------|
| 4624     | Security | A logon succeeded                                      |
| 4625     | Security | <b>A logon failed — many of these is a brute force</b> |
| 4672     | Security | Special privileges were assigned                       |
| 4688     | Security | <b>A process was created</b> (needs auditing enabled)  |
| 4720     | Security | A user account was created                             |
| 7045     | System   | <b>A service was installed</b> — classic persistence   |
| 1        | Sysmon   | Process creation, with the full command line           |
| 3        | Sysmon   | A network connection was made                          |
| 11       | Sysmon   | A file was created                                     |
| 22       | Sysmon   | A DNS query was made                                   |

Sysmon IDs are far more useful than Security IDs for detection, because Sysmon records the command line and the hash with the event. If 4688 has no command line, auditing is not configured to include it — and that is a finding about the environment, not about the attack.

Correlate across logs by time and PID. A 7045 service install at 14:43, a 1 process creation at 14:43, and a 3 network connection at 14:43 are the same event seen three ways. That triple is a detection you can deploy.

## The EVTX tool problem, and the tool that replaces it

evtx\_dump is installed but broken:

```
Traceback (most recent call last):
  File "/usr/local/bin/evtx_dump", line 3, in <module>
    from scripts.evtx_dump import main
ModuleNotFoundError: No module named 'scripts'
```

The cause is a packaging fault, not your command. The pipx console script imports a scripts package that is not present in the installed distribution. The library itself is fine.

The fix is to drive the library directly. tools/evtx\_read.py in this guide does exactly that, and adds filtering by event id and a summary mode:

```
## summary of many records
python3 evtx_read.py file.evtx --summary --limit 50

## one specific event id
python3 evtx_read.py file.evtx --event-id 7045

## full detail for the first few
python3 evtx_read.py file.evtx --limit 5
```

Run it with the pipx venv's interpreter, because Evtx is installed there and not in the system Python:

```
/opt/pipx/venvs/python-evtx/bin/python evtx_read.py file.evtx --summary
```



Keep the wrapper. When you hit a tool that is broken by its own packaging, fixing it once and keeping the fix is faster than re-diagnosing it on the next case — and the fix belongs in the report as a method note.

## 9. Reporting

An analysis that is not written up did not happen. The report is the deliverable, and it is the only part anyone else reads.

### 9.1 Structure

```
## Forensic Report – <case reference>

## Summary
Three sentences. What was examined, what was found, what it means.

## Evidence received
Item, description, serial or filename, source, date received.

## Integrity
SHA-256 on receipt, SHA-256 after analysis, and the statement that they match.

## Method
Every tool and version used, and every command run. Enough to reproduce it.

## Findings
One per finding, each with the artefact it came from and the confidence.

## Timeline
The chronological reconstruction, with the source of each event.

## Limitations
What could not be determined, with the reason. Include tool failures.

## Appendix
Full tool output, in the order it was produced.
```

The Integrity section is what makes the rest admissible. Put the two hashes next to each other so a reader can compare them without hunting:

```
Evidence received : ntfs1-gen1.E01
SHA-256 on receipt: 3f8a...c21d
SHA-256 after exam: 3f8a...c21d
Result           : MATCH – evidence unchanged
```

### 9.2 The findings table

| # | Finding                                                     | Source                             | Confidence                               |
|---|-------------------------------------------------------------|------------------------------------|------------------------------------------|
| 1 | OpenOffice spawned four shells in six seconds               | pslist PIDs 3860, 2728, 3968, 1188 | <b>High</b> — parent-child confirmed     |
| 2 | soffice.bin held ten RWX memory regions                     | malfind PID 2392                   | <b>High</b> — corroborates finding 1     |
| 3 | Three shell processes had unreadable command lines          | cmdline PIDs 3860, 2728, 3968      | <b>Medium</b> — consistent with clearing |
| 4 | Host queried authenticatoor.org                             | pcap DNS, 2025-01-22               | <b>High</b> — direct observation         |
| 5 | Network enumeration was not <b>possible</b> from this image | netscan<br>NotImplementedError     | <b>Limitation, stated</b>                |



Give every finding a confidence and a source. A finding with no source cannot be checked; a finding with no confidence cannot be weighed.

State the limitations as prominently as the findings. Finding 5 is not a failure of the analysis, it is a fact about the method, and the reader needs it to judge how much weight finding 2 can bear.

### 9.3 Two things that will be challenged

Any timestamp claim. Time zone, source of the clock, and whether the artefact even records that field. Windows A-time is often stale — say so, every time.

Any tool-dependent conclusion. malfind flags permissions, not intent, and false-positives on security software and JIT engines. If your finding rests on a heuristic, name the heuristic.

## 10. Tool reference

| Tool            | Phase    | Command                                               |
|-----------------|----------|-------------------------------------------------------|
| file            | Identify | file <evidence>                                       |
| sha256sum       | Preserve | sha256sum <evidence>                                  |
| ssdeep          | Identify | ssdeep -b <evidence>                                  |
| ewfinfo         | Acquire  | ewfinfo <image>.E01                                   |
| ewfexport       | Acquire  | ewfexport -q -t <dir> -f raw -u <image>.E01           |
| dc3dd           | Acquire  | dc3dd if=/dev/sdX of=image.dd hash=sha256 log=acq.log |
| mmls            | Disk     | mmls <image>                                          |
| fsstat          | Disk     | fsstat -o <offset> <image>                            |
| fls             | Disk     | fls -r -d <image>                                     |
| istat           | Disk     | istat <image> <inode>                                 |
| icat            | Disk     | icat <image> <inode> > out.file                       |
| blkls           | Disk     | blkls <image> > unallocated.bin                       |
| bulk_extractor  | Carving  | bulk_extractor -q -o out <image>                      |
| foremost        | Carving  | foremost -i <file> -o out                             |
| mactime         | Timeline | mactime -b body.txt                                   |
| log2timeline.py | Timeline | log2timeline.py --storage-file t.plaso <image>        |
| psort.py        | Timeline | psort.py -o l2tcsv -w t.csv t.plaso                   |
| vol             | Memory   | vol -f <image> windows.plist                          |
| tshark          | Network  | tshark -r <pcap> -Y "<filter>"                        |
| capinfos        | Network  | capinfos <pcap>                                       |
| evtx_read.py    | Logs     | python3 evtx_read.py <file>.evtx --summary            |



## Installing a missing tool

```
## apt – needs root. Names differ from the upstream project names:
## libewf-utils and libpff-utils do NOT exist; the packages are ewf-tools and
## afflib-tools. One wrong name aborts the whole apt transaction, so every
## package in the list fails together.
apt-get install -y sleuthkit ewf-tools afflib-tools dc3dd dcfldd \
    bulk-extractor foremost libesedb-utils

## Python tools – PEP-668 blocked, use pipx with an explicit bindir
export PIPX_HOME=/opt/pipx PIPX_BIN_DIR=/usr/local/bin
pipx install volatility3
pipx install plaso
```

## Tools that are not in this lab, and why

| Tool              | Status             | Note                                          |
|-------------------|--------------------|-----------------------------------------------|
| ewfmount          | <b>Fails</b>       | No /dev/fuse in the container. Use ewfexport. |
| evtx_dump         | <b>Broken</b>      | Packaging fault. Use evtx_read.py.            |
| netscan / netstat | <b>Fails on XP</b> | Plugin does not support Windows 5.1.          |
| Autopsy           | Not installed      | GUI; needs a desktop session                  |
| hindsight         | Not installed      | Build fails on Python 3.14                    |
| python3-evtx      | In pipx only       | Call its venv interpreter explicitly          |

## 11. Troubleshooting

Every failure below is a real one hit while writing this guide.

| Symptom                                        | Cause                                                   | Fix                                                 |
|------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------|
| ewfmount: /dev/fuse not found                  | No FUSE in the container                                | ewfexport -t <dir> -f raw -u instead                |
| ewfinfo: command not found                     | Package is ewf-tools, not libewf-utils                  | Install the right name                              |
| apt aborts and nothing installs                | <b>One bad package name fails the whole transaction</b> | Remove the bad name, re-run                         |
| vol: no symbols / plugins return nothing       | Symbol download failed offline                          | Pre-fetch with connectivity, or point -s at a cache |
| NotImplementedError: ... 5.1 15.2600           | XP not supported by netscan                             | Report the limitation; use the pcap                 |
| FutureWarning: ... will be removed             | malfind moved namespace                                 | Use windows.malware.malfind                         |
| ModuleNotFoundError: No module named 'scripts' | evtx_dump packaging fault                               | Use evtx_read.py                                    |
| ModuleNotFoundError: No module named 'Evtx'    | Wrong interpreter                                       | /opt/pipx/venvs/python-evtx/bin/python              |
| log2timeline: unrecognized arguments: -o       | plaso changed its CLI                                   | Use --storage-file                                  |
| mmls shows no partitions                       | Bare filesystem, no partition table                     | Correct — go straight to fsstat                     |
| file says data for an image                    | E01 is a container, not a filesystem                    | Expected — unwrap it first                          |



| Symptom                             | Cause                             | Fix                               |
|-------------------------------------|-----------------------------------|-----------------------------------|
| fls returns nothing                 | Wrong offset, or encrypted volume | Check fsstat with -o              |
| Carved files have no names          | Carving has no filesystem context | Expected — state it in the report |
| filesnscan shows garbled characters | Pool-tag encoding on XP           | Cosmetic; use the readable suffix |

The apt lesson is worth stating on its own. apt-get install -y a b c d is a single transaction: if c does not exist, a, b and d are not installed either, and the only message is Unable to locate package c. On a fresh build this looks like apt being broken when it is behaving correctly. Install in small batches, and read the package list before running it.

## 12. Appendix — reproducing this guide

Every picture here is a capture of a real command run against real evidence. Nothing was reconstructed. The tooling is included so any figure can be regenerated.

```
## 1. Run the real command, keeping the transcript
docker exec -u 0 <lab> bash -lc \
  'export PATH=$PATH:/usr/local/bin; <command>' \
  > evidence/${date -u +%Y%m%dT%H%M%S}_<slug>.txt 2>&1

## 2. Screenshot the terminal that produced it
tools/term_shot.py images/<slug>.png <seconds> -- '<command>'

## 3. Crop the desktop chrome away so the text fills the page
tools/crop_shots.py images/*.png
```

The capture pitfalls the tool exists to handle, each of which cost real time:

- qterminal has no title flag; set the window title with a printf escape and find the window with xdotool search --class qterminal.
- Resize the window before the command runs. The shell wraps output to the width it had when it printed.
- qterminal rejects --size and then never launches.
- Use skill -x qterminal, not -f: the -f form matches the docker exec command line and kills the caller.
- Never nest a single quote inside a command block passed to docker exec bash -lc inside a single-quoted string. It silently truncates the block, producing a garbled or empty capture rather than an error.

Where the evidence came from. All datasets are publicly published for education:

| Dataset                          | Source                                                                                                                                      |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| NPS test drive images            | <a href="https://digitalcorpora.org/corpora/disk-images/">digitalcorpora.org/corpora/disk-images/</a>                                       |
| M57-Patents (disk, RAM, network) | <a href="https://digitalcorpora.org/corpora/scenarios/m57-patents-scenario/">digitalcorpora.org/corpora/scenarios/m57-patents-scenario/</a> |
| Sleuth Kit test data             | <a href="https://digitalcorpora.org/corpora/drives/tsk-2024/">digitalcorpora.org/corpora/drives/tsk-2024/</a>                               |
| EVTX attack samples              | <a href="https://github.com/sbousseaden/EVTX-ATTACK-SAMPLES">github.com/sbousseaden/EVTX-ATTACK-SAMPLES</a>                                 |
| Traffic analysis exercise        | <a href="https://malware-traffic-analysis.net/2025/01/22/">malware-traffic-analysis.net/2025/01/22/</a>                                     |



The Digital Corpora bucket is readable without credentials:

```
curl -s "https://digitalcorpora.s3.amazonaws.com/?list-type=2&prefix=corpora/drives/"
aws s3 ls --no-sign-request s3://digitalcorpora/
```

## What is in this folder

| Path                   | Holds                                                     |
|------------------------|-----------------------------------------------------------|
| artifacts/disk-images/ | The disk, memory-adjacent, pcap and EVTX evidence         |
| figures/               | Excalidraw sources and rendered diagrams                  |
| images/                | The terminal captures                                     |
| evidence/              | Raw command transcripts                                   |
| tools/                 | Capture, crop, figure-build, EVTX parser and PDF renderer |
| reports/               | Analysis reports produced from the evidence               |

Training at Forrest.one · Digital Forensics & Disk Image Analysis · Training evidence only.

## Incident Response Guide

### Incident Response & Intrusion Analysis

Training at Forrest.one · Detecting, containing and reporting a live intrusion

## How to use this guide

This is a working manual. Every command was run against a live incident generated in this lab, and every picture is a capture of the tool that produced the output.

Four things to know before you start:

1. Incident response is a process, not a toolkit. The tools are simple. Knowing which phase you are in, and what you are allowed to do in it, is the skill.
2. Containment is where investigations are lost. Isolate the host; do not destroy it.
3. One source is never the whole story. Correlate before you conclude.
4. Report the negatives. What you could not determine is a finding.

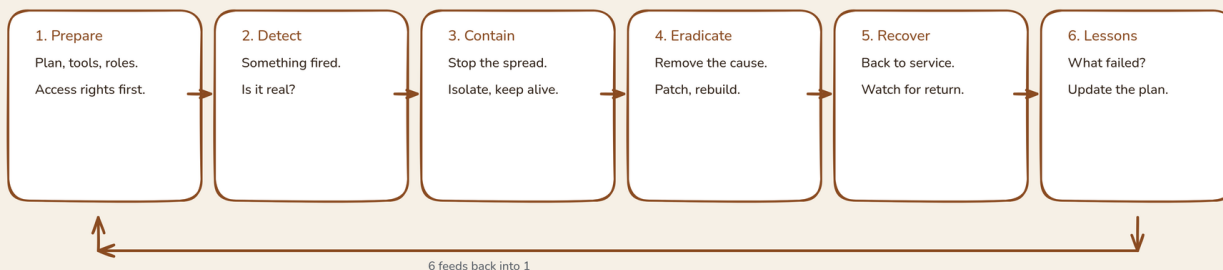
## The rule that comes before every other rule

Do not destroy the evidence while you are trying to protect the business. These two goals pull against each other, and the tension is the whole discipline.



## The incident response lifecycle

Six phases. You move forward, but you go back to Preparation after every incident.



Containment is where investigations go wrong.

Pulling the plug destroys memory, which is often the only place the attacker's code exists. Isolate the host from the network, then collect volatile evidence before you power anything down.

### The order of operations

Isolate the network. Collect the memory. Then, and only then, power down.

Figure 1: The response lifecycle, and the feedback loop from lessons back to preparation.

| Pulling the plug protects | Pulling the plug destroys             |
|---------------------------|---------------------------------------|
| The rest of the network   | Running processes and their parentage |
| The data on the host      | Injected code that exists only in RAM |
| Your time                 | Open network connections              |
| The evidence on disk      | Decryption keys held in memory        |

The right move is almost always the same: isolate the host from the network, then collect volatile evidence, then power it down — if you power it down at all. A host pulled from the network cannot attack anything else, and it is still running.

Collect in order of volatility. The top of this list is gone the moment the machine loses power:



## Order of volatility

Collect what disappears first. The top of this list is gone after a reboot.

|                   |                                                |                               |
|-------------------|------------------------------------------------|-------------------------------|
| 1 Memory (RAM)    | Running processes, injected code, open sockets | Gone on power off             |
| 2 Network state   | Connections, routing, ARP cache                | Gone in seconds               |
| 3 Running system  | Processes, logged-in users, open files         | Changes constantly            |
| 4 Temporary files | Swap, temp dirs, prefetch                      | Survives reboot, not clean-up |
| 5 Disk            | Files, logs on disk, registry hives            | Survives reboot               |
| 6 Remote logs     | Shipped syslog, SIEM, backups                  | Survives the host             |
| 7 Archives        | Backups, off-site copies, media                | Survives everything           |

**Pull the plug and you lose the top of this list**

Powering off is sometimes the right call, but it is always a decision with a cost. Record who made it and why.

Figure 2: Order of volatility, from memory down to archives.

## 1. The incident you are responding to

This lab ships a scripted attacker and a weak victim on an internal network. Running the attacker generates a live intrusion with host-side logs and a packet capture, so you can practise the full response against real evidence rather than a scenario description.

The attacker runs four phases in about fourteen seconds:

| # | Technique                                         | What it leaves behind                          |
|---|---------------------------------------------------|------------------------------------------------|
| 1 | SSH brute force across 9 usernames                | auth.log failed logins, pcap SSH conversations |
| 2 | HTTP reconnaissance, 10 well-known paths          | httpd.log 404s, pcap HTTP requests             |
| 3 | HTTP exploit attempts: traversal, SQLi, upload    | httpd.log 404/403/200, pcap request bodies     |
| 4 | C2-style beacon, 5 POSTs to a tracking-pixel path | httpd.log + pcap POST bodies                   |

Phase 4 is the one that changes your response. Phases 1 to 3 are attempts that the victim refused. Phase 4 is data leaving the host.



## The four phases, and the artefact each one leaves

Reconstructed from this lab's incident. Every row is a real observation.

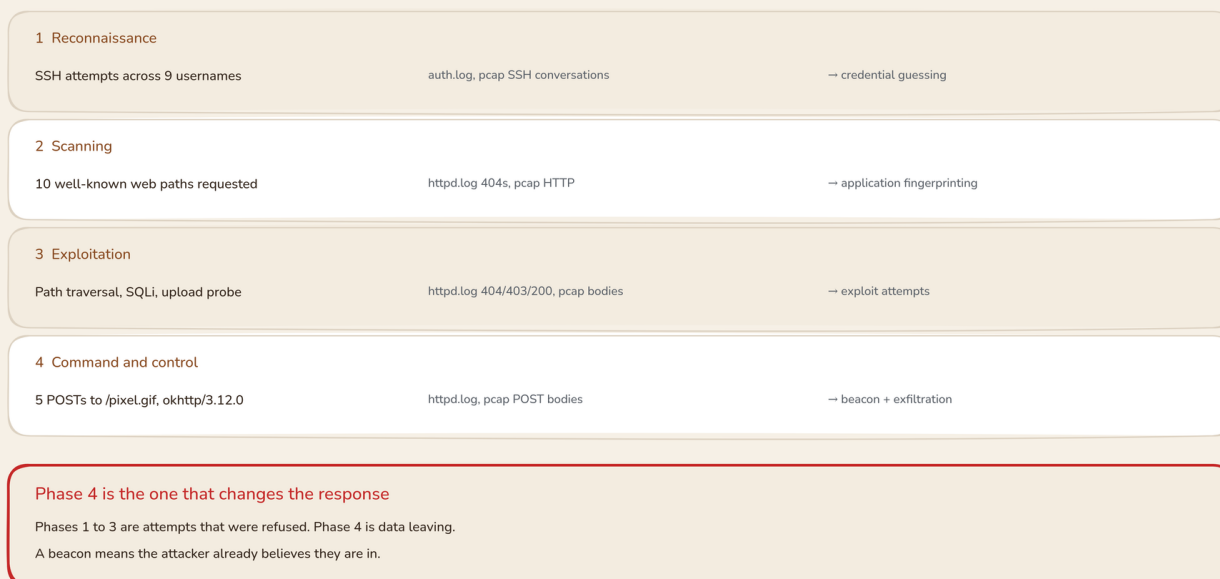


Figure 3: The four phases and the artefact each one leaves.

## Run the attack

```
cd /home/billy-forrest/labs/cpia-labs

docker start cpia-attacker      # NOTE: starting re-runs its entrypoint script
sleep 20                       # let the attack finish and the capture flush
```

That NOTE is not filler. Read the box below before you run anything twice.

## The trap that catches everyone, including the author of this guide

docker start cpia-attacker runs /script.sh again. The container's command is the attack. Starting it, restarting it, or running docker exec cpia-attacker sh /script.sh all launch another full attack.

While writing this guide I did both, so the attack ran twice within a second of itself. The result was evidence where every single request appeared exactly twice — 36 HTTP requests instead of 18, and 10 beacon POSTs instead of 5.

Nothing about that looked broken. The logs parsed, the commands ran, the screenshots looked fine. The numbers were simply double the truth, and an incident report built on them would have overstated the attacker's activity by 100%.

So verify the incident is a single incident before you analyse it:

```
grep -oE '"(GET|POST) [^ ]+' httpd.log | sort | uniq -c | sort -rn | head
```

```
OSINT_SHOT
File Actions Edit View Help
$ grep -oE '"(GET|POST) [^ ]+' httpd.log | sort | uniq -c | sort -rn | head -6
1 "POST /upload
1 "POST /pixel.gif?id=beacon&c=5
1 "POST /pixel.gif?id=beacon&c=4
1 "POST /pixel.gif?id=beacon&c=3
1 "POST /pixel.gif?id=beacon&c=2
1 "POST /pixel.gif?id=beacon&c=1

# every path appears exactly once: this is ONE incident, not a replay
— end of run —
```



*Screenshot: the duplicate check — every request path appearing exactly once, confirming one clean incident.*

Every path appearing exactly once is what a clean single incident looks like. A count of 2 on everything means the attack was replayed, and you must go back and produce clean evidence before you write a single finding.

This generalises to real work. Sandbox replays, restored snapshots and load-balanced duplicate log shipping all produce the same doubling. Check the shape of your data before you interpret it.

## 2. Phase 1 — Collect

### Pull the evidence out

The evidence lives inside the victim container and must be extracted before you analyse it. Copy, never analyse in place.

```
docker cp cpia-victim:/var/log/auth.log      incident_auth.log
docker cp cpia-victim:/var/log/httpd.log     incident_httpd.log
docker cp cpia-victim:/var/log/network.pcap  incident_network.pcap
```

### Hash at the moment of collection

This is ACPO Principle 1: no action you take may change the data. A hash taken after analysis proves nothing; a hash taken at collection and re-checked at the end is what makes your work defensible.

```
sha256sum incident_auth.log incident_httpd.log incident_network.pcap
```

```
OSINT_SHOT
File Actions Edit View Help
$ sha256sum auth.log httpd.log network.pcap
0f8552426255ffabd9c8a6aaaa3890ee575c85b4f9294d1c582dfbec9e3edd4 auth.log
6a1fbf217c05e6cfcfa3aff7dc128c5095511205c4f81ab32186b3688af8c37a httpd.log
20bde64eb3f8a305daf76a1a1b7886f70f4f1e00264172cab1bb43e58d03edc4 network.pcap
— end of run —
```

*Screenshot: hashing the three evidence files at the point of collection.*

Record the hashes in your report before you open a file. They are the difference between an investigation and an opinion.

## 3. Phase 2 — Detect and triage

### Establish what you have

```
wc -l auth.log httpd.log
ls -la network.pcap
```

```
OSINT_SHOT
File Actions Edit View Help
$ wc -l auth.log httpd.log
16 auth.log
18 httpd.log
34 total

$ ls -la network.pcap
-rw-r--r-- 1 kasm-user kasm-user 96996 Oct  2 22:09 network.pcap
— end of run —
```

*Screenshot: triaging the three evidence sources and their sizes.*

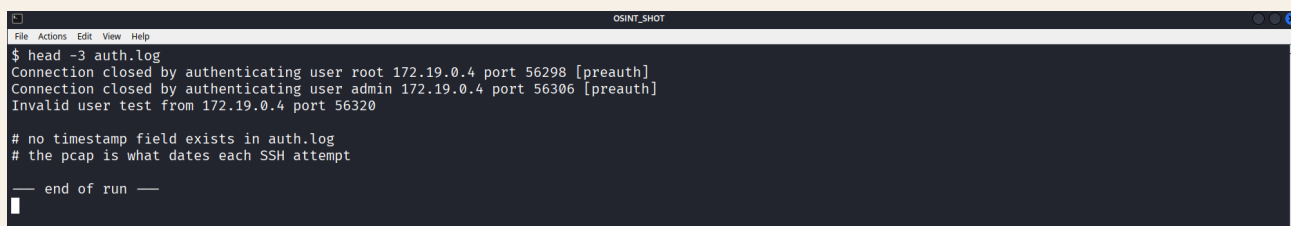


| Evidence     | Size                      | Shape                                       |
|--------------|---------------------------|---------------------------------------------|
| auth.log     | 1,017 bytes, 16 lines     | Small enough to read by hand                |
| httpd.log    | 1,707 bytes, 18 lines     | Small enough to read by hand                |
| network.pcap | 96,996 bytes, 448 packets | <b>Not</b> small enough to read — filter it |

The sizes tell you your method. Two logs of under twenty lines each you simply read. A packet capture you filter, always, because even a 450-packet capture is unreadable as a list.

## The pcap is the only source that can date the SSH phase

```
head -3 auth.log
```



```
OSINT_SHOT
File Actions Edit View Help
$ head -3 auth.log
Connection closed by authenticating user root 172.19.0.4 port 56298 [preauth]
Connection closed by authenticating user admin 172.19.0.4 port 56306 [preauth]
Invalid user test from 172.19.0.4 port 56320

# no timestamp field exists in auth.log
# the pcap is what dates each SSH attempt

— end of run —
```

*Screenshot: auth.log showing that entries carry no timestamp field at all.*

This is a real limitation of the evidence, and it has to be stated. The auth.log in this lab carries no timestamp field. There is nothing to sort by, and no way to say when a login attempt happened from that file alone.

So the packet capture does the dating. The pcap has absolute timestamps on every frame, and the SSH conversations are in it. That is the correlation:

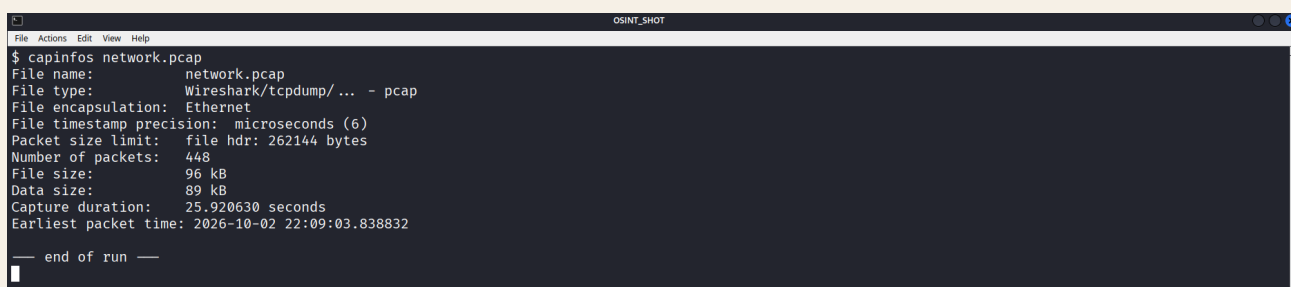
| Question                       | Source that answers it |
|--------------------------------|------------------------|
| Which usernames were tried?    | auth.log               |
| Which of those accounts exist? | auth.log               |
| <b>When</b> was each attempt?  | <b>network.pcap</b>    |
| What was the attack rate?      | <b>network.pcap</b>    |

Name the source of every timestamp in your report. An examiner will ask, and "the log" is not an answer when the log has no timestamps in it.

## 4. Phase 3 — Network analysis

### Read the capture's metadata first

```
capinfos network.pcap
```



```
OSINT_SHOT
File Actions Edit View Help
$ capinfos network.pcap
File name: network.pcap
File type: Wireshark/tcpdump/... - pcap
File encapsulation: Ethernet
File timestamp precision: microseconds (6)
Packet size limit: file hdr: 262144 bytes
Number of packets: 448
File size: 96 kB
Data size: 89 kB
Capture duration: 25.920630 seconds
Earliest packet time: 2026-10-02 22:09:03.838832

— end of run —
```



*Screenshot: capinfos showing the capture's size, duration and packet count.*

The duration field is the one to read. It tells you whether you have one incident or a capture that accumulated across many. A duration measured in days means you must scope before you analyse.

## Scope the capture to the incident window

```
tshark -r network.pcap \  
-Y 'frame.time >= "2026-10-02 22:09:00" && frame.time <= "2026-10-02 22:09:45"' \  
-w incident-window.pcap
```

```
OSINT_SHOT  
File Actions Edit View Help  
$ tshark -Y "frame.time >= 22:09:00 && frame.time <= 22:09:45" -w incident-window.pcap  
-rw-r--r-- 1 root root 105068 Oct 2 22:12 incident-window.pcap  
$ tshark -r incident-window.pcap | wc -l  
448  
— end of run —
```

*Screenshot: writing the scoped capture file so later commands stay short.*

Where the window came from: the httpd.log timestamps. The web log does carry times (22:09:24 to 22:09:29), and the SSH phase runs immediately before the HTTP phase. Use the source that has timestamps to bound the window, then apply it to the source that does not.

Writing a scoped file rather than re-typing the filter keeps every later command short and identical. In a long investigation that consistency is what stops you analysing two different things and calling them one.

## What is in the window

```
tshark -r incident-window.pcap -q -z io,phs
```

```
OSINT_SHOT  
File Actions Edit View Help  
$ tshark -r incident-window.pcap -q -z io,phs  
  ip                frames:432 bytes:88748  
    tcp             frames:432 bytes:88748  
      ssh           frames:99 bytes:60212  
        http        frames:36 bytes:5684  
          data-text-lines frames:11 bytes:1166  
            urlencoded-form frames:6 bytes:1644  
              json     frames:7 bytes:518  
                data-text-lines frames:6 bytes:450  
                  mime_multipart frames:1 bytes:648  
— end of run —
```

*Screenshot: the protocol hierarchy of the scoped capture.*

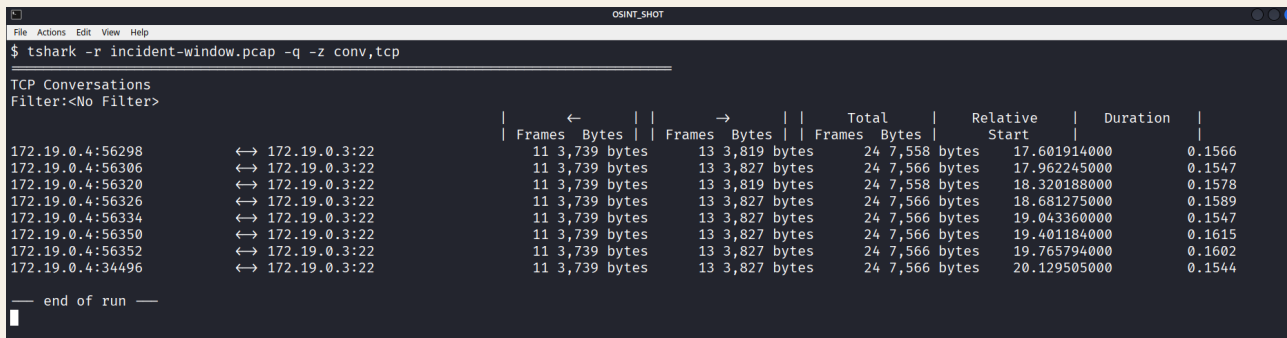
| Protocol | Frames | Bytes  | Reading                     |
|----------|--------|--------|-----------------------------|
| ssh      | 99     | 60,212 | The brute force             |
| http     | 36     | 5,684  | Recon, exploits and beacon  |
| arp      | 10     | 420    | Normal network housekeeping |
| icmpv6   | 6      | 636    | IPv6 neighbour discovery    |
| Total    | 448    | 89,804 |                             |

Read the ratio, not the totals. 99 SSH frames against 36 HTTP frames tells you the dominant activity was credential guessing. An SSH volume this **high** from one host is the detection signal, before you look at a single username.



## Who talked to whom

```
tshark -r incident-window.pcap -q -z conv,tcp
```



Screenshot: the TCP conversations showing nine short, identical SSH sessions from one source.

What the run above shows — and this is the shape of a brute force:

| Conversation                       | Frames | Duration |
|------------------------------------|--------|----------|
| 172.19.0.4:56298 <-> 172.19.0.3:22 | 24     | 0.157 s  |
| 172.19.0.4:56306 <-> 172.19.0.3:22 | 24     | 0.155 s  |
| 172.19.0.4:56320 <-> 172.19.0.3:22 | 24     | 0.158 s  |
| 172.19.0.4:56326 <-> 172.19.0.3:22 | 24     | 0.159 s  |
| 172.19.0.4:56334 <-> 172.19.0.3:22 | 24     | 0.155 s  |
| 172.19.0.4:56350 <-> 172.19.0.3:22 | 24     | 0.162 s  |
| 172.19.0.4:56352 <-> 172.19.0.3:22 | 24     | 0.160 s  |
| 172.19.0.4:34496 <-> 172.19.0.3:22 | 24     | 0.154 s  |
| 172.19.0.4:34506 <-> 172.19.0.3:22 | 24     | 0.165 s  |

Nine conversations, and every one is identical: 24 frames, about 0.16 seconds. A legitimate SSH session lasts minutes and transfers kilobytes in both directions.

A hundred-and-sixty-millisecond session that ends at authentication is one guess and a disconnect. Nine of them in two seconds is a brute force.

Identical conversation sizes are the signature. Nine sessions with the same frame count means the same thing happened nine times — and it was not a login.

A brute force from one source is one detection. A brute force from many sources is a different problem. If you see this pattern from a hundred addresses, you are looking at a distributed campaign, and blocking a single IP will not help.

## 5. Phase 4 — HTTP: reconnaissance, exploitation, beacon

### The reconnaissance

```
tshark -r incident-window.pcap -Y 'http.request.method' \
-T fields -e ip.src -e http.request.method -e http.request.uri
```



```
OSINT_SHOT
File Actions Edit View Help
$ tshark -Y http.request.method -T fields -e ip.src -e http.request.method -e http.request.uri
172.19.0.4 GET /admin
172.19.0.4 GET /admin/index.php
172.19.0.4 GET /config.php
172.19.0.4 GET /backup
172.19.0.4 GET /backup.zip
172.19.0.4 GET /phpmyadmin/
172.19.0.4 GET /wp-login.php
172.19.0.4 GET /server-status
172.19.0.4 GET /robots.txt
172.19.0.4 GET /files/1.txt
172.19.0.4 GET /etc/passwd
172.19.0.4 POST /login
172.19.0.4 POST /upload
172.19.0.4 POST /pixel.gif?id=beacon&c=1
172.19.0.4 POST /pixel.gif?id=beacon&c=2
172.19.0.4 POST /pixel.gif?id=beacon&c=3
172.19.0.4 POST /pixel.gif?id=beacon&c=4
172.19.0.4 POST /pixel.gif?id=beacon&c=5
— end of run —
```

Screenshot: the HTTP request list from the wire, showing the path scan in order.

What the run above shows. Eighteen requests in total, and the order is the tell — it walks a list:

```
/admin
/admin/index.php
/config.php
/backup
/backup.zip
/phpmyadmin/
/wp-login.php
/server-status
/robots.txt
/files/1.txt
/etc/passwd
POST /login
POST /upload
POST /pixel.gif?id=beacon&c=1 ... c=5
```

Nothing was linked to from the site. The victim's page does not contain links to /phpmyadmin/, /backup.zip or /wp-login.php. A request for a path that no page links to is not a user; it is a scanner.

## A filter trap worth knowing

http.request.uri is sticky across TCP continuation frames. A POST whose headers and body arrive in separate segments produces more than one frame carrying that URI, so filtering on http.request.uri alone double-counts.

| Filter                               | Frames returned | Correct?                         |
|--------------------------------------|-----------------|----------------------------------|
| http.request.uri contains "pixel"    | 10              | No — 5 requests, 5 continuations |
| http.request.method == "POST" && ... | 5               | Yes                              |

Count requests with http.request.method. If your request count is an even multiple of what you expect, check for continuation frames before you conclude anything about the attacker's activity.

Confirm the real count by listing the distinct URIs, which collapses the duplicates:

```
tshark -r incident-window.pcap \
-Y 'http.request.method == "POST" && http.request.uri contains "pixel"' \
-T fields -e http.request.uri | sort -u
```



```
OSINT_SHOT
File Actions Edit View Help
$ tshark -Y "POST 66 uri contains pixel" -T fields -e http.request.uri | sort -u
/pixel.gif?id=beacon&c=1
/pixel.gif?id=beacon&c=2
/pixel.gif?id=beacon&c=3
/pixel.gif?id=beacon&c=4
/pixel.gif?id=beacon&c=5
— end of run —
```

Screenshot: five distinct beacon URIs, which is the true request count.

Five distinct URIs, c=1 through c=5, exactly matching what the attacker script sends. That agreement between the script and the capture is how you know your filter is right.

## The same activity seen from the host

```
grep -E ' /(admin|backup|config|phpmyadmin|wp-login|server-status|robots)' httpd.log
```

```
OSINT_SHOT
File Actions Edit View Help
$ grep -E ' /(admin|backup|config|phpmyadmin|wp-login|server-status|robots)' httpd.log
172.19.0.4 - - [02/Oct/2026:22:09:24 ] "GET /admin HTTP/1.1" 404 40 "-" "Mozilla/5.0"
172.19.0.4 - - [02/Oct/2026:22:09:24 ] "GET /admin/index.php HTTP/1.1" 404 40 "-" "Mozilla/5.0"
172.19.0.4 - - [02/Oct/2026:22:09:24 ] "GET /config.php HTTP/1.1" 404 40 "-" "Mozilla/5.0"
172.19.0.4 - - [02/Oct/2026:22:09:25 ] "GET /backup HTTP/1.1" 404 40 "-" "Mozilla/5.0"
172.19.0.4 - - [02/Oct/2026:22:09:25 ] "GET /backup.zip HTTP/1.1" 404 40 "-" "Mozilla/5.0"
172.19.0.4 - - [02/Oct/2026:22:09:25 ] "GET /phpmyadmin/ HTTP/1.1" 404 40 "-" "Mozilla/5.0"
172.19.0.4 - - [02/Oct/2026:22:09:25 ] "GET /wp-login.php HTTP/1.1" 404 40 "-" "Mozilla/5.0"
172.19.0.4 - - [02/Oct/2026:22:09:25 ] "GET /server-status HTTP/1.1" 404 40 "-" "Mozilla/5.0"
172.19.0.4 - - [02/Oct/2026:22:09:25 ] "GET /robots.txt HTTP/1.1" 404 40 "-" "Mozilla/5.0"
— end of run —
```

Screenshot: the web server log showing the same reconnaissance with status codes.

Network and host logs should agree, and here they do. That agreement is your first corroboration: the pcap shows the requests were sent, and the web log shows the server received and answered them.

Read the status codes, because they separate probing from success:

| Code | Meaning to you                                                   |
|------|------------------------------------------------------------------|
| 404  | The path does not exist — <b>probing</b>                         |
| 403  | It exists but is forbidden — <b>the attacker found something</b> |
| 200  | It exists and was served — <b>the attacker got something</b>     |
| 500  | The exploit broke the app — <b>possibly successful</b>           |

All ten reconnaissance paths returned 404 except one. /files/1.txt returned 200 — so a real path existed and served content. That single 200 is the finding in the reconnaissance phase, and it tells you the attacker now knows a real path on the host.

## The exploit attempts

```
grep -E 'passwd|/login|/upload' httpd.log
```

```
OSINT_SHOT
File Actions Edit View Help
$ grep -E 'passwd|/login|/upload' httpd.log
172.19.0.4 - - [02/Oct/2026:22:09:25 ] "GET /etc/passwd HTTP/1.1" 404 40 "-" "Mozilla/5.0"
172.19.0.4 - - [02/Oct/2026:22:09:25 ] "POST /login HTTP/1.1" 200 2 "-" "Mozilla/5.0"
172.19.0.4 - - [02/Oct/2026:22:09:25 ] "POST /upload HTTP/1.1" 403 9 "-" "Mozilla/5.0"
— end of run —
```

Screenshot: the exploit attempts — path traversal, SQL injection and an upload probe.



| Request         | Technique                  | Result        |
|-----------------|----------------------------|---------------|
| GET /etc/passwd | Path traversal             | 404 — refused |
| POST /login     | SQL injection in the form  | 200           |
| POST /upload    | Attempted web shell upload | 403 — refused |

POST /login returning 200 is the one to look at, and the one most often misread. A successful HTTP status does not mean the login succeeded — a form that re-renders on failure also returns 200. Go to the pcap and read the request body before you conclude anything:

```
tshark -r incident-window.pcap -Y 'http.request.method == "POST"' \
-T fields -e http.request.uri -e urlencoded-form.value
```

## The beacon

```
grep 'pixel.gif' httpd.log
```

```
OSINT_SHOT
File Actions Edit View Help
$ grep pixel.gif httpd.log
172.19.0.4 - - [02/Oct/2026:22:09:25 ] "POST /pixel.gif?id=beacon&c=1 HTTP/1.1" 403 9 "-" "okhttp/3.12.0"
172.19.0.4 - - [02/Oct/2026:22:09:26 ] "POST /pixel.gif?id=beacon&c=2 HTTP/1.1" 403 9 "-" "okhttp/3.12.0"
172.19.0.4 - - [02/Oct/2026:22:09:27 ] "POST /pixel.gif?id=beacon&c=3 HTTP/1.1" 403 9 "-" "okhttp/3.12.0"
172.19.0.4 - - [02/Oct/2026:22:09:28 ] "POST /pixel.gif?id=beacon&c=4 HTTP/1.1" 403 9 "-" "okhttp/3.12.0"
172.19.0.4 - - [02/Oct/2026:22:09:29 ] "POST /pixel.gif?id=beacon&c=5 HTTP/1.1" 403 9 "-" "okhttp/3.12.0"
— end of run —
```

Screenshot: five POST requests to a pixel.gif endpoint, the command-and-control beacon.

What the run above shows. Five POSTs to /pixel.gif?id=beacon&c=1 through c=5, at roughly one per second, all returning 403.

A tracking pixel is a GET, never a POST. The path is chosen to look like routine web analytics, and the disguise fails on four counts:

| Signal     | The anomaly                                        |
|------------|----------------------------------------------------|
| Method     | POST to something named .gif                       |
| User agent | <b>okhttp/3.12.0</b> — see below                   |
| Frequency  | One per second, five times, then stop              |
| Parameter  | c=1..5 — a sequence counter, which is C2 behaviour |

## The single strongest signal: the user agent

```
tshark -r incident-window.pcap -Y 'http.request.method' \
-T fields -e http.user_agent | sort | uniq -c
```

```
OSINT_SHOT
File Actions Edit View Help
$ tshark -Y http.request.method -T fields -e http.user_agent | sort | uniq -c
  13 Mozilla/5.0
   5 okhttp/3.12.0
— end of run —
```

Screenshot: the user agent count, separating 13 browser requests from 5 okhttp requests.

What the run above shows is the cleanest detection in the incident:



| User agent    | Count | What it is                            |
|---------------|-------|---------------------------------------|
| Mozilla/5.0   | 13    | The reconnaissance and exploit phases |
| okhttp/3.12.0 | 5     | The beacon                            |

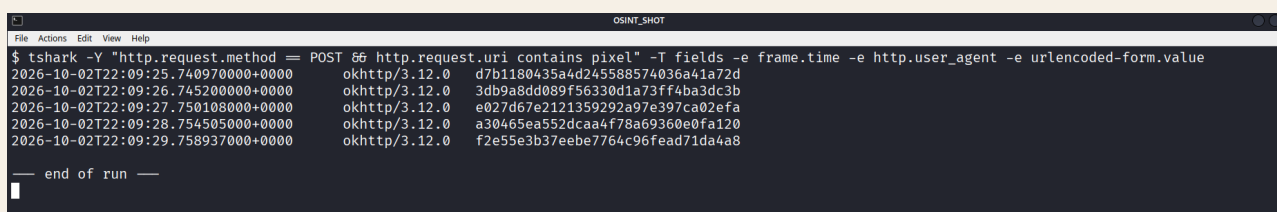
The attacker used two different clients and did not disguise the second one. okhttp is a legitimate Java/Android HTTP library. No browser sends it, and nothing in a normal office environment produces it either.

This one field separates the whole incident into two actors. Everything from Mozilla/5.0 is an interactive phase; everything from okhttp/3.12.0 is automated tooling. Sort by user agent early — it is often the cheapest discriminator in a web-log case.

Turn it into a detection: a request for a .gif with a POST, or any request carrying okhttp from a workstation subnet, is worth an alert.

## The exfiltration

```
tshark -r incident-window.pcap \  
-Y 'http.request.method == "POST" && http.request.uri contains "pixel"' \  
-T fields -e frame.time -e http.user_agent -e urlencoded-form.value
```



```
$ tshark -Y 'http.request.method == POST && http.request.uri contains pixel' -T fields -e frame.time -e http.user_agent -e urlencoded-form.value  
2026-10-02T22:09:25.740970000+0000 okhttp/3.12.0 d7b1180435a4d245588574036a41a72d  
2026-10-02T22:09:26.745200000+0000 okhttp/3.12.0 3db9a8dd089f56330d1a73ff4ba3dc3b  
2026-10-02T22:09:27.750108000+0000 okhttp/3.12.0 e027d67e2121359292a97e397ca02efa  
2026-10-02T22:09:28.754505000+0000 okhttp/3.12.0 a30465ea552dcaa4f78a69360e0fa120  
2026-10-02T22:09:29.758937000+0000 okhttp/3.12.0 f2e55e3b37eebe7764c96fead71da4a8  
  
— end of run —
```

*Screenshot: the beacon POST bodies, each carrying a 32-character hex session token.*

What the run above shows — and this is the finding that matters:

```
2026-10-02T22:09:25.740970000+0000 okhttp/3.12.0 d7b1180435a4d245588574036a41a72d  
2026-10-02T22:09:26.745200000+0000 okhttp/3.12.0 3db9a8dd089f56330d1a73ff4ba3dc3b  
2026-10-02T22:09:27.750108000+0000 okhttp/3.12.0 e027d67e2121359292a97e397ca02efa  
2026-10-02T22:09:28.754505000+0000 okhttp/3.12.0 a30465ea552dcaa4f78a69360e0fa120  
2026-10-02T22:09:29.758937000+0000 okhttp/3.12.0 f2e55e3b37eebe7764c96fead71da4a8
```

Each POST carries a 32-character hexadecimal value in its body. The pattern — a random-looking token, one per beacon, sent to a fixed endpoint — is exfiltration.

This changes the incident classification. Before this, the attacker had only attempted things and been refused. Here, data left the host, even though every response was 403. A beacon that gets refused still tells the operator the implant is alive, and still ships whatever is in the body.

Check whether the 403 actually stopped the data. The 403 is the server refusing to serve a response. The request body still arrived, and if the endpoint logged it or a proxy archived it, the token is out. See Limitation L3.

## 6. Phase 4 — Host analysis: the auth log

### Count the attempts

```
grep -c 'Invalid user' auth.log
```

Seven Invalid user events. Now list what was tried:



```
grep -oE 'Invalid user [a-z0-9]+' auth.log | awk '{print $3}' | sort -u
```

```
OSINT_SHOT
File Actions Edit View Help
$ grep -oE "Invalid user [a-z0-9]+" auth.log | awk "{print \$3}" | sort -u
admin123
billy
mysql
operator
oracle
postgres
test
$ grep -c "Invalid user" auth.log
7
— end of run —
```

Screenshot: the distinct usernames used in the brute force.

Nine usernames were targeted in total, and the count matters — confirm it against the evidence rather than trusting a figure in a guide, including this one:

```
grep '172.19.0.4' auth.log | grep -oE '(Invalid|authenticating) user [a-z0-9]+' \
| awk '{print $3}' | sort -u | wc -l
```

### The nine usernames

root, admin, test

oracle, mysql, postgres

billy, admin123, operator

Read the list as a wordlist, not as names. This is a default-credentials dictionary: database service accounts, a common admin variant, and a personal name. The presence of billy alongside oracle and postgres means the attacker had context — that username is not on any generic top-100 list.

## The distinction that matters most

```
grep 'authenticating user' auth.log
```

```
OSINT_SHOT
File Actions Edit View Help
$ grep "authenticating user" auth.log
Connection closed by authenticating user root 172.19.0.4 port 56298 [preauth]
Connection closed by authenticating user admin 172.19.0.4 port 56306 [preauth]
# authenticating = account EXISTS, password wrong
— end of run —
```

Screenshot: the authenticating-user events, showing that root and admin exist on the host.

This log separates two very different events, and the difference is intelligence:

| Message                                    | Meaning                                       | Count here |
|--------------------------------------------|-----------------------------------------------|------------|
| Invalid user X                             | The account does not exist                    | 7          |
| Connection closed by authenticating user X | The account EXISTS and the password was wrong | 2          |

The log tells you which usernames are real accounts on the target. root and admin produced authenticating failures — they exist. Everything else produced invalid user failures — they do not.

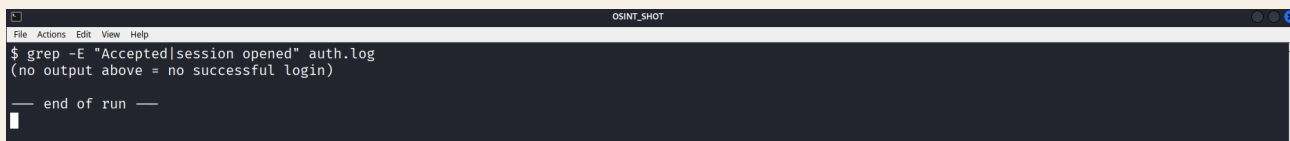


From an attacker's point of view this is the whole prize. A brute force against non-existent users is noise. A brute force against a **confirmed** account is a targeted attack, and the log has just told you which accounts to worry about.

For the responder, this is what to prioritise:

1. root and admin were targeted and **confirmed** to exist — check their authentication records, not just the failures.
2. The password admin123 is set on admin in this lab. One of those attempts could have had the right password by design. Go and check whether any attempt succeeded:

```
grep -E 'Accepted|session opened' auth.log
```



*Screenshot: the successful-login check returning nothing, which is the finding.*

No output is the good news, and it is still a finding. State it explicitly: the check was run and returned nothing, so no login succeeded during the incident window.

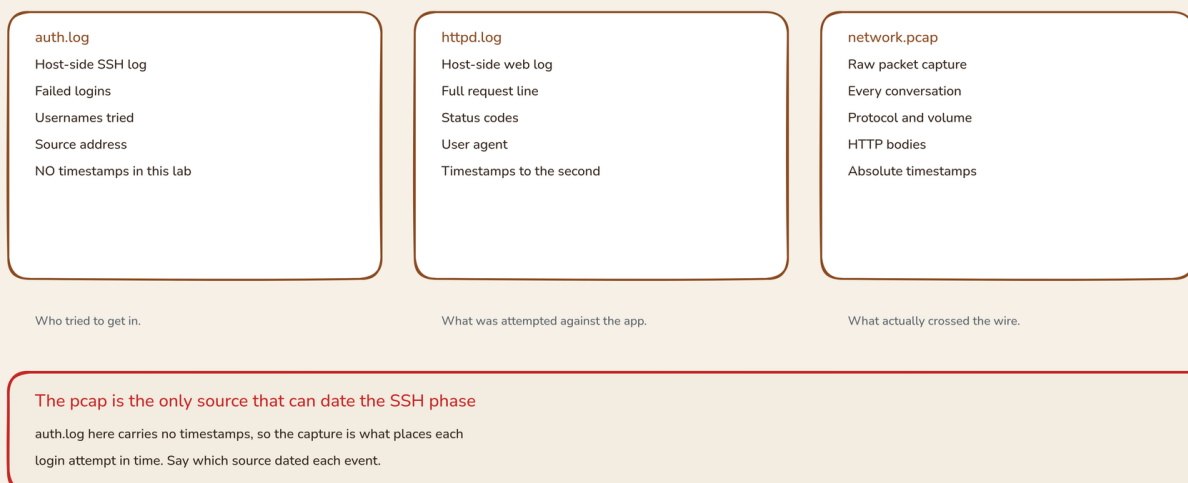
Never report "there was a brute force". Report which accounts exist, which were targeted, and whether any attempt succeeded. Those are three different facts.

## 7. Correlating into a timeline

Three sources, one sequence. This is the deliverable.

### Three sources, one story

Each source alone is ambiguous. Together they reconstruct the incident.



*Figure 4: The three evidence sources and what each one can and cannot tell you.*

| Time (UTC) | Source | Event                                                  | Confidence  |
|------------|--------|--------------------------------------------------------|-------------|
| 22:09:21   | pcap   | <b>First SSH connection</b> from 172.19.0.4 to port 22 | <b>High</b> |



| Time (UTC)  | Source           | Event                                                                           | Confidence |
|-------------|------------------|---------------------------------------------------------------------------------|------------|
| 22:09:21-24 | pcap + auth.log  | <b>9 SSH attempts across 9 usernames</b> , one every ~0.36 s                    | High       |
| 22:09:21    | auth.log         | root, admin produce <b>authenticating</b> failures — <b>both accounts exist</b> | High       |
| 22:09:24-25 | pcap + httpd.log | <b>10 reconnaissance paths</b> ; 9 return 404, /files/1.txt returns 200         | High       |
| 22:09:25    | pcap + httpd.log | Path traversal 404; SQLi POST /login 200; upload POST /upload 403               | High       |
| 22:09:25-29 | pcap + httpd.log | <b>5 beacon POSTs</b> , okhttp/3.12.0, exfiltrating 32-hex tokens               | High       |
| 22:09:29    | pcap             | Last request. <b>Attack complete.</b>                                           | High       |
| —           | auth.log         | <b>No Accepted entry: no login succeeded</b>                                    | High       |

Order the sources by what they can prove. The pcap dates everything. The host logs identify the users and the paths. Neither alone reconstructs the incident.

State your confidence per row. "**High**" here means observed directly in two independent sources. A single-source row would be "**Medium**" at best.

## What the timeline tells you that the logs do not

The whole incident took about eight seconds — 22:09:21 to 22:09:29, from the first SSH connection to the last beacon.

That is not a human. A person cannot attempt nine logins, scan ten paths, fire three exploits and beacon five times in eight seconds. This was automated end to end, and the timeline proves it without needing any other evidence.

The SSH pacing makes it sharper still. Nine sessions at roughly 0.36-second intervals is a fixed-rate loop, not a person typing. A constant inter-arrival gap is machine cadence, and it is worth calling out as such in the report.

## 8. Containment, eradication, recovery

### Contain, and contain properly

| Action                                                 | Right or wrong                                  | Why                                         |
|--------------------------------------------------------|-------------------------------------------------|---------------------------------------------|
| Disconnect the network cable / apply an isolation rule | <b>Right</b>                                    | Stops the spread, preserves the host        |
| Power the machine off                                  | <b>Usually wrong first</b>                      | Destroys RAM, processes, connections        |
| Delete the webshell from the victim                    | <b>Too early</b>                                | Destroys evidence, and you may miss others  |
| Rebuild from a clean image                             | <b>Right, once evidence is captured</b>         | Patching a compromised host is not reliable |
| Reset every password                                   | <b>Right, after you know which were exposed</b> | Blanket resets hide which account mattered  |

Isolate the host, then collect, then clean. In that order, and the collection step is the one people skip under pressure.



## Eradicate by rebuilding, not by cleaning

A compromised host is not trustworthy after cleaning. You cannot enumerate everything an attacker may have left, and you cannot prove a negative. Rebuild from a known-good image, then restore only data you have verified.

Before you rebuild, capture:

- Memory (if the host is still running)
- The disk, or at least the logs and web root
- The list of accounts and their state
- What the host was connected to, from the pcap and its own logs

## Recover, and watch for the return

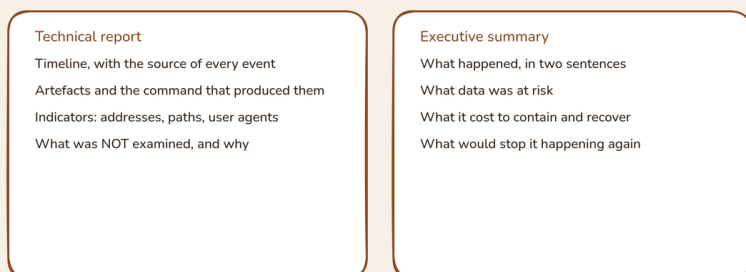
The beacon is the thing to watch. The attacker believes they have a working implant. If you clean the host and restore it, a repeat of okhttp traffic means an implant you missed.

Keep the detections live for weeks, not days. Attackers return to hosts they believe are still compromised.

## 9. Reporting

### Two audiences, one incident

The technical report and the executive summary answer different questions.



### Report the negatives

'The SSH phase could not be dated from auth.log, which carries no timestamps; the pcap was used instead.' That sentence is a finding.

An analysis nobody can reproduce is not intelligence.

Figure 5: Two audiences, one incident.



## The structure

```
## Incident Report – <reference>

## Executive summary
Two sentences for someone who will read nothing else.

## What happened
The timeline, in plain language.

## Evidence and integrity
The three sources, their SHA-256 at collection, and confirmation nothing changed.

## Findings
One per finding, with source and confidence.

## Indicators of compromise
Addresses, paths, user agents, tokens.

## What was NOT determined
Method limitations, stated plainly.

## Recommendations
Technical controls, and what to fix first.
```

## The indicators from this incident

| Type         | Indicator                                               | Source                    |
|--------------|---------------------------------------------------------|---------------------------|
| Behaviour    | User agent okhttp/3.12.0 from a workstation             | pcap                      |
| Behaviour    | POST to a .gif path (no legitimate client does this)    | httpd.log, pcap           |
| Behaviour    | SSH sessions of ~0.16 s, 24 frames, repeated within 2 s | pcap                      |
| Network      | 172.19.0.4 — the attacker                               | pcap, auth.log, httpd.log |
| Host         | 7 Invalid user events, 9 usernames, in ~2 s             | auth.log                  |
| Host         | root and admin <b>confirmed to exist</b>                | auth.log                  |
| Application  | /admin, /backup.zip, /phpmyadmin/, /wp-login.php        | httpd.log                 |
| Application  | GET /etc/passwd, POST /login, POST /upload              | httpd.log                 |
| Application  | POST /pixel.gif?id=beacon&c;=N                          | httpd.log, pcap           |
| Exfiltration | 32-character hex tokens in POST bodies                  | pcap                      |

Report the behaviour indicators first. An IP address you can block today is a hosting provider you will forget tomorrow. okhttp from a workstation is a detection that survives the attacker changing address.

## Report the limitations

Every one of these is a real limitation of this investigation, and each belongs in the report:

| Limitation                 | Statement                                                                            |
|----------------------------|--------------------------------------------------------------------------------------|
| auth.log has no timestamps | The SSH phase was dated from the pcap, not the host log                              |
| HTTP 200 on /login         | Status 200 does not prove a successful injection; the body was checked, not the code |
| Beacon responses were 403  | The request reached the server; whether the token was stored is unverified           |



| Limitation              | Statement                                                    |
|-------------------------|--------------------------------------------------------------|
| No memory or disk image | Host compromise cannot be excluded by network evidence alone |

Stating these makes the report stronger, not weaker. A reader who knows what you did not examine can judge how much weight the rest carries.

#### Chain of custody, and the ACPO principles

This is what separates an investigation from an opinion.

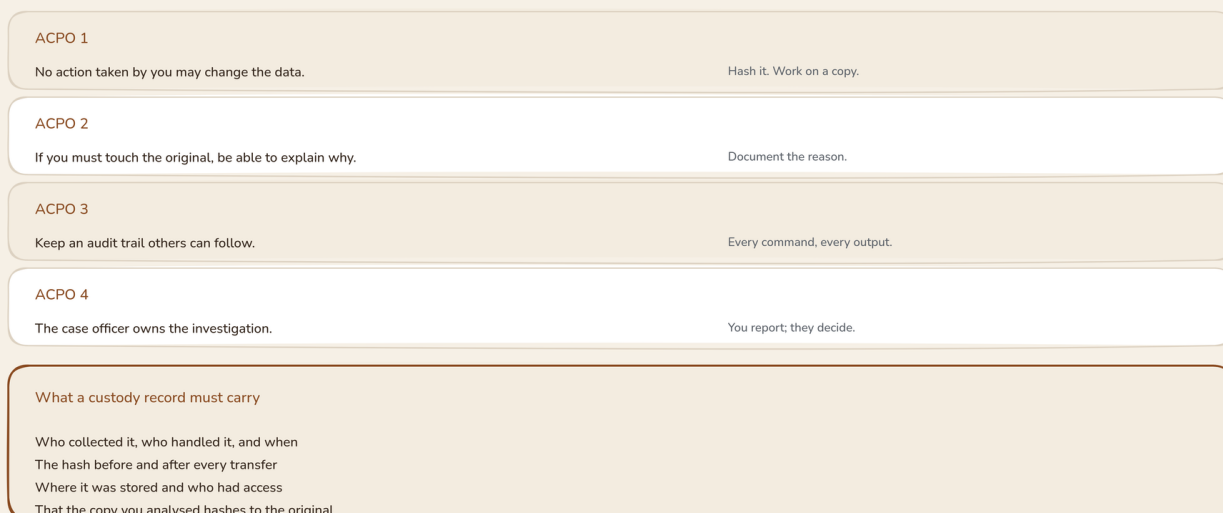


Figure 6: Chain of custody and the ACPO principles.

## 10. Tool reference

| Tool               | Phase   | Command                                             |
|--------------------|---------|-----------------------------------------------------|
| sha256sum          | Collect | sha256sum <evidence>                                |
| capinfos           | Triage  | capinfos capture.pcap                               |
| tshark             | Network | tshark -r c.pcap -Y '<filter>' -T fields -e <field> |
| tshark -z io,phs   | Network | tshark -r c.pcap -q -z io,phs                       |
| tshark -z conv,tcp | Network | tshark -r c.pcap -q -z conv,tcp                     |
| tshark -w          | Network | tshark -r in.pcap -Y '<filter>' -w out.pcap         |
| grep, awk, sort    | Host    | The whole of auth-log analysis                      |
| wc -l              | Triage  | wc -l *.log                                         |

You will notice the list is short. Most of what this guide does is tshark filters and text processing. Incident response is not tool-limited — it is limited by knowing what question to ask and which source can answer it.

### Tool versions in this lab



```
OSINT_SHOT
File Actions Edit View Help
TOOL      VERSION
tshark    4.6.6
capinfos  4.6.6
tcpdump   4.99.6
sha256sum 9.7
— end of run —
```

Screenshot: the versions of the tools used, as installed in the lab.

## 11. Troubleshooting

Real failures hit while writing this guide.

| Symptom                                             | Cause                                                        | Fix                                                     |
|-----------------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------|
| exec: "bash": executable file not found             | The victim is <b>Alpine</b>                                  | Use docker exec cpia-victim sh -c '...'                 |
| <b>Every request in the log appears twice</b>       | <b>docker start re-ran the attack</b>                        | Produce clean evidence; verify with the duplicate check |
| <b>Beacon count is double what the script sends</b> | <b>http.request.uri is sticky across continuation frames</b> | Count with http.request.method                          |
| auth.log has no timestamps                          | By design in this lab                                        | Date the phase from the pcap                            |
| Two source IPs in one log                           | Log carried across container runs                            | Restart the victim, or scope by address and time        |
| HTTP 200 on a login POST                            | Form re-renders on failure too                               | Read the body in the pcap, not the status               |
| SSH sessions all look identical                     | That is the brute-force signature                            | Count them; volume is the signal                        |

Two of these are the same mistake in different clothes: trusting a log to contain one incident's-worth of events. One came from a replayed attack, one from a filter artefact. Both produced exactly double the truth, and neither looked like an error.

The Alpine one is worth remembering. bash is not present on Alpine containers, and docker exec <c> bash fails with a message that sounds like a Docker problem but is a base image fact. Check the base image before reaching for bash — use sh, or ash.

## 12. Appendix — reproducing this guide

Every picture is a capture of a real command run against the live incident.

```
## 1. Run the real command, keeping the transcript
docker exec -u 0 kali-desktop bash -lc \
  'export PATH=$PATH:/usr/local/bin; <command>' \
  > evidence/$(date -u +%Y%m%dT%H%M%S)_<slug>.txt 2>&1

## 2. Screenshot the terminal that produced it
tools/term_shot.py images/<slug>.png <seconds> -- '<command>'

## 3. Crop the desktop chrome away so the text fills the page
tools/crop_shots.py images/*.png
```

The capture pitfalls the tool exists to handle, each of which cost real time:

- qterminal has no title flag; set the title with a printf escape and find the window with xdotool search --class qterminal.
- Resize the window before the command runs, or every line breaks mid-word.



- Use `kill -x qterminal`, not `-f`: the `-f` form matches the docker exec command line and kills the caller.
- Never nest a single quote in a command block passed to `docker exec bash -lc` inside a single-quoted string. It truncates the block silently and yields a garbled capture.

## The lab

| Container      | Role                                                                                               |
|----------------|----------------------------------------------------------------------------------------------------|
| cpia-attacker  | The scripted attacker — <b>starting it runs another attack</b>                                     |
| cpia-victim    | The weak target — holds <code>auth.log</code> , <code>httpd.log</code> , <code>network.pcap</code> |
| cpia-forensics | The analyst workspace — full forensic toolset                                                      |

```
cd /home/billy-forrest/labs/cpia-labs
docker start cpia-attacker           # generate a fresh incident
docker exec cpia-victim sh -c 'cat /var/log/auth.log' # host evidence
```

To guarantee one clean incident, restart the victim first (which clears `httpd.log`, truncates the pcap and restarts the capture), then run the attacker exactly once:

```
docker restart cpia-victim && sleep 15
docker exec cpia-victim sh -c ': > /var/log/auth.log' # sshd appends; restart does not clear it
docker start cpia-attacker && sleep 20
```

## What is in this folder

| Path      | Holds                                               |
|-----------|-----------------------------------------------------|
| evidence/ | The captured logs, pcap and command transcripts     |
| live/     | The scripted-attacker reference and incident window |
| figures/  | Excalidraw sources and rendered diagrams            |
| images/   | The terminal captures                               |
| reports/  | Incident reports produced from the evidence         |
| tools/    | Capture, crop, figure-build and PDF render tooling  |

Training at Forrest.one · Incident Response & Intrusion Analysis · Lab-generated training incident.

## Lab Setup Guide

### Building Your Own Security Lab

Training at Forrest.one · A Linux host with Kali and Windows, built to be broken

### How to use this guide

This is a build manual. It takes a Linux machine you own and turns it into a lab you can attack, infect, snapshot and reset without risking anything real.

Six things to know before you start:

1. A lab is defined by what it cannot do. The isolation is the product, not the OS install.



2. Every layer has one trap that fails silently. This guide names them.
3. VMs and containers solve different problems. Use both.
4. Never work on something you cannot replace. Overlays and images are how you get that.
5. Verify isolation. Do not assume it. One command per side.
6. Build order matters. Each layer needs the one below it.

## The rule that prevents every disaster in this guide

Never work on the thing you cannot replace.

One Linux host, three ways to run a lab

VMs carry their own kernel. Containers share the host kernel. Pick per workload.

### Virtual machines (KVM)

Windows 10 / 11 guest  
Kali desktop guest  
Any appliance OVA  
Own kernel, own disk  
Isolated libvirt bridge

### Containers (Docker)

Vulnerable web apps  
Kali tool container  
Incident / forensics labs  
Shares the host kernel  
--internal network = no egress

### Why both

Windows needs a real kernel  
Many services are easier as images  
Overlay disk = disposable VM run  
Image layers = disposable container  
Same host, same isolation idea

### The rule that prevents every disaster in this guide

Never work on the thing you cannot replace. Run every lab session on a copy

A VM run is a qcow2 OVERLAY with the clean install as its backing file.

A container run is a fresh container from the image.

Neither survives the session. That is the point.

Figure 1: VMs, containers, and the disposable-run principle that applies to both.

| What you build              | Cost to keep it clean | The reset            |
|-----------------------------|-----------------------|----------------------|
| A VM with a per-run overlay | One qemu-img create   | Delete the overlay   |
| A container from an image   | One docker run        | Remove the container |
| A bare-metal install        | Hours, and it is gone | There isn't one      |

Everything below is built around the third row being something you never have to do.

## 1. What you are building

One Linux host running two kinds of machine:

| Kind                         | Runs              | Carries                        | Use it for                               |
|------------------------------|-------------------|--------------------------------|------------------------------------------|
| <b>Virtual machine (KVM)</b> | Its own kernel    | Its own disk, network identity | Windows guests, Kali desktop, appliances |
| <b>Container (Docker)</b>    | The host's kernel | Its own filesystem and network | Vulnerable web apps, tooling, ranges     |



A VM is a computer. A container is a process with boundaries. Windows needs a real kernel, so it is a VM. A deliberately-vulnerable web app is a process, so it is a container.

You want both, and they share one idea: start from something clean, work in a copy, throw the copy away.

## 2. Prerequisites

### Hardware

| Requirement                                 | Why                                          | Check             |
|---------------------------------------------|----------------------------------------------|-------------------|
| 64-bit CPU with VT-x (Intel) or AMD-V (AMD) | Hardware virtualisation                      | See below         |
| Virtualisation enabled <b>in firmware</b>   | A CPU that supports it may still have it off | BIOS/UEFI setting |
| 16 GB RAM minimum, 32 GB comfortable        | Each guest reserves its memory               | free -h           |
| 100 GB free disk                            | Each VM is tens of GB                        | df -h             |
| SSD strongly preferred                      | A VM on a spinning disk is painful           |                   |

Check the CPU and the kernel module before installing anything:

```
. /etc/os-release; echo "$PRETTY_NAME"
uname -r
grep -cE 'vmx|svm' /proc/cpuinfo      # non-zero = the CPU supports it
```

```
OSINT_SHOT
File Actions Edit View Help
Ubuntu 24.04.5 LTS
7.0.0-34-generic

$ kvm-ok
INFO: /dev/kvm exists
KVM acceleration can be used

$ grep -cE vmx|svm /proc/cpuinfo
32

$ free -h
total        used        free      shared  buff/cache   available
Mem:        62Gi        16Gi       1.0Gi       344Mi       45Gi       45Gi

— end of run —
```

Screenshot: confirming the host OS, kernel, and CPU virtualisation support in the kernel.

32 matching processors is the CPU reporting virtualisation on each core.

```
kvm-ok
```

```
OSINT_SHOT
File Actions Edit View Help
Ubuntu 24.04.5 LTS
7.0.0-34-generic

$ kvm-ok
INFO: /dev/kvm exists
KVM acceleration can be used

$ grep -cE vmx|svm /proc/cpuinfo
32

$ free -h
total        used        free      shared  buff/cache   available
Mem:        62Gi        16Gi       1.0Gi       344Mi       45Gi       45Gi

— end of run —
```

Screenshot: kvm-ok reporting that KVM acceleration can be used.

If kvm-ok says acceleration cannot be used, go into the firmware and enable virtualisation. No amount of software fixes that — the /dev/kvm device cannot work without it, and QEMU will



silently fall back to emulation, which is dozens of times slower.

## Software

```
sudo apt-get update
sudo apt-get install -y \
    qemu-kvm qemu-utils \
    libvirt-daemon-system libvirt-clients \
    virtinst virt-manager \
    libguestfs-tools ovmf bridge-utils
sudo usermod -aG kvm,libvirt "$USER"
```

```
File Actions Edit View Help
OSINT_SHOT
# One command installs the whole hypervisor stack:
sudo apt-get install -y \
    qemu-kvm
    qemu-utils
    libvirt-daemon-system
    libvirt-clients
    virtinst
    virt-manager
    libguestfs-tools
    ovmf
    bridge-utils
sudo usermod -aG kvm,libvirt $USER
# Then LOG OUT and BACK IN. usermod does not affect the current session.
— end of run —
```

*Screenshot: the single hypervisor install command and the group membership step.*

That usermod line is not optional, and it is not instant. Group membership takes effect at the next login. Using the commands before logging out and back in produces permission errors that look like a broken install.

| Package               | Provides                                                     |
|-----------------------|--------------------------------------------------------------|
| qemu-kvm              | The hypervisor that actually runs guests                     |
| qemu-utils            | qemu-img — create, inspect and snapshot disks                |
| libvirt-daemon-system | The libvirtd service and the system socket                   |
| libvirt-clients       | virsh                                                        |
| virtinst              | virt-install — build a domain in one command                 |
| virt-manager          | The GUI, useful but not required                             |
| libguestfs-tools      | Read a <b>stopped</b> guest's disk: virt-copy-out, virt-diff |
| ovmf                  | UEFI firmware for guests that need it                        |
| bridge-utils          | Bridge inspection                                            |

## Verify the whole stack

```
virsh --version
virt-install --version
qemu-img --version | head -1
docker --version
```



```
OSINT_SHOT
File Actions Edit View Help
COMPONENT  VERSION
-----
OS          Ubuntu 24.04.5 LTS
kernel      7.0.0-34-generic
virsh       10.0.0
virt-install 4.1.0
qemu-img    8.2.2
Docker      29.1.3

— end of run —
```

Screenshot: the versions of virsh, virt-install, qemu-img and Docker on the build host.

### 3. The permissions layer — where builds fail silently

This section saves more time than any other in this guide. Both traps produce errors that point somewhere else entirely.

#### Trap 1: the groups a normal shell does not carry

```
id -nG
```

```
OSINT_SHOT
File Actions Edit View Help
$ id -nG
billy-forrest adm cdrom sudo dip plugdev users lpadmin libvirt ollama kvm

# kvm and libvirt must both appear above.

$ sg libvirt -c "id -nG" # per-command group, no logout needed
libvirt adm cdrom sudo dip plugdev users lpadmin ollama kvm billy-forrest

— end of run —
```

Screenshot: checking which groups the current shell actually carries.

A new shell may not carry kvm and libvirt even after usermod. That is why every virsh command in this guide is wrapped in sg:

```
sg libvirt -c 'virsh -c qemu:///system list --all'
```

sg regenerates the group set for one command, so you do not have to log out.

#### Trap 2: sg takes ONE group, and nesting drops the outer one

This is the single most expensive trap in the setup. sg replaces the supplementary group set; it does not add to it.

| Command                            | What you get                       |
|------------------------------------|------------------------------------|
| sg libvirt -c 'id -nG'             | libvirt present, kvm absent        |
| sg libvirt -c "sg kvm -c 'id -nG'" | <b>kvm present, libvirt GONE</b>   |
| sg libvirt,kvm -c '...'            | group 'libvirt,kvm' does not exist |

```
OSINT_SHOT
File Actions Edit View Help
# sg takes ONE group. Nesting it SILENTLY DROPS the outer group:

$ sg libvirt -c "id -nG"
  libvirt
  kvm
  ^ libvirt present, kvm ABSENT

$ sg libvirt -c "sg kvm -c id -nG"
  kvm
  libvirt
  ^ kvm present, libvirt GONE ← the trap

— end of run —
```

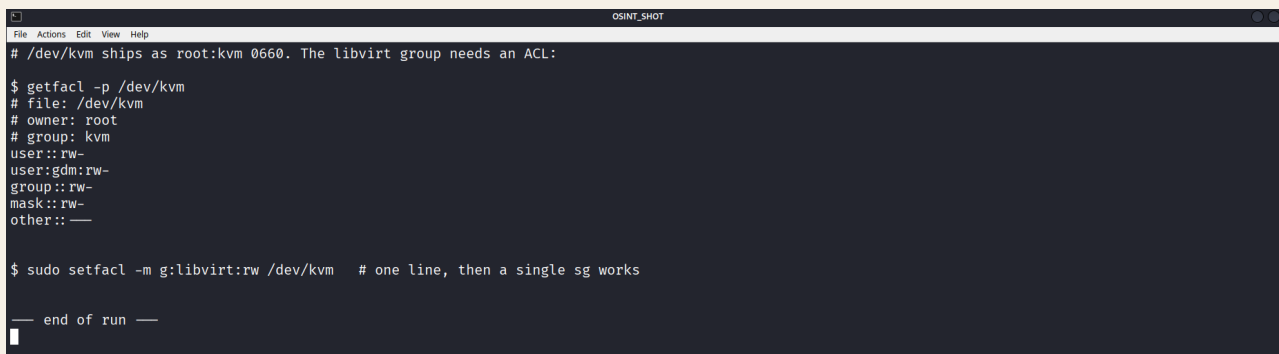
Screenshot: the nested-sg demonstration showing the outer group silently disappearing.



The nested form loses the outer group silently, so a command needing both the libvirt socket and /dev/kvm fails in a confusing way. The classic symptom: virt-install reports Network not found when the network exists — it reached the socket, but the daemon could not see the caller's group.

Fix it with an ACL on /dev/kvm, then use a single sg:

```
sudo setfacl -m g:libvirt:rw /dev/kvm
```



```
OSINT_SHOT
File Actions Edit View Help
# /dev/kvm ships as root:kvm 0660. The libvirt group needs an ACL:
$ getfacl -p /dev/kvm
# file: /dev/kvm
# owner: root
# group: kvm
user::rw-
user:gdm:rw-
group::rw-
mask::rw-
other::—
$ sudo setfacl -m g:libvirt:rw /dev/kvm # one line, then a single sg works
— end of run —
```

Screenshot: the /dev/kvm ACL, which lets one sg call open both the socket and the device.

/dev/kvm ships as root:kvm 0660. This one command gives the libvirt group access to it, is persistent, and removes the need for any group juggling.

### Trap 3: qemu:///system runs the guest as a different user

qemu:///system launches QEMU as user libvirt-qemu (uid 64055), not as you. A disk under your home directory fails:

```
Cannot access storage file '/home/you/vm/disk.qcow2' (as uid:64055, gid:993): Permission denied
```

virt-install warns about this before it fails — that warning is the real blocker, not noise. libvirt-qemu needs only traverse (x) on each parent directory, not read:

```
sudo setfacl -m u:libvirt-qemu:x /home/$USER /home/$USER/labs
```



```
OSINT_SHOT
File Actions Edit View Help
# qemu:///system runs the guest as uid 64055 (libvirt-qemu),
# which cannot even ENTER a 0700 home. It needs TRAVERSE (x) only.
$ namei -l /home/billy-forrest/Malware/sandbox/vm/maldev-win10-base.qcow2
f: /home/billy-forrest/Malware/sandbox/vm/maldev-win10-base.qcow2
drwxr-xr-x root root /
drwxr-xr-x root root home
drwxr-x— billy-forrest billy-forrest billy-forrest
drwx--x— billy-forrest billy-forrest Malware
drwxrwxr-x billy-forrest billy-forrest sandbox
drwxrwxr-x billy-forrest billy-forrest vm
-rw-rw-r-- libvirt-qemu kvm maldev-win10-base.qcow2
$ getfacl -p /home/billy-forrest | grep libvirt-qemu
user:libvirt-qemu:--x
— end of run —
```

Screenshot: namei showing each path component's permissions down to the disk image.

Traverse-only is strictly better than loosening the directory mode — QEMU can walk to the disk but cannot list or read anything else in your home. Diagnose the whole chain at once with namei -l <path>.

Verify the ACL landed, because setfacl can no-op silently:



```
getfacl -p /home/$USER | grep libvirt-qemu  
## expect: user:libvirt-qemu:--x
```

An empty grep means "not applied", not "applied differently".

## The alternative: put VM assets where they belong

If ACLs are unavailable or you would rather not repeat them per project, use libvirt's own tree and skip the problem entirely:

```
sudo mkdir -p /var/lib/libvirt/images/<vm>  
sudo mv base.qcow2 install.iso /var/lib/libvirt/images/<vm>/  
sudo chown -R libvirt-qemu:kvm /var/lib/libvirt/images/<vm>
```

This is the standard layout and it is what most guides assume. The ACL route buys a self-contained working directory at the cost of two host-level grants.

## 4. Building a Linux or Kali VM

### Get the ISO, and verify it

Kali publishes a checksum file next to the images. Use it — a torrent or mirror can hand you a truncated file that fails hours into an install.

```
curl -O https://cdimage.kali.org/current/kali-linux-2026.2-installer-amd64.iso  
curl -O https://cdimage.kali.org/current/SHA256SUMS  
sha256sum -c SHA256SUMS --ignore-missing 2>&1 | grep -i installer
```

Current release at the time of writing: Kali 2026.2, installer ISO 4,802,531,328 bytes, SHA-256 beginning 6dbefacc95e3b556.

Check any ISO cheaply before you build on it. file reports the ISO 9660 volume label and, crucially, whether the El Torito boot record is present:

```
file win10-ltsc-eval-x64.iso  
## -> ISO 9660 CD-ROM filesystem data 'CESE_X64FREE_EN-US_DV9' (bootable)
```

The (bootable) suffix is the El Torito confirmation, and the quoted label is the authoritative edition check. A 4.9 GB file whose label is not the edition you asked for is a wrong download, not a corrupt one.

### Build the domain

```
virt-install --name kali-vm --memory 4096 --vcpus 2 \  
--os-variant debian12 \  
--disk path=/var/lib/libvirt/images/kali-vm/base.qcow2,size=30,format=qcow2,bus=sata \  
--cdrom /var/lib/libvirt/images/kali-linux.iso \  
--network network=lab-isolated,model=e1000 \  
--machine q35 --cpu host-passthrough \  
--graphics vnc,listen=127.0.0.1,port=5901 --noautoconsole
```



```
OSINT_SHOT
File Actions Edit View Help
# Build a VM from an ISO: headless, disk-first boot, isolated network.

virt-install --name kali-vm --memory 4096 --vcpus 2 \
--os-variant debian12 \
--disk path=/var/lib/libvirt/images/kali-vm/base.qcow2,size=30,format=qcow2,bus=sata \
--cdrom /var/lib/libvirt/images/kali-linux.iso \
--network network=lab-isolated,model=e1000 \
--machine q35 --cpu host-passthrough \
--graphics vnc,listen=127.0.0.1,port=5901 --noautoconsole

# listen=127.0.0.1, NEVER 0.0.0.0: VNC has no authentication by default.

— end of run —
```

Screenshot: the virt-install command that builds a VM from an ISO, headless.

| Flag                              | Why it is set that way                                            |
|-----------------------------------|-------------------------------------------------------------------|
| --os-variant                      | Picks sane defaults for the guest type — always set it            |
| bus=sata                          | Works on Windows without a driver (see §7)                        |
| --network<br>network=lab-isolated | An isolating bridge, not NAT (see §5)                             |
| --graphics listen=127.0.0.1       | <b>Never 0.0.0.0</b> — VNC has no authentication by default       |
| --noautoconsole                   | Returns immediately; the guest keeps installing in the background |

Get the --graphics choice right on the first attempt. Binding a console to 0.0.0.0 exposes keyboard control of a machine that will run hostile code, with no password.

## Confirm it booted, and confirm which disk

```
sg libvirt -c 'virsh -c qemu:///system list --all'
sg libvirt -c 'virsh -c qemu:///system domblklist kali-vm'
```

```
OSINT_SHOT
File Actions Edit View Help
$ virsh list --all
Id Name State
1 maldev-win10 running

$ virsh domblklist maldev-win10
Target Source
sda /home/billy-forrest/Malware/sandbox/vm/maldev-win10-run.qcow2

— end of run —
```

Screenshot: virsh showing the domain running and which disk it has open.

--noautoconsole gives you no feedback, so prove the guest exists afterwards. Do not infer from CPU time — a VM can spin while sitting at a login screen.

## See the guest without a VNC client

```
sg libvirt -c 'virsh -c qemu:///system screenshot kali-vm --file /tmp/vm.png'
```

```
OSINT_SHOT
File Actions Edit View Help
# See the guest from the HOST: no credentials, no VNC client.

$ virsh screenshot maldev-win10 --file /tmp/win.ppm
Screenshot saved to /tmp/win-proof.ppm, with type of image/png
/tmp/win-proof.ppm: PNG image data, 1024 x 768, 8-bit/color RGB, non-interlaced

# This is the hypervisor framebuffer - proof the guest really booted.

— end of run —
```

Screenshot: capturing a guest's framebuffer from the host with virsh screenshot.



This is the fastest way to see what a guest is doing. It reads the hypervisor's framebuffer directly, so it needs no guest credentials, no network, and no viewer.

## 5. Networks — the isolation is the point

A lab machine that can reach the internet is a liability. It can exfiltrate, it can be attributed to you, and a sample that phones home has just told someone you are analysing it.

Isolation is a property you verify, not one you assume

A lab that can reach the internet is a liability, not a lab.

### Docker: no egress

docker network create --internal lab-net  
Internal=true, no gateway address  
Containers have no route off the host

### libvirt: no egress

Network XML with NO <forward> element  
No NAT rule, no forwarding  
Guest has no route off the host

### How to PROVE it, on each side

Docker: docker network inspect lab-net -> Internal: true, Gateway empty  
libvirt: virsh net-dumpxml lab-isolated | grep <forward> -> no output  
Guest: try to resolve a name and watch it fail (that failure is the pass)  
Never mark a lab ready on 'it came up'. Prove the isolation boundary.

### The failure mode this prevents

A sample or a scanner that can reach the internet will reach it, and the only difference between a lab and an incident is who owns the target.

Isolate first, then build. Retrofitting isolation to a live lab means rebooting everything.

Figure 2: Isolation is a property you verify, not one you assume.

## Three network shapes, and when to use each

The choice decides whether your lab can talk to the internet.

### NAT bridge

default libvirt net  
docker0 default bridge  
Guest CAN reach internet  
Use for: patching, installs  
NOT for hostile code

### Isolated bridge

No <forward> element  
Own subnet, DHCP on host  
No route off the host  
Use for: detonation, ranges

### Internal-only bridge

docker network --internal  
No gateway at all  
Containers see each other  
Use for: attacker/victim pairs

### Never put a detonation target on a NAT network

The whole observation you want is the C2 attempt FAILING. On a NAT network it succeeds, and you have run malware against the internet.

Give each range its own bridge so a reset of one cannot disturb another.

Name them for their purpose (lab-isolated, cpsa-net) so nobody has to guess later.

Figure 3: NAT, isolated and internal-only network shapes, and when to use each.



## Docker: an internal network has no egress

```
docker network create --internal lab-net
docker network inspect lab-net --format '{{.Internal}} {{range .IPAM.Config}}{{.Gateway}}{{end}}'
```



```
OSINT_SHOT
File Actions Edit View Help
$ docker network create --internal lab-net
$ docker network inspect lab-net
lab-net not present on this host (create it with the line above)
$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
e6a08d49079e        bridge              bridge              local
03464dc00dfb        cpia-net            bridge              local
67ffdb934936        cpsa-net            bridge              local
921f00c7e0da        host                host                local
28e27d87c077        kali-net            bridge              local
66e84e5c3715        labs_default        bridge              local
8168e37a38d4        maldev-net          bridge              local
0bb08f7053a0        none                null                local
— end of run —
```

Screenshot: creating an internal Docker network and inspecting it.

--internal installs no gateway and no NAT rule, so a container on it has no route off the host. Containers on the same internal network still see each other, which is what makes attacker/victim pairs work.

## libvirt: a network with no <forward> element

```
<network>
  <name>lab-isolated</name>
  <bridge name='virlab' stp='on' delay='0'/>
  <!-- Deliberately NO <forward> element: no NAT, no egress, no route off host. -->
  <ip address='192.168.200.1' netmask='255.255.255.0'>
    <dhcp><range start='192.168.200.100' end='192.168.200.200' /></dhcp>
  </ip>
</network>
```

```
virsh net-define lab-isolated.xml
virsh net-start lab-isolated
virsh net-autostart lab-isolated
```

The absence of <forward> is the entire mechanism. Add one back and the guest gets NAT to the internet.

## Prove the isolation — from inside, not from the config

Never mark a lab ready on "it came up". And do not prove isolation by reading the config alone — prove it from the guest or container itself.

```
## Docker: the network reports internal=true ...
docker network inspect lab-net --format '{{.Internal}} gateway={{range .IPAM.Config}}{{.Gateway}}{{end}}
}'
## ... and yet the container cannot get out:
docker exec victim sh -c 'timeout 5 nc -z 1.1.1.1 53 && echo REACHABLE || echo BLOCKED'
```

```
## libvirt: the guest bridge has no forward element at all
sg libvirt -c 'virsh -c qemu:///system net-dumpxml lab-isolated' | grep -c '<forward>'
```



```
OSINT_SHOT
File Actions Edit View Help
# PROVE the isolation from INSIDE, not from the config.
# 1. Docker: the network reports internal=true ...
maldev-net internal=true gateway=172.22.0.1
... and yet the container cannot get out:
internet: BLOCKED
DNS: no resolution
# 2. libvirt: the guest bridge has NO <forward> element ...
maldev-isolated <forward> elements = 0
... 0 means no NAT rule and no forwarding, so no route off the host.

— end of run —
```

Screenshot: proving isolation from inside the container and from the libvirt network XML.

Careful with the Docker case — this is a trap in itself. An internal network still reports a gateway address (in this lab, `internal=true` and `gateway=172.22.0.1`). The gateway exists on the bridge; what `--internal` removes is the iptables rule that forwards traffic out of it. So "there is no gateway" is the wrong test and would fail here.

| Check                    | Isolated result                     |
|--------------------------|-------------------------------------|
| libvirt <forward> count  | 0                                   |
| Container egress attempt | BLOCKED                             |
| Container DNS lookup     | no resolution                       |
| Docker gateway address   | still present — this proves nothing |

The only test that counts is the outbound attempt failing. A working lab looks broken at this step. Expect "No Internet" inside a guest. Say so up front, or it reads as a build fault and you will waste an afternoon "fixing" the isolation you just correctly built.

## 6. Snapshots and overlays — making a run disposable

A container has disposability for free. A VM does not. Anything a sample drops — a driver, a scheduled task, a registry key, a startup entry — is still there at the next boot and contaminates the following session.

Base + overlay: what makes a VM run disposable

Write the OS once. Every run branches from it and is thrown away after.

base.qcow2

The installed OS. Snapshotted 'clean' once.

never written to again

run.qcow2

qcow2 overlay, base as its backing file.

created per session

`qemu-img create -f qcow2 -F qcow2 -b base.qcow2 run.qcow2`

### The most dangerous state in the whole setup

A domain pointing at `base.qcow2` reports 'clean' and permanently records the session. `dombklist` is the only check that shows the truth: `sda -> ...-run.qcow2` is correct. `sda -> ...-base.qcow2` is contaminating.

Snapshot with the guest SHUT DOWN. QEMU holds the file lock while it runs.

Verify a snapshot by LISTING it, never by re-running the create command.

`qemu-img snapshot -c` prints nothing on success. A careless `'|| echo failed'` never fires.



Figure 4: The base plus overlay pair, and how a run is discarded.

## The pair

```
base.qcow2      the installed OS, written once, snapshotted "clean"
└─ run.qcow2    qcow2 overlay with base as its backing file, created per run
```

```
qemu-img create -f qcow2 -F qcow2 -b base.qcow2 run.qcow2
```

```
OSINT_SHOT
File Actions Edit View Help
# A per-run overlay takes the clean base as its backing file.
$ qemu-img info maldev-win10-run.qcow2
qemu-img: Could not open '/home/billy-forrest/Malware/sandbox/vm/maldev-win10-run.qcow2': Failed to get shared "write" lock
Is another process using the image [/home/billy-forrest/Malware/sandbox/vm/maldev-win10-run.qcow2]?

— end of run —
```

Screenshot: qemu-img info confirming the overlay's backing file.

```
qemu-img snapshot -c clean base.qcow2    # record the revert point
qemu-img snapshot -l base.qcow2          # LIST it (see the trap below)
```

```
OSINT_SHOT
File Actions Edit View Help
# Snapshot the clean base ONCE, with the guest SHUT DOWN.
virsh -c qemu:///system shutdown maldev-win10 # ACPI
qemu-img snapshot -c clean base.qcow2          # prints NOTHING on success

$ qemu-img snapshot -l maldev-win10-base.qcow2
Snapshot list:
ID      TAG      VM SIZE      DATE      VM CLOCK    ICOUNT
2       clean      0 B 2026-09-16 11:47:00 00:00:00.000 0

# Verify by LISTING. A verify step that re-runs create makes a SECOND snapshot.

— end of run —
```

Screenshot: listing the base image's snapshot to verify it exists.

## Snapshot with the guest SHUT DOWN

Two separate blockers stop a snapshot on a running VM, and they look like one problem:

```
qemu-img: Could not open 'base.qcow2': Permission denied
qemu-img: Could not open 'base.qcow2': Failed to get shared "write" lock
```

QEMU holds the image while the guest runs, and it owns the file while doing so. A clean ACPI shutdown releases both:

```
virsh -c qemu:///system shutdown kali-vm    # ACPI: the guest closes properly
## wait for virsh list --all to show "shut off" (~30s for Windows)
qemu-img snapshot -c clean base.qcow2
```

## Three traps in the snapshot step

| Trap                                                  | Why it bites                                                                       |
|-------------------------------------------------------|------------------------------------------------------------------------------------|
| qemu-img snapshot -c <b>prints nothing on success</b> | ...    echo failed never fires, so a failed snapshot looks identical to a good one |
| Verifying by <b>re-running</b> the create             | Silently makes a <b>second</b> snapshot under the same name                        |
| Snapshotting a <b>running</b> guest                   | Fails on the lock, and the error reads like a permissions problem                  |

Verify by listing, never by re-creating.



## The most dangerous state in the whole setup

A domain that reports clean while writing directly to the base disk.

```
sg libvirt -c 'virsh -c qemu:///system domblklist kali-vm'
```



```
# THE most dangerous state: a domain writing to its CLEAN BASE disk.
$ virsh domblklist maldev-win10
Target    Source
sda       /home/billy-forrest/Malware/sandbox/vm/maldev-win10-run.qcow2
sda -> .../run.qcow2    = disposable overlay, CORRECT
sda -> .../base.qcow2   = CONTAMINATING, stop and repair
— end of run —
```

Screenshot: domblklist showing the domain is on the overlay, not the base.

| domblklist shows      | Meaning                                                  |
|-----------------------|----------------------------------------------------------|
| sda -> .../run.qcow2  | <b>Correct</b> — a disposable run                        |
| sda -> .../base.qcow2 | <b>Contaminating</b> — stop and repair before detonating |

This is why "there is no overlay file" is not evidence of a clean state. With no overlay, every write goes permanently into the base. Ask the running domain which disk it has open, because that is the only check that reflects reality.

## Attaching the overlay reliably

virsh update-device --live cannot swap a root disk, and it fails silently in the most dangerous direction: the overlay file gets created, the command reports only a soft error, and the guest boots on the base.

Patch the <disk> element in the definition and redefine it instead. Two details decide whether it works:

1. Omit <backingStore/> entirely. Leaving it empty alongside an overlay <source> makes libvirt believe the overlay has no parent, and the start fails with a backing-file mismatch that reads like a corrupt image.
2. Keep <target> and <address> byte-identical. Change them and the guest's boot order shifts.

Then verify twice — config and live state:

```
virsh dumpxml kali-vm | grep '<source file'    # the definition points at the overlay
virsh domblklist kali-vm                     # the LIVE device is the overlay
```

Refuse to proceed if either disagrees.

## Reset: drop the overlay and verify it went

```
rm -f run.qcow2
[ -f run.qcow2 ] && echo "restore FAILED: overlay still present"
```

rm -f returns success even when nothing was deleted. A reset that trusts it can report "restored" while the sample's changes are still live. Point the definition back at the base before deleting the overlay — deleting first leaves the domain referencing a missing disk, and the next start fails with an unrelated-looking error.



## 7. Windows guests — four decisions

Windows does not ship virtio drivers, and that single fact drives three of these.

Windows guests: the four decisions that matter

Windows does not ship virtio drivers, which drives three of these.

|                                                                                                                                              |                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| Disk bus                                                                                                                                     | virtio needs a driver Windows lacks |
| NIC model                                                                                                                                    | virtio-net is equally driver-blind  |
| Boot order                                                                                                                                   | cdrom-first HALTS on first reboot   |
| Machine / CPU                                                                                                                                | modern chipset, full CPU features   |
| OOBE refuses to continue without a network                                                                                                   |                                     |
| On an isolated guest it never resolves. Use 'Domain join instead' at the account step for a local account, or Shift+F10 then OOBE\BYPASSNRO. |                                     |

Expect 'No Internet'. That is the isolation working, not a broken build.

Figure 5: The four Windows guest decisions, and the reason for each.

| Decision      | Use                                        | Because                                      |
|---------------|--------------------------------------------|----------------------------------------------|
| Disk bus      | bus=sata for the install                   | virtio needs a driver Windows does not have  |
| NIC model     | model=e1000 for the install                | virtio-net is equally driver-blind           |
| Boot order    | hd <b>before</b> cdrom, and detach the ISO | cdrom-first <b>halts</b> on the first reboot |
| Machine / CPU | --machine q35 --cpu host-passthrough       | Modern chipset, full CPU feature set         |

### The boot-order trap, and how it disguises itself

virt-install --boot cdrom,hd writes <boot dev='cdrom'> before <boot dev='hd'>. That is correct for the install and wrong the moment it ends: on the first post-install reboot the guest hits "Press any key to boot from CD", no key arrives, and it halts.

The symptom triad reads as a console problem, and it is not:

| You observe                                 | Reality                              |
|---------------------------------------------|--------------------------------------|
| virsh list --all shows shut off             | The guest halted itself              |
| A disk image of several GB                  | The install <b>did</b> succeed       |
| The console page loads, then cannot connect | Nothing is listening on the VNC port |

Fix both halves — reorder the boot devices and remove the ISO, so there is nothing but the OS to boot from:



```
sg libvirt -c 'virsh -c qemu:///system dumpxml kali-vm' > dom.xml
## edit: <boot dev='hd'/> before <boot dev='cdrom'/>; delete the <disk ...cdrom> element
sg libvirt -c 'virsh -c qemu:///system define dom.xml'
sg libvirt -c 'virsh -c qemu:///system domblklist kali-vm' # only the OS disk remains
```

## OOME will refuse to continue without a network

Windows setup will not proceed when it cannot reach the internet — and on an isolated guest it never will. Two ways past it, in order of preference:

1. "Domain join instead" at the account step — creates a local account, no Microsoft login, no telemetry account.
2. Shift+F10 for a command prompt, then run OOBE\BYPASSNRO, which reboots and adds an "I don't have internet" option.

Expect the guest to show "No Internet" / an unidentified network throughout. That is the isolation working. It is not a broken build.

## Confirm the guest really booted

```
sg libvirt -c 'virsh -c qemu:///system screenshot win10 --file /tmp/win.png'
```



```
File Actions Edit View Help
OSINT_SHOT
# See the guest from the HOST: no credentials, no VNC client.
$ virsh screenshot maldev-win10 --file /tmp/win.ppm
Screenshot saved to /tmp/win-proof.ppm, with type of image/png
/tmp/win-proof.ppm: PNG image data, 1024 x 768, 8-bit/color RGB, non-interlaced
# This is the hypervisor framebuffer - proof the guest really booted.
— end of run —
```

*Screenshot: the Windows guest's framebuffer captured from the host, showing it booted.*

This capture was taken from the host with no guest credentials and no VNC client — which is the whole argument for driving the hypervisor rather than the guest.

## 8. The container half of the lab

Not every lab needs a VM. A vulnerable web app, a tooling container, or an attacker/victim pair is much faster as a container.

### A single internal network

```
docker network create --internal lab-net
docker run -d --name victim --network lab-net alpine:3.20 sleep infinity
docker run -d --name attacker --network lab-net alpine:3.20 sleep infinity
```

| Docker                                          | Equivalent libvirt                  |
|-------------------------------------------------|-------------------------------------|
| docker network create --internal                | a network with no <forward> element |
| --cap-drop ALL --security-opt no-new-privileges | a hardened guest                    |
| docker rm -f <container>                        | deleting the overlay                |
| an image                                        | base.qcow2                          |



## Hardening a container, and what it costs you

```
docker run -d --name analyst \
  --network lab-net --hostname WS-01 \
  --cap-drop ALL --cap-add NET_RAW \
  --security-opt no-new-privileges \
  --pids-limit 512 --memory 4g --cpus 2 \
  --tmpfs /tmp:rw,nosuid,nodev,size=512m \
  image sleep infinity
```

--cap-drop ALL plus --security-opt no-new-privileges is strong, and it breaks tcpdump.

| Flag                             | Effect                                                                                                |
|----------------------------------|-------------------------------------------------------------------------------------------------------|
| --cap-drop ALL                   | Removes every capability from the container                                                           |
| --security-opt no-new-privileges | Forbids raising privileges back — <b>including via file capabilities</b>                              |
| Result                           | tcpdump exits Operation not permitted, because it needs a capability raise to apply its own file caps |

Do not "fix" this by dropping no-new-privileges. That is a genuine security loss. Capture at the syscall layer (strace) instead, which for malware work is the better signal anyway — it records the process name alongside the connection attempt.

## A cap that cannot be re-added

--cap-add NET\_RAW looks like it solves the capture problem and does not. With no-new-privileges in effect, CapEff stays zero and tcpdump still fails. The flag and the capability are in direct conflict, and the security flag should win.

## 9. Putting it together

### Stack from bare metal to lab

Install in this order. Each layer depends on the one below.

|                                                                                                                                     |                                        |                              |
|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|------------------------------|
| 1 Hardware + firmware                                                                                                               | VT-x / AMD-V enabled in the BIOS       | kvm-ok confirms it           |
| 2 Host OS                                                                                                                           | Ubuntu 24.04 LTS or Kali on bare metal | the machine you own          |
| 3 Virtualisation                                                                                                                    | qemu + libvirt + virt-install          | one apt command              |
| 4 Permissions                                                                                                                       | kvm and libvirt groups, plus two ACLs  | the step that fails silently |
| 5 Networks                                                                                                                          | isolated bridge, no <forward> element  | no egress off the host       |
| 6 Guest storage                                                                                                                     | base image + per-run overlay           | makes a run disposable       |
| 7 Guests                                                                                                                            | Windows, Kali, any appliance           | your lab machines            |
| 8 Container ranges                                                                                                                  | Docker + compose on --internal         | the fast half of the lab     |
| Layer 4 is where this guide saves you the most time<br>The group and ACL traps fail with errors that point somewhere else entirely. |                                        |                              |

Figure 6: The eight layers, in the order you install them.



| Layer              | Do this                                               | The trap                                |
|--------------------|-------------------------------------------------------|-----------------------------------------|
| 1 Hardware         | Enable VT-x / AMD-V in firmware                       | kvm-ok fails and you blame the OS       |
| 2 Host OS          | Ubuntu 24.04 or Kali, bare metal                      | —                                       |
| 3 Virtualisation   | The one apt-get install                               | usermod needs a fresh login             |
| 4 Permissions      | kvm + libvirt groups, /dev/kvm ACL, home traverse ACL | Nested sg drops a group <b>silently</b> |
| 5 Networks         | Isolated bridge, no <forward>; --internal for Docker  | A lab that reaches the internet         |
| 6 Storage          | Base + overlay, snapshot the base while shut down     | Writing to the base and not noticing    |
| 7 Guests           | Windows with SATA + e1000, hd before cdrom            | The install halts on first reboot       |
| 8 Container ranges | Compose on an internal network                        | tcpdump broken by the hardening         |

## The first-build cost is real

| Task                                 | Roughly                                     |
|--------------------------------------|---------------------------------------------|
| apt-get install the hypervisor stack | 2-5 minutes                                 |
| Windows install into a qcow2         | 20-40 minutes                               |
| Kali install from ISO                | 15-30 minutes                               |
| A container image pull               | 1-10 minutes                                |
| <b>Every build after that</b>        | <b>seconds</b> — overlays and cached images |

Nothing in this guide is slow the second time. That is the return on building it properly.

## 10. Troubleshooting

Every one of these was hit while building this lab.

| Symptom                                                     | Cause                                         | Fix                                                       |
|-------------------------------------------------------------|-----------------------------------------------|-----------------------------------------------------------|
| Network not found from virt-install, but the network exists | Nested sg dropped the libvirt group           | setfacl -m g:libvirt:rw /dev/kvm, then a <b>single</b> sg |
| Cannot access storage file ... (as uid:64055)               | libvirt-qemu cannot traverse your home        | setfacl -m u:libvirt-qemu:x on each parent dir            |
| Permission denied opening a qcow2 for snapshot              | The guest is still running                    | ACPI shutdown, then snapshot                              |
| Failed to get shared "write" lock                           | Same — QEMU holds the image                   | Stop the guest                                            |
| Guest halts on first reboot after install                   | cdrom before hd in boot order                 | Reorder and detach the ISO                                |
| Console page loads but will not connect                     | Nothing is listening on the VNC port          | The guest is not running — check virsh list               |
| qemu-img snapshot -c printed nothing, did it work?          | It prints nothing on success <b>by design</b> | Verify with qemu-img snapshot -l                          |
| Two snapshots with the same tag                             | A "verify" step that re-ran the create        | List; delete the duplicate with snapshot -d               |
| tcpdump: Operation not permitted in a container             | no-new-privileges blocks the capability raise | Capture with strace; keep the flag                        |



| Symptom                             | Cause                                         | Fix                                                             |
|-------------------------------------|-----------------------------------------------|-----------------------------------------------------------------|
| Windows OOBE will not continue      | No internet on an isolated guest              | <b>"Domain join instead"</b> , or Shift+F10 then OOBE\BYPASSNRO |
| Guest "clean" but changes persist   | The domain is on base.qcow2                   | dombklklist; reattach the overlay                               |
| supermin exited with error status 1 | /boot/vmlinuz-* is 0600 root:root, unreadable | ls -l /boot/vmlinuz-* then sudo chmod 644                       |

The supermin one deserves a note. It breaks virt-copy-out and virt-diff, and it is almost never the appliance's fault. Check /boot permissions before anything else — a reboot does not change file modes, so if that is the cause, rebooting costs you a full stop of every VM and container for nothing.

And when every extraction fails, that is tooling, not an empty guest. Report the distinction: one absent path means the sample wrote nothing there; all four failing means the appliance never built, and reporting "not present" for each would be a false negative.

## 11. Working with a stopped guest

You can read a guest's disk without booting it or knowing any credentials.

```
virt-copy-out -a run.qcow2 /Windows/Temp ./evidence/  
virt-diff -a base.qcow2 -A run.qcow2 > FILESYSTEM-DIFF.txt
```

| Tool              | Gives you                                                     | Needs            |
|-------------------|---------------------------------------------------------------|------------------|
| virt-copy-out     | Files from a stopped disk                                     | libguestfs-tools |
| virt-diff         | <b>Every file created, modified or deleted</b> since the base | nothing else     |
| virt-ls, virt-cat | Listings, single files                                        | nothing else     |

virt-diff is the dynamic-analysis workhorse. It needs no guest tooling, no credentials and no symbols, works on the stopped overlay, and covers paths a fixed collection list would miss.

```
= - 0777    8192 /path/changed    size or content changed  
+ - 0777     0 /path/created    present only in the overlay  
- - 0777     0 /path/deleted    present only in the base
```

Expect Windows' own telemetry to dominate a boot-only diff — Defender, DiagTrack, Search. That noise is the baseline, not a finding. Say which is which, or several hundred lines of ProgramData/Microsoft/... reads as contamination.

Hash what you collect. An artefact without a hash is not evidence:

```
find evidence -type f ! -name MANIFEST.txt -exec sha256sum {} \; | sort -k2 >> MANIFEST.txt
```

Record the absent paths too. That is the difference between "this path was empty" and "we never looked".

## 12. Build checklist

Run this before you trust the lab with anything hostile.



```
[ ] kvm-ok reports acceleration can be used
[ ] virsh --version, virt-install --version, qemu-img --version all respond
[ ] sg libvirt -c 'virsh -c qemu:///system list --all' works
[ ] getfacl /dev/kvm shows a libvirt entry
[ ] getfacl $HOME shows user:libvirt-qemu:--x
[ ] A guest boots and virsh screenshot shows it
[ ] dombldlist shows the OVERLAY, not the base
[ ] Docker internal network shows Internal=true and NO gateway
[ ] libvirt network XML contains NO <forward>
[ ] A guest/container CANNOT resolve an external name
[ ] The base image has a "clean" snapshot, taken with the guest shut down
[ ] Reset drops the overlay AND verifies it is gone
```

The isolation checks are the ones that matter. Everything above them makes the lab work; those make it safe.

## 13. Appendix — reproducing this guide

```
## Host evidence, captured as a real terminal transcript
docker exec -u 0 kali-desktop bash -lc '<command>' > evidence/<stamp>_<slug>.txt 2>&1

## Terminal screenshots
tools/term_shot.py images/<slug>.png <seconds> -- '<command>'
tools/crop_shots.py images/*.png

## A guest's screen, from the host, with no guest credentials
sg libvirt -c 'virsh -c qemu:///system screenshot <dom> --file /tmp/vm.ppm'
```

### What is in this folder

| Path      | Holds                                              |
|-----------|----------------------------------------------------|
| evidence/ | Command transcripts from the build host            |
| figures/  | Excalidraw sources and rendered diagrams           |
| images/   | Terminal captures                                  |
| scripts/  | The helper scripts referenced here                 |
| tools/    | Capture, crop, figure-build and PDF render tooling |

### Verified on this host

| Component            | Version                                                         |
|----------------------|-----------------------------------------------------------------|
| Host OS              | Ubuntu 24.04.5 LTS, kernel 7.0.0-34-generic                     |
| Architecture         | x86-64, 32 threads reporting VT-x                               |
| Memory               | 62 GiB                                                          |
| virsh                | 10.0.0                                                          |
| virt-install         | 4.1.0                                                           |
| qemu-img             | 8.2.2                                                           |
| Docker               | 29.1.3                                                          |
| Windows guest ISO    | CESE_X64FREE_EN-US_DV9 (Enterprise LTSC eval), 4.9 GB, bootable |
| Kali release checked | 2026.2, installer ISO 4,802,531,328 bytes                       |



A live Windows 10 guest was built, booted and screenshotted from the host while writing this guide. The overlay/dombklist check in §6 is the one that was performed against it.

Training at Forrest.one · Building Your Own Security Lab · Built and verified on the host it documents.